ABW Target System Design

The Agent Bestiary Workspace as it is intended to work once remediation is complete

Status: Target

Status	**Target.** Normative design, not implemented. No sentence in this document describes current behaviour unless it is tagged **[K]** with a `path:line` citation
Date	2026-11-06
Baseline	`main` @ `a3cea516`; branch `integrity/spec-36` @ `3268680b`. That branch adds the LLD, SPEC 36 (with R0.1 amended), the Lattice spec and the directive at their `docs/architecture/` paths, and the committed G01 fix and CI ratchet. Line anchors in **[K]** citations refer to the committed tree at `3268680b`
Audience	Senior C++ systems engineers who will hold implementers to it. Rust idioms introduced here are mapped in Appendix C
Key words	**MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **MAY** follow RFC 2119

Contents

0. Reading guide
1. System context and invariants
2. Physical architecture
3. The ledger crate (`abw-ledger`)
4. What an agent is, in the target
5. Claims and the Verification Lattice
6. Provenance (ΞPROV)
7. Evidence-based grounding
8. Gates as events
9. Money
10. Vertical slices
11. Loops and policy
12. Extension surfaces in the target
13. Security model
14. Migration from current state
15. Conformance test catalogue
16. Open decisions

- Appendix A: gap closure map
- Appendix B: divergence from SPEC 36
- Appendix C: Rust → C++ mental model
- Appendix D: DDL validation record

---

0. Reading guide

0.1 Authority

When sources conflict, the higher rank wins. This document never overrides a higher-ranked source. Where it had to choose, the choice is recorded in Appendix B or §16.

Rank	Document	Role
1	`docs/architecture/ABW_VERIFICATION_LATTICE_SPEC_v0.1.md` ("Lattice spec")	**The design.** Normative for claims, dimensions, verdicts, judge, routing, events and loops
2	`docs/architecture/ABW_CODING_AGENT_DIRECTIVE.md` ("directive")	**Rules and phasing.** §3 hard rules and §5 phase boundaries are invariants of the target. Its §4 dispositions decide scope
3	`docs/specs/SPEC_36_INTEGRITY_REMEDIATION.md` ("SPEC 36")	Detailed mechanisms. Used only where it agrees with ranks 1–2
4	`docs/ABW_LOW_LEVEL_DESIGN.md` ("LLD")	**Current state only.** Gap IDs G01–G33 and the fact base for kept components
5	The code	Ground truth for what the target keeps unchanged

The rank-1 and rank-2 files also exist in the main working tree under the names `docs/architecture/ABW Verification Lattice — Specification v0.1.md` and `docs/ABW — Coding Agent Directive Lattice as Target.md`. Their contents are byte-identical to the rank-1 and rank-2 files.

0.2 Legend

Tag	Meaning	Citation form
[S]	Taken from the Lattice spec	`[S §Section]`
[R]	Taken from the directive	`[R §n]` or `[R rule n]`
[M]	A SPEC 36 mechanism, adopted because it agrees with [S] and [R]	`[M Rn.m]`
[K]	Kept from current code, unchanged in the target	`path:line`
[P]	This document's proposal where all sources are silent. **Every [P] appears in §16 as `OD-nn`**	`[P OD-nn]`

Untagged normative sentences are derivations: they follow directly from tagged sentences in the same section.

0.3 Vocabulary rule

Under rule 6 of [R §3], the words **Presence, Liveness, Truth, Grounding and Binding** appear in this document only as the claim-level Lattice dimensions. The platform self-checks the code currently calls the "ladder" (`src/ladder.rs:65-112` [K: current names, to be renamed]) are called **`platform_health`**, with these rungs:

- `schema_present`
- `write_path_live`
- `rollup_consistent`
- `field_tool_sourceable`
- `input_port_match`

The current gate named `Grounding` (`src/gate_trust.rs:98`) becomes `grounding_enforcement`.

0.4 How to read the diagrams

Sequence diagrams are numbered (`autonumber`). Prose and tables refer to frames as **⟨n⟩**. Notes are not numbered. Each transaction boundary is drawn as a `BEGIN` note and a `COMMIT` message.

---

1. System context and invariants

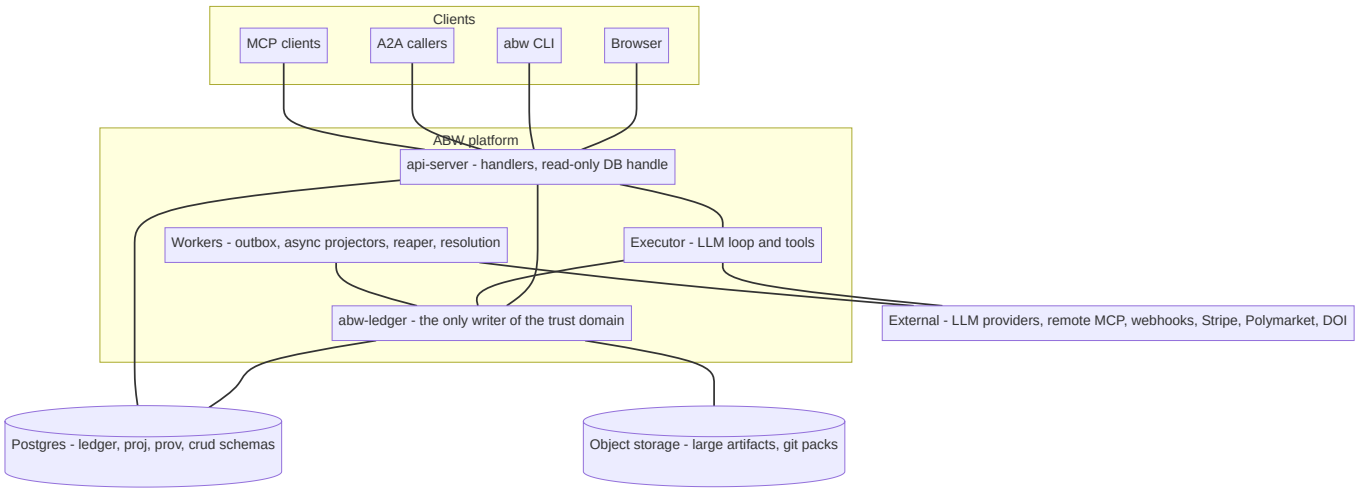
1.1 Context

ABW is a platform on which **agents** (versioned configurations run by a bounded tool-calling LLM loop) make **claims** about their own work. The target splits the system into two consistency domains [M §0]:

- **The trust domain** is event-sourced. It covers:
  - claims and their verification;
  - gate decisions;
  - money;
  - agent versions;
  - episodes;
  - provenance;
  - published artifacts (DPP passports).

  History is an append-only event log. Current state is a projection of that log.
- **Everything else stays CRUD.** That includes UI state, creatures, forecasts' working state, workspace files, ontology and memory. These tables are written by ordinary transactions under the rules in §14.5.

The trust domain can be *cited* by the CRUD domain. The reverse is not possible: a trust record MUST NOT depend on a mutable CRUD row for its meaning. Anything a trust record refers to is an artifact (content-addressed bytes) or another event.



1.2 Invariants

Each invariant names the Lattice spec principle (`[S P1]` … `[S P6]` , in the order they are listed in the Lattice spec "Principles") or section it implements, and the directive rule (`[R rule n]`). The **Phase** column gives the directive §5 phase whose acceptance gate first makes the invariant true. A conformance test for each invariant is listed in §15.

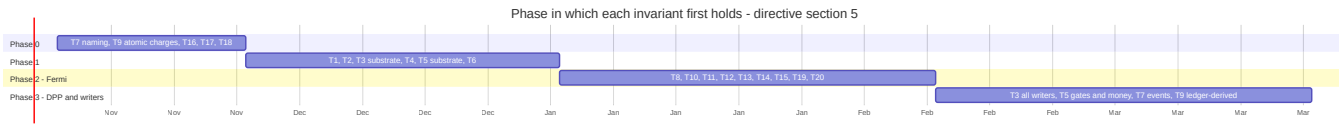
ID	Invariant	Implements	Phase
T1	**The write boundary is a crate.** `abw-ledger` is the only code that holds a write-capable database handle. Handlers receive `ReadDb`, a read-only handle. A handler that writes directly **does not compile**	[S §Purpose] (the CQRS command boundary); [R rule 1], [R §5 Phase 1]	1
T2	**Trust history is events.** Each stream has gap-free sequence numbers and is appended with expected-version compare-and-swap. Event envelopes are typed and versioned (`event_type`, `schema_version`). Artifacts are content-addressed	[S §Dimensions] (Presence and Liveness instruments); [R §5 Phase 1]	1
T3	**Projections update in the append transaction.** A projection row changes only in a transaction that appends the event justifying it. The database enforces this. Old mutable trust tables become projections and nothing else writes them	[S §Event schema]; [R §5 Phase 1, Phase 3]	1 (substrate) / 3 (all writers)
T4	**One canonical form, one hash format.** Canonical JSON is RFC 8785 (JCS). Every hash is `sha256:<64 lowercase hex>` over the canonical bytes	[R rule 11]; G22	1
T5	**A record is atomic with its effect.** Every decision, charge, version and verdict is committed in the same transaction as the effect it describes. There is no write-behind, no detached task and no fire-and-forget for these records	[S P4] (a human decision "is an event with the same standing"); [R rule 10]	1 (substrate) / 3 (gates, money)
T6	**The database enforces immutability.** Events, artifacts, provenance, policy versions and the immutable projections carry `BEFORE UPDATE OR DELETE` and `BEFORE TRUNCATE` triggers, and foreign keys into them are `ON DELETE RESTRICT`. `abw_owner` runs migrations. `abw_app` has `INSERT`, not `UPDATE`, `DELETE` or `TRUNCATE`, on the event and artifact tables	[R §5 Phase 1]; G15	1

ID	Invariant	Implements	Phase
T7	**A control refuses or it does not exist.** A check named gate, contract, binding or validation blocks the effect it guards. A check that only records is typed `Observation` and records `Undetermined` with a `reason_class`, never `Refused` or `Approved`	[R rule 4], [R rule 5]	0 (naming) / 3 (events)
T8	**Evidence is separate from verdict.** Evidence collection and verdict computation live in different modules, and no function both stamps provenance and amends a claim. The crate dependency graph enforces this	[S P1]; [R rule 7]	2
T9	**Money is a ledger.** Every wallet balance is the fold of its `wallet:` stream. Within each transaction the deltas sum to zero (double entry). Credit is reserved before work and settled in the episode-close transaction. Retries are idempotent	[R §4 G01, G04], [R §5 Phase 3]	0 (atomic charges) / 3 (ledger-derived)
T10	**Interpretive is never provenance.** LLM-judge output is typed `interpretive`. The database makes it unreachable in any Grounding walk and unusable as a source	[S §Judge "Interpretive, never provenance"]; [R rule 8]	2
T11	**Verdicts are reproducible.** A verdict vector is a pure function of `(claim, artifacts, stream heads, provenance graph, θ version)`. `theta_version` is part of the vector. Replay reproduces the vector bit for bit	[S P5], [S §Event schema]; [R §5 Phase 2]	2
T12	**The verdict vocabulary is closed.** Every dimension returns exactly one of `pass	fail	unsupported`		
T13	**Lattice, not ladder.** All five dimensions are evaluated on every evaluation, without short-circuit. Dependencies are declared in one table, not implied by order. One `lattice.evaluated` event carries the whole vector	[S P2], [S §Event schema]	2

ID	Invariant	Implements	Phase
T14	**A failure is a diagnosis, never terminal.** Every non-pass carries a failure class, evidence reference, expected, observed, suggested action and remaining budget. The worst outcome is `Parked` with owner notification. Nothing is silently dropped	[S P3], [S §Diagnosis]	2
T15	**Evidence, not verdict, at the routing boundary.** The lattice outputs a vector; a separately versioned routing policy picks the action. The judge report and the vector reach `route()` as separate typed inputs. The judge cannot raise any dimension	[S P1], [S §Judge "Rules" 3], [S §Routing]	2
T16	**One word, one meaning.** Presence, Liveness, Truth, Grounding and Binding name only Lattice dimensions	[R rule 6]	0
T17	**Tenancy on every read by ID.** Reading an object by opaque ID returns 404 unless the caller is authorised for that object	[R §4 G02]	0
T18	**No user-supplied URL reaches a private address.** Every outbound request to a tenant-supplied host goes through one guard that resolves the host, classifies the address, pins the IP and refuses redirects	[R §4 G03]	0
T19	**Binding is registration, scored at outcome.** A claim becomes scoreable when a `commitment.bound` event exists. Its Brier score is computed from that claim's own `p` when `outcome.observed` is appended	[S P6], [S §Loops]	2
T20	**The outer loop proposes; a human accepts.** No θ or routing policy changes without a human-accepted `policy.adjusted` event. A proposal that breaches the human-load ceiling is rejected	[S §Loops "two costs"]; [M R7.11]	2

Every invariant maps to a directive phase. A [P] mechanism that realises an invariant is listed in §16; the invariant itself always comes from [S] or [R].

1.3 Where each invariant becomes true



The bar lengths are illustrative. Each phase has a pass/fail acceptance gate, not a date. The phases are the directive's, not SPEC 36's (OD-38, Appendix B B5). The gates are restated as tests in §14.

---

2. Physical architecture

2.1 Processes

The target keeps one deployable binary, `api-server` (`Cargo.toml:262-263` [K]). It runs in two **roles**, selected by `ABW_ROLE`, and both roles are the same build artefact.

Role	Runs	Replica count	Why
`api`	Axum router, handlers, executor (agent runs inside the request task or a request-owned task), inline commands through `abw-ledger`	Exactly **1** until Phase 3 completes (§2.6)	Process-local state exists until Phase 3 (G21 is Deferred [R §4])
`worker`	Outbox consumers, async projectors, episode reaper, hold reaper, resolution sweeper, outcome observer, outer-loop proposer	0 or 1 until Phase 3. After Phase 3, N, because every job is claimed with `FOR UPDATE SKIP LOCKED`	Separates long and periodic work from request latency. Until Phase 3 the default is `ABW_ROLE=api+worker` in one process [P OD-01]

Other binaries are kept as they are: `fermi`, `agent-mcp-server`, and the operational scripts (LLD §1.3). The one change is that an operational script which writes trust state MUST link `abw-ledger` and go through its command API. No script holds a raw write pool.

2.2 Tokio runtime and blocking work

- **Runtime.** There is one `#[tokio::main]` multi-thread runtime (`src/api_server.rs:2312` [K]), with worker threads equal to the CPU count.

- **Blocking work** MUST run on `tokio::task::spawn_blocking`. The existing pattern for git reads is `src/handlers/beacons.rs:249` [K]. Blocking work includes:
  - libgit2 calls;
  - Ed25519 signing;
  - JCS canonicalisation of artifacts over 64 KiB;
  - the FPL Monte Carlo executor (`src/executor.rs:249-251` [K], seeded `StdRng`).
- **Task ownership.** No `tokio::spawn` may carry a trust effect (T5).
  - Every spawned task is either a request-owned `JoinHandle`, awaited before the response, or a worker loop owned by the `worker` role's supervisor.
  - The supervisor restarts a failed loop with backoff and records the failure as an `observation` (see §8.4).
  - `tests/claims/no_detached_trust_effects.rs` [M R2.7] holds an allowlist of `tokio::spawn` sites. Every entry carries a reason and is classified `non_trust`.

2.3 Connection pools

There are **two pools**, and **the only place a write-capable pool exists is inside `abw-ledger`** (T1).

Pool	Role	Size	Owner	Session settir
`ReadDb`	`abw_read`	20 max, 2 min	`AppState` (handlers, executor, readers)	`default_transaction_rea` (role-level, Appendix D ca
`LedgerPool`	`abw_app`	10 max, 2 min	Private field of `abw_ledger::Ledger`	`lock_timeout = 2s`, `idle_in_transaction_ses = 30s`, `statement_timeou`
Migration	`abw_owner`	1, at boot only	Migration runner in `abw-ledger::migrate`	Dropped before `axum::s`

Total connections are 30, against the current 25 (`src/api_server.rs:2330` [K]). The current settings `statement_cache_capacity(0)` and `DISCARD ALL` on connect (`src/api_server.rs:2327, 2337` [K]) are kept while a transaction-mode pooler fronts Postgres.

No LLM call, tool call, embedding call or HTTP request may run while a `LedgerPool` transaction is open. The episode pipeline (§4.5) therefore uses three short transactions instead of one long one. The rule is enforced in two ways:

- `idle_in_transaction_session_timeout` kills any violator at the database.
- `abw_ledger::Tx` is `!Send` across `.await` points that are not ledger calls, by design (§3.9).

2.4 Postgres layout and roles

There is one database with four schemas:

Schema	Contents	Written by
`ledger`	`stream_kinds`, `streams`, `events`, `event_types`, `event_type_retirements`, `artifacts`, `command_dedup`, `aggregate_snapshots`, `projector_checkpoints`, `outbox`, `policy_versions`	`abw_app` through `abw-ledger` only
`prov`	`nodes`, `edges`	`abw_app` through `abw-ledger` only
`proj`	Trust-domain projections: `wallets`, `credit_holds`, `episodes`, `agent_versions`, `agent_heads`, `claims`, `gate_decisions`, `field_evidence`, `external_sources`, `publication_releases`, `publication_tokens`, plus async read models	Inline projectors inside the append transaction, or async projectors through checkpoints. Money projections are written by a `SECURITY DEFINER` trigger, so `abw_app` has no `UPDATE` on them
`public`	The CRUD domain (about 110 tables)	`abw_app` through `abw_ledger::crud` (§14.5)

Roles are specified in §13.2.

2.5 Git and object storage

- **Workspace git repositories** stay libgit2 (`agent-bestiary/ontology/src/workspace_git.rs:40-56` [K]).
  - Until Phase 3 completes, they stay on a persistent mounted volume under `GIT_REPOS_PATH`. A boot sentinel write and read verifies the volume [M §13.3.3].
  - A git commit can never join a Postgres transaction. Every git write that a trust record depends on is therefore an **outbox job**, emitted in the transaction that decides it.
  - A trust record never depends on git state that could still change. A DPP release stores `source_commit` *and* the artifact SHAs of every file it read (§10.2).
- **Object storage** (S3-compatible) holds artifacts over `ARTIFACT_INLINE_MAX` (256 KiB) [M R3.2].
  - The upload precedes the referencing transaction.
  - The key is the SHA, so an upload is idempotent.
  - An orphaned object (uploaded, then the transaction rolled back) is harmless and is garbage-collected after 7 days by a worker that checks `ledger.artifacts`.

2.6 Single replica until Phase 3, then horizontal scale

G21 is **Defer**: "Single replica until Phase 3 completes" [R §4]. The target MUST therefore enforce single-replica operation mechanically until then.

1. **Boot fence.** Each `api`-role process takes `SELECT pg_try_advisory_lock(hashtext('abw-api-singleton'))` on a dedicated session connection and holds it for the life of the process. If the lock is already held, boot fails with an explicit error [M §13.3.2].
2. **Fence on the write path.** `abw-ledger` refuses to open `LedgerPool` unless the boot fence is held, so a second process cannot write even if it bypasses boot.

The path to horizontal scale. These are the Phase 3 exit criteria for lifting the fence:

Process-local state today (LLD §1.4)	Target home	Phase
LLM rate-limit windows (`src/api_server.rs:112` [K])	`proj.rate_windows` with `INSERT … ON CONFLICT DO UPDATE` token bucket per principal and class, or Redis if latency demands it [P OD-02]	3
Gate counters and queue (`src/gate_trust.rs:445-480`)	Deleted. Gate decisions are events (§8); counts are a projection	3
`write_accounting` counters	Deleted with the last direct write (Phase 3 ratchet reaches 0)	3
Liveness report `LATEST` (`src/liveness_trust.rs:914`)	`platform_health` runs emit `platform_health.observed` events; latest is a projection	3
Registry usage stats	Projection over `episode.closed`	2
Caches (`posterior_cache`, `projection_cache`, `export_consent`)	Remain per process as **caches only**: keyed by content SHA or with TTL, never authoritative. `export_consent` moves to a table (it is a grant, not a cache)	3
Git on local disk	Git on a shared volume, or bare repositories in object storage with a per-workspace advisory lock for writers	3

After Phase 3, the `api` role MAY run N replicas. Correctness does not depend on N: the ledger's compare-and-swap, `SKIP LOCKED` job claiming, and per-workspace advisory locks are all multi-writer safe.

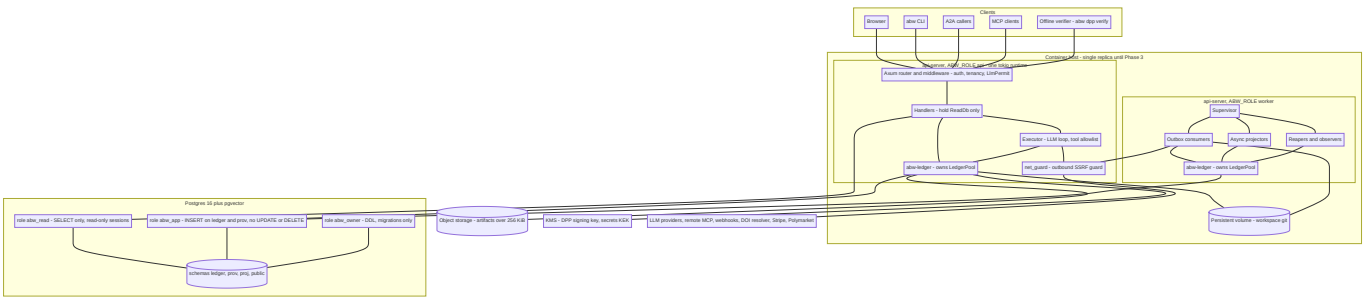
2.7 Background workers and their owners

Every worker runs in the `worker` role, under the supervisor. Every worker that writes trust state does so through `abw-ledger` commands.

Worker	Trigger	Reads
`outbox.<topic>` consumers	`ledger.outbox` rows, `FOR UPDATE SKIP LOCKED`	Outbox payload
Async projectors	Fence query (§3.5)	`ledger.events`
`episode_reaper`	Every 60 s	`proj.episodes WHERE status='running' AND opened_at < now() - EPISODE_MAX_RUNTIME`
`hold_reaper`	Every 60 s	`proj.credit_holds WHERE state='open' AND expires_at < now()`
`outcome_observer`	Per resolution source (Polymarket sweeper `src/handlers/polymarket.rs:1473` [K: kept as a source], manual resolution, DPP audit)	External state
`binding_deadline`	Every 5 min	`proj.claims WHERE state='Bound' AND deadline < now()`
`judge.run`	Outbox topic	Claim, task, artifact
`claim.evaluate`	Outbox topic	Claim stream, cited streams, evidence, provenance
`claim.correct`	Outbox topic	Diagnoses, original episode
`source.resolve`	Outbox topic	`proj.external_sources` row

Worker	Trigger	Reads
`git.commit`, `a2a.webhook`, `notify.*`	Outbox topics	Job payload
`outer_loop`	Daily	Outer-loop projection
`artifact_gc`	Daily	Object storage listing vs `ledger.artifacts`

2.8 Deployment diagram


Clients
Browser
abw CLI
ADA callers
MCP clients
Offline verifier - abw dpp verify
Container/VM - single replica until Phase 3
api-server, ABW_ROLE set - HTTP, MCP, webhooks
Axum router and middleware - auth, tenancy, LimPermit
Handlers - hold ReadDb only
Executor - LLM loop, tool allowlist
abw-ledger - owns LedgerPool
net_guard - outbound SSRF guard
api-server, ABW_ROLE worker
Supervisor
Outbox consumers
Async projectors
Reapers and observers
abw-ledger - owns LedgerPool
Persistent volume - workspace-git
Postgres 16 plus pgvector
role abw_read - SELECT only, read-only sessions
role abw_app - INSERT on ledger and prov, no UPDATE or DELETE
role abw_owner - DDL, migrations only
schemas ledger, prov, proj, public
Object storage - artifacts over 256 KiB
KMS - DPP signing key, secrets KEK
LLM providers, remote MCP, webhooks, DOI resolver, Stripe, Polymarket

3. The ledger crate (`abw-ledger`)

`abw-ledger` (path `crates/abw-ledger`) is the write boundary (T1) and the event store (T2). It owns:

- the only write-capable connection pool;
- the append algorithm;
- inline projections;
- the transactional outbox;
- JCS and hashing;
- the migration runner;
- the `crud` write module for the non-trust domain.

It depends on `sqlx` , `serde_json` , `sha2` and an RFC 8785 implementation [P OD-03]. It does **not** depend on the root `fermi` crate. Domain crates depend on it, not the other way round (§3.9).

3.1 DDL

All DDL below was executed against PostgreSQL 18.6 as written, and the negative cases were exercised under each role. The record is in Appendix D. Production targets Postgres 16 (LLD §1.1); nothing below needs a feature newer than 13 (`xid8` , `pg_current_xact_id`).

3.1.1 Immutability primitive

```
CREATE FUNCTION ledger.forbid_mutation() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION 'ABW-T6: % on %.% is forbidden (append-only)', TG_OP,
TG_TABLE_SCHEMA, TG_TABLE_NAME
    USING ERRCODE = 'insufficient_privilege';
END $$;
```

The trigger fires for **every** role, including the table owner, so it is the second layer behind grants. Grants stop `abw_app` ; the trigger stops `abw_owner` and anything with ownership. Superuser can still `ALTER TABLE … DISABLE TRIGGER` . That is an operator action outside the threat model, and it is detected by the nightly hash-chain audit (§3.10).

3.1.2 `stream_kinds` and `streams`

```
CREATE TABLE ledger.stream_kinds (
  kind         text PRIMARY KEY CHECK (kind ~ '^[a-z][a-z_]{1,31}$'),
  concurrency  text NOT NULL CHECK (concurrency IN ('exact','commutative')),
  description  text NOT NULL);

CREATE TABLE ledger.streams (
  stream_id   text PRIMARY KEY COLLATE "C"
              CHECK (stream_id ~ '^[a-z][a-z_]{1,31}:[A-Za-z0-9._@-]{1,128}$'),
  kind        text NOT NULL REFERENCES ledger.stream_kinds ON DELETE RESTRICT,
  tenant_id   text NOT NULL,
  head        bigint NOT NULL CHECK (head >= 0),
  created_at  timestamptz NOT NULL DEFAULT now(),
  CHECK (split_part(stream_id, ':', 1) = kind));
CREATE INDEX streams_tenant_kind ON ledger.streams (tenant_id, kind);

-- head may only move forward; identity columns never change; rows are never deleted
CREATE FUNCTION ledger.streams_head_monotonic() RETURNS trigger LANGUAGE plpgsql AS
$$
BEGIN
  IF TG_OP = 'DELETE' THEN RAISE EXCEPTION 'ABW-T6: streams rows are never deleted';
END IF;
  IF NEW.stream_id IS DISTINCT FROM OLD.stream_id OR NEW.kind IS DISTINCT FROM
OLD.kind
     OR NEW.tenant_id IS DISTINCT FROM OLD.tenant_id OR NEW.head <= OLD.head THEN
    RAISE EXCEPTION 'ABW-T2: streams.% may only advance head (% -> %)',
OLD.stream_id, OLD.head, NEW.head;
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER t_streams_monotonic BEFORE UPDATE OR DELETE ON ledger.streams
  FOR EACH ROW EXECUTE FUNCTION ledger.streams_head_monotonic();

-- gap-free: at COMMIT, every advance of head is matched by exactly that many events
CREATE FUNCTION ledger.streams_gap_free() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE lo bigint := CASE WHEN TG_OP = 'INSERT' THEN 0 ELSE OLD.head END; n bigint;
BEGIN
  SELECT count(*) INTO n FROM ledger.events
   WHERE stream_id = NEW.stream_id AND seq > lo AND seq <= NEW.head;
  IF n <> NEW.head - lo THEN
    RAISE EXCEPTION 'ABW-T2: stream % advanced % -> % but % events were written',
NEW.stream_id, lo, NEW.head, n;
  END IF;
  RETURN NULL;
END $$;
CREATE CONSTRAINT TRIGGER t_streams_gap_free AFTER INSERT OR UPDATE ON
ledger.streams
```

```
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
ledger.streams_gap_free();
```

- **Stream identity.** A stream ID is `<kind>:<id>`, for example `claim:0192d6c8-…`, `wallet:<uuid>`, `agent:<uuid>`, `episode:<uuid>`, `publication:<uuid>`, `policy:<uuid>`.
- `COLLATE "C"` makes ordering bytewise and locale-independent. Lock ordering (§3.2) depends on it.
- **Seeded stream kinds:**

Kind	Concurrency	Holds
`claim`	exact	One claim's lifecycle, all attempts (§5)
`episode`	exact	One agent run (§4.6)
`agent`	exact	One agent's versions (§4.2)
`wallet`	exact	One wallet's money (§9)
`publication`	exact	One DPP passport's releases (§10.2)
`policy`	exact	One policy's versions (§11.2)
`gate`	exact	Gate decisions that have no other subject stream, one stream per `(gate, tenant, day)` [POD-04]
`toolcall`	exact	Unused as a stream; tool invocations are events on `episode:` (kind reserved for remote-tool replay)
`telemetry`	commutative	Non-trust counters; `ExpectedVersion::Any` is permitted **only** here

`ExpectedVersion::Any` is rejected at compile time for every other kind (§3.8) [M R2.2].

3.1.3 `event_types`: the contract-versioning mechanism (G33)

```
CREATE TABLE ledger.event_types (
  event_type      text NOT NULL CHECK (event_type ~ '^[a-z][a-z_]*\.[a-z][a-z_]*$'),
  schema_version  int  NOT NULL CHECK (schema_version >= 1),
  stream_kind     text NOT NULL REFERENCES ledger.stream_kinds ON DELETE RESTRICT,
  epistemic       text NOT NULL CHECK (epistemic IN

('evidentiary','interpretive','structural','decision','observation')),
  json_schema     jsonb NOT NULL,
  json_schema_sha text NOT NULL CHECK (json_schema_sha ~ '^sha256:[0-9a-f]{64}$'),
  upcasts_from    int CHECK (upcasts_from IS NULL OR upcasts_from = schema_version -
1),
  introduced_by   text NOT NULL,                      -- migration file name
  PRIMARY KEY (event_type, schema_version));

CREATE TABLE ledger.event_type_retirements (
  event_type text NOT NULL, schema_version int NOT NULL,
  retired_by text NOT NULL, retired_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (event_type, schema_version),
  FOREIGN KEY (event_type, schema_version) REFERENCES ledger.event_types ON DELETE
RESTRICT);

CREATE TRIGGER t_types_immutable BEFORE UPDATE OR DELETE ON ledger.event_types
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
```

The `epistemic` column classifies every event type. `interpretive` is the class that §6 makes unreachable; `observation` is the class that §8.4 keeps distinct from controls. The table is append-only. A changed schema is a new `(event_type, schema_version)` row, never an edit (§12.6).

3.1.4 `events`

```
CREATE TABLE ledger.events (
  global_pos      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  stream_id       text   NOT NULL COLLATE "C" REFERENCES ledger.streams ON DELETE
RESTRICT,
  seq             bigint NOT NULL CHECK (seq >= 1),
  event_type      text   NOT NULL,
  schema_version  int    NOT NULL,
  txid            xid8   NOT NULL DEFAULT pg_current_xact_id(),
  payload         jsonb  NOT NULL,
  payload_sha     text   NOT NULL CHECK (payload_sha ~ '^sha256:[0-9a-f]{64}$'),
  command_id      uuid   NOT NULL,                    -- the command that produced
this event
  correlation_id  uuid   NOT NULL,                    -- request / claim correlation
  causation_pos   bigint REFERENCES ledger.events (global_pos) ON DELETE RESTRICT,
  actor           text   NOT NULL,
  actor_kind      text   NOT NULL CHECK (actor_kind IN
('human','agent','platform','tool')),
  recorded_at     timestamptz NOT NULL DEFAULT now(),
  UNIQUE (stream_id, seq),
  FOREIGN KEY (event_type, schema_version) REFERENCES ledger.event_types ON DELETE
RESTRICT);

CREATE INDEX events_fence       ON ledger.events (txid, global_pos);          --
async projector fence (3.5)
CREATE INDEX events_type_pos    ON ledger.events (event_type, global_pos);    --
per-type projector scans
CREATE INDEX events_correlation ON ledger.events (correlation_id);            --
request reconstruction
CREATE UNIQUE INDEX events_money_idem ON ledger.events (stream_id, (payload-
>>'idempotency_key'))
  WHERE payload ? 'idempotency_key';                                          -- T9
idempotency

-- seq may not run ahead of the head the CAS just set
CREATE FUNCTION ledger.events_seq_matches_head() RETURNS trigger LANGUAGE plpgsql AS
$$
DECLARE h bigint;
BEGIN
  SELECT head INTO h FROM ledger.streams WHERE stream_id = NEW.stream_id;
  IF NEW.seq > h THEN
    RAISE EXCEPTION 'ABW-T2: event seq % beyond stream head % for %', NEW.seq, h,
NEW.stream_id;
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER t_events_seq BEFORE INSERT ON ledger.events
  FOR EACH ROW EXECUTE FUNCTION ledger.events_seq_matches_head();
```

```
CREATE TRIGGER t_events_immutable   BEFORE UPDATE OR DELETE ON ledger.events
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
CREATE TRIGGER t_events_no_truncate BEFORE TRUNCATE ON ledger.events
  FOR EACH STATEMENT EXECUTE FUNCTION ledger.forbid_mutation();
```

Physical notes:

- **Row width.** `payload` is `jsonb`, TOASTed above about 2 KiB. Payloads MUST stay under 64 KiB [P OD-05]. Anything larger is an artifact referenced by SHA. Typical events are 300 B – 4 KiB.
- **Two orders.** `global_pos` is an identity value. It is allocated at `INSERT` time, **not** commit time, so it is not a commit order. That is why async readers use `(txid, global_pos)` and a snapshot fence, never `global_pos` alone (§3.5).
- **The one order Rust cares about** is the per-stream `seq`. It is gap-free (enforced by `t_streams_gap_free` at commit) and dense.
- **`payload_sha`** is `sha256:` over the JCS bytes of the *envelope-independent* payload (§3.6). `abw-ledger` computes it. The database cannot recompute JCS, so the nightly audit re-derives it in Rust.
- **No `jsonb` round-trip is trusted for hashing.** `jsonb` normalises numbers and reorders keys (validated: `{"b":1,"a":[1.0,"x"]}` reads back as `{"a": [1.0, "x"], "b": 1}`). The hash is therefore computed from the Rust value *before* serialisation into `jsonb`, and re-verification canonicalises the decoded `jsonb` value. JCS output is invariant under both transformations: key order is re-sorted, and `1.0` canonicalises to `1` either way.

3.1.5 `artifacts`

```
CREATE TABLE ledger.artifacts (
  sha              text PRIMARY KEY CHECK (sha ~ '^sha256:[0-9a-f]{64}$'),
  media_type       text NOT NULL,
  canonicalization text NOT NULL CHECK (canonicalization IN ('jcs-rfc8785','raw-
utf8','raw-bytes')),
  byte_len         bigint NOT NULL CHECK (byte_len >= 0),
  bytes            bytea,                                  -- inline when byte_len <=
262144
  storage_ref      text,                                   -- object key = sha,
otherwise
  created_pos      bigint NOT NULL,                        -- the event that first
cited it
  created_at       timestamptz NOT NULL DEFAULT now(),
  CHECK ((bytes IS NULL) <> (storage_ref IS NULL)),
  CHECK (bytes IS NULL OR octet_length(bytes) = byte_len),
  CHECK (bytes IS NULL OR sha = 'sha256:' || encode(sha256(bytes), 'hex')));
ALTER TABLE ledger.artifacts ADD FOREIGN KEY (created_pos) REFERENCES ledger.events
(global_pos)
  DEFERRABLE INITIALLY DEFERRED;
CREATE TRIGGER t_artifacts_immutable BEFORE UPDATE OR DELETE ON ledger.artifacts
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
CREATE TRIGGER t_artifacts_no_trunc  BEFORE TRUNCATE ON ledger.artifacts
  FOR EACH STATEMENT EXECUTE FUNCTION ledger.forbid_mutation();
```

- **The database recomputes the hash for inline bytes.** The last `CHECK` means a caller cannot store bytes under a SHA they do not hash to; Appendix D case 14 tests this. That makes Presence's instrument trustworthy without trusting Rust.
- **Large artifacts** are checked twice:
  - by `abw-ledger`, before the `INSERT`, by streaming the object back and hashing it;
  - nightly, by the audit worker.
- **Insert idiom.** Inserts are `INSERT … ON CONFLICT (sha) DO NOTHING`. Content addressing makes the store idempotent, and two writers storing the same bytes never conflict except on the unique-index entry, which costs microseconds.
- **Canonicalisation rules:**

Value	`canonicalization`
JSON documents (resolved cards, enforced outputs, packets, manifests)	`jcs-rfc8785`
Model text	`raw-utf8`
Images and binary attachments	`raw-bytes`

The same JSON value therefore always yields the same SHA regardless of producer.

3.1.6 `command_dedup`, `aggregate_snapshots`, `projector_checkpoints`, `outbox`

```
CREATE TABLE ledger.command_dedup (
  idempotency_key text PRIMARY KEY,                    -- client- or caller-derived
  command_type    text NOT NULL,
  request_sha     text NOT NULL CHECK (request_sha ~ '^sha256:[0-9a-f]{64}$'),
  first_pos       bigint REFERENCES ledger.events (global_pos) DEFERRABLE INITIALLY
DEFERRED,
  result          jsonb NOT NULL,                      -- the response returned the
first time
  created_at      timestamptz NOT NULL DEFAULT now());
CREATE TRIGGER t_dedup_immutable BEFORE UPDATE OR DELETE ON ledger.command_dedup
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();

CREATE TABLE ledger.aggregate_snapshots (
  stream_id         text NOT NULL COLLATE "C" REFERENCES ledger.streams ON DELETE
RESTRICT,
  seq               bigint NOT NULL,
  aggregate_type    text NOT NULL,
  aggregate_version int  NOT NULL,                     -- bump when apply() changes
  state             jsonb NOT NULL,
  state_sha         text NOT NULL,
  created_at        timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (stream_id, aggregate_type, aggregate_version, seq));

CREATE TABLE ledger.projector_checkpoints (
  projector    text PRIMARY KEY,
  fence_txid   xid8   NOT NULL DEFAULT '0'::xid8,
  fence_pos    bigint NOT NULL DEFAULT 0,
  lease_owner  text,
  lease_until  timestamptz,
  updated_at   timestamptz NOT NULL DEFAULT now());

CREATE TABLE ledger.outbox (
  id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  topic           text   NOT NULL,
  dedupe_key      text   NOT NULL UNIQUE,
  source_pos      bigint NOT NULL REFERENCES ledger.events (global_pos) ON DELETE
RESTRICT,
  payload         jsonb  NOT NULL,
  attempts        int    NOT NULL DEFAULT 0,
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  last_error      text,
  done_at         timestamptz,
  dead_at         timestamptz,
  CHECK (NOT (done_at IS NOT NULL AND dead_at IS NOT NULL)));
CREATE INDEX outbox_ready ON ledger.outbox (topic, next_attempt_at)
  WHERE done_at IS NULL AND dead_at IS NULL;
```

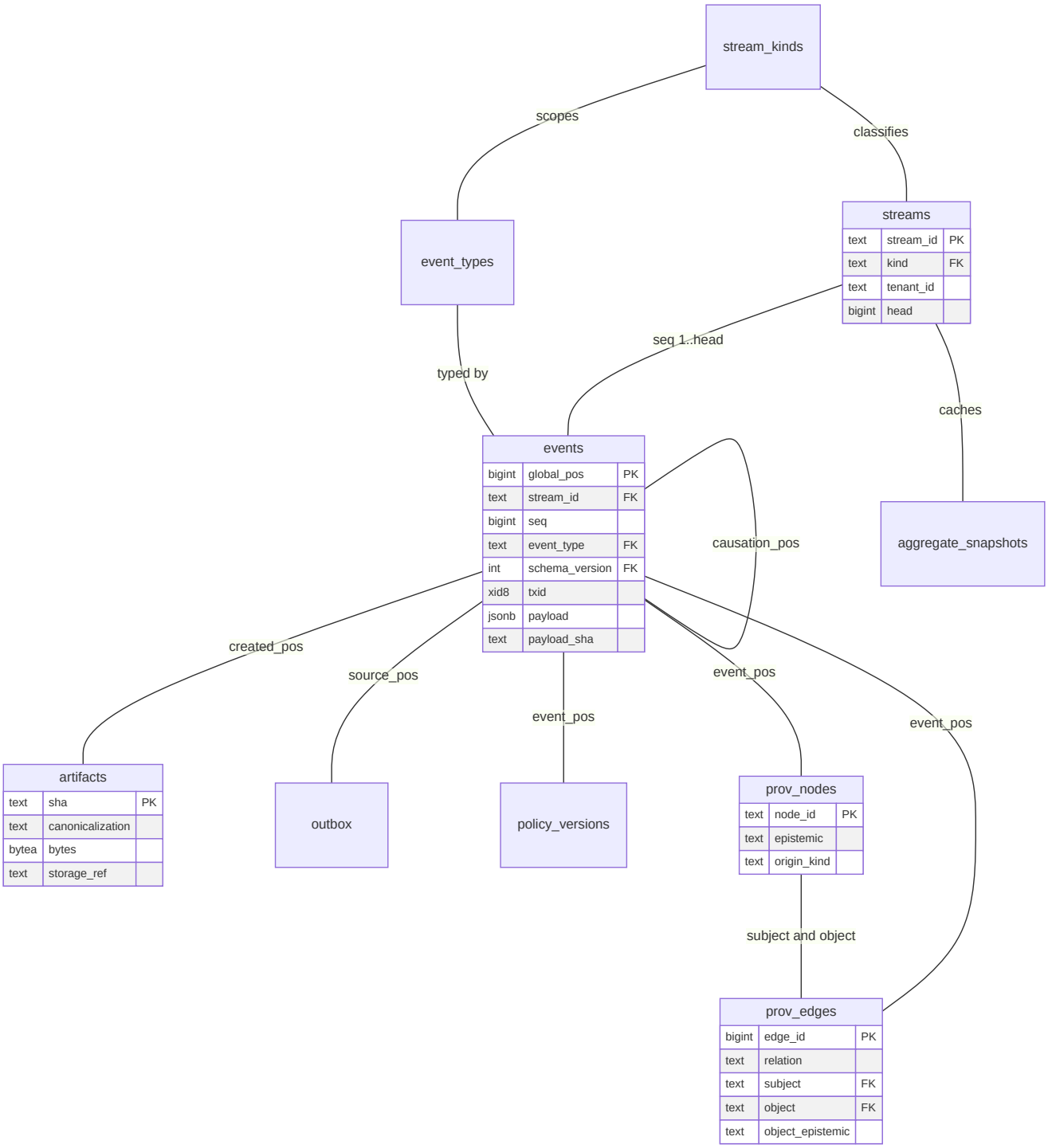
- **The outbox lives in the ledger schema and is written only by `abw-ledger`.** The directive freezes "outbox, queue or event infrastructure outside the ledger crate" [R §2 freeze 4]; this is the inside. Every outbox row references the event that caused it (`source_pos`), so a side effect without a recorded decision cannot exist. The outbox is **mutable by design**: it is a work queue, not history. Completion is recorded as an event on the causing stream (e.g. `a2a.webhook_delivered`), so the outbox row can be pruned after 30 days without losing history.
- **Snapshots are a cache.** They never participate in a verdict. `abw-admin verify-snapshots` re-folds the stream and compares `state_sha` [M R2.5].

3.1.7 `policy_versions` (θ)

```
CREATE TABLE ledger.policy_versions (
  policy_id      uuid NOT NULL,
  version        int  NOT NULL CHECK (version >= 1),
  policy_kind    text NOT NULL CHECK (policy_kind IN
('lattice','gate','evaluator')),
  tenant_id      text,                                -- NULL = platform default
  doc            jsonb NOT NULL,                      -- see 11.2 for the schema
  doc_sha        text NOT NULL CHECK (doc_sha ~ '^sha256:[0-9a-f]{64}$'),
  supersedes     int,
  accepted_by    text NOT NULL,                       -- human actor id
  rationale      text NOT NULL,
  active_from    timestamptz NOT NULL,
  event_pos      bigint NOT NULL REFERENCES ledger.events (global_pos) ON DELETE
RESTRICT,
  PRIMARY KEY (policy_id, version),
  CHECK (supersedes IS NULL OR supersedes = version - 1));
CREATE TRIGGER t_policy_immutable BEFORE UPDATE OR DELETE ON ledger.policy_versions
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
```

This table is the materialisation of the `policy:<policy_id>` stream. `version` equals the stream `seq` of the `policy.adjusted` event that created the row. The `theta_version` stamped into every verdict vector is the string `"<policy_id>@<version>"`, plus `doc_sha` in the vector's inputs. Replay loads the document by key, not by "latest" (T11).

3.1.8 ER view



3.2 The append algorithm

Isolation level: `READ COMMITTED` for command transactions, with explicit row locks [P OD-06]. Here is why:

- The compare-and-swap (`UPDATE … WHERE head = $expected`) is a **guarded single-row update**. Under `READ COMMITTED` , a concurrent writer blocks on the row lock. When it wakes, Postgres re-evaluates the `WHERE` against the *new* row version (EvalPlanQual). The loser sees `head ≠ expected` and updates 0 rows: a clean, typed conflict with no serialisation error.
- Under `REPEATABLE READ` the loser gets SQLSTATE `40001` instead (validated, Appendix D case 20). That is also correct, but it is indistinguishable from other serialisation failures.
- **Reads that a decision depends on** are protected by locks, not by isolation level: the aggregate fold, the cited stream heads, and the wallet balance. `abw-ledger` takes `FOR SHARE` on every stream it reads for a decision and `FOR NO KEY UPDATE` on every stream it appends to, in **one ordered pass** (§3.2.2).

3.2.1 Step-by-step

`Ledger::dispatch(cmd)` for a command `C` that appends to write set `W` (one or more streams) after reading set `R` :

Step	SQL	Locks taken	Notes
0	*(no SQL)* `C::validate()`	—	Pure: schema and invariants of the command alone. Rejection returns a typed `Rejection` before any connection is acquired
1	`BEGIN ISOLATION LEVEL READ COMMITTED`	—	Connection from `LedgerPool`
2	`INSERT INTO ledger.command_dedup (idempotency_key, …) VALUES (…) ON CONFLICT DO NOTHING RETURNING 1`	Unique-index entry on `idempotency_key`	Only if `C` is idempotent-keyed. Zero rows: `SELECT result, request_sha …` . If `request_sha` matches, `ROLLBACK` and return the stored `result` (replay). If it differs, `ROLLBACK` and return 409 `IdempotencyMismatch`
3	`SELECT * FROM ledger.lock_streams($W, $R)` (below)	`FOR NO KEY UPDATE` on `W` , `FOR SHARE` on `R \ W` , in `stream_id COLLATE "C"` order	Returns `(stream_id, head)` for each. Absent streams return no row
4	`SELECT … FROM ledger.aggregate_snapshots WHERE stream_id = $s AND aggregate_type = $t AND aggregate_version = $v ORDER BY seq DESC LIMIT 1` , then `SELECT seq, event_type, schema_version, payload FROM ledger.events WHERE stream_id = $s AND seq > $snap_seq ORDER BY seq`	none (MVCC read; the stream row lock from step 3 freezes the tail)	Fold (§3.3). The index `events (stream_id, seq)` makes this an index range scan
5	*(no SQL)* `C::decide(&state, &ctx)`	—	Pure. Returns `Vec<NewEvent>` per stream, or `Rejection`

Step	SQL	Locks taken	Notes
6a	For new streams: `INSERT INTO ledger.streams (stream_id, kind, tenant_id, head) VALUES ($s, $k, $t, $n) ON CONFLICT (stream_id) DO NOTHING RETURNING head`	Row lock on the new row	Zero rows means another transaction created the stream first: `Conflict { expected: NoStream, actual }`
6b	For existing streams: `UPDATE ledger.streams SET head = head + $n WHERE stream_id = $s AND head = $expected RETURNING head`	Already held from step 3	Zero rows: `Conflict { expected, actual }`. With step 3's lock this cannot happen for the holder; it does happen when step 3 is skipped (`append_unlocked`, used only by single-stream commands with an `Exact` version supplied by the client)
7	`INSERT INTO ledger.artifacts … ON CONFLICT (sha) DO NOTHING` (per artifact)	Unique-index entry	Artifacts the events cite. `created_pos` is resolved by the deferred FK at commit
8	`INSERT INTO ledger.events (stream_id, seq, event_type, schema_version, payload, payload_sha, command_id, correlation_id, causation_pos, actor, actor_kind) VALUES … RETURNING global_pos`	Row-exclusive on new rows	`seq = expected+1 ..= expected+n`. BEFORE triggers run here: `t_events_seq`, the wallet chain check (§9.2), and the JSON-schema check in Rust before the insert [M R2.2]
9	Inline projector statements (§3.4)	Row locks on projection rows	Through `proj.*` writer functions or the money trigger, each citing a `global_pos` from step 8

Step	SQL	Locks taken	Notes
10	`INSERT INTO prov.nodes / prov.edges …`	—	Provenance for the events, same transaction (§6)
11	`INSERT INTO ledger.outbox (topic, dedupe_key, source_pos, payload) VALUES … ON CONFLICT (dedupe_key) DO NOTHING`	—	Side effects to perform after commit
12	`UPDATE ledger.command_dedup …` is **not** used. Instead the result was computed before step 2, or step 2 is deferred to here as `INSERT … ON CONFLICT DO NOTHING` with the final `result`	—	The result is deterministic from the events, so it is known now
13	`COMMIT`	Deferred constraint triggers run: `t_streams_gap_free`, wallet double-entry, projection justification (§3.4.2), deferred FKs	Any violation aborts the whole transaction: no partial append is possible
14	`NOTIFY abw_events, '<stream_id>:<head>'` is issued *inside* the transaction before step 13	—	Delivered only on commit. Feeds the shared `PgListener` fan-out (G31)

The lock helper:

```
CREATE FUNCTION ledger.lock_streams(p_write text[], p_read text[])
RETURNS TABLE (stream_id text, head bigint, mode text) LANGUAGE plpgsql AS $$
DECLARE r record;
BEGIN
  FOR r IN SELECT s.id, bool_or(s.w) AS w
             FROM (SELECT unnest(p_write) COLLATE "C" AS id, true AS w
                   UNION ALL SELECT unnest(p_read) COLLATE "C", false) s
            GROUP BY s.id ORDER BY s.id
  LOOP
    IF r.w THEN
      RETURN QUERY SELECT x.stream_id::text, x.head, 'write'::text
                     FROM ledger.streams x WHERE x.stream_id = r.id FOR NO KEY
UPDATE;
    ELSE
      RETURN QUERY SELECT x.stream_id::text, x.head, 'read'::text
                     FROM ledger.streams x WHERE x.stream_id = r.id FOR SHARE;
    END IF;
  END LOOP;
END $$;
```

A stream in both sets is locked once, in write mode. Validated output from Appendix D case 21:

```
 stream_id                                    | head | mode
 episode:L                                    |    2 | read
 wallet:22222222-2222-2222-2222-222222222222 |    1 | write
```

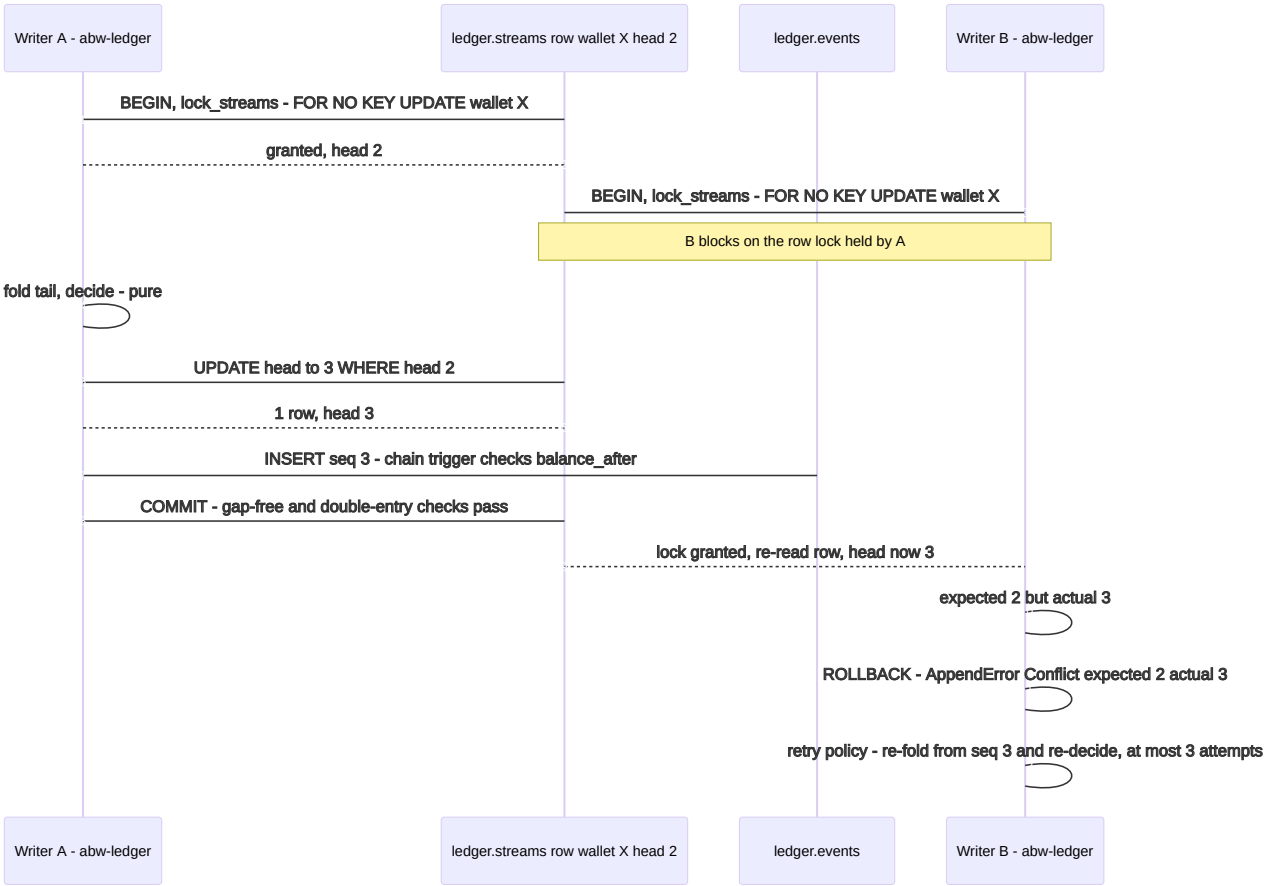
3.2.2 Lock order and deadlock freedom

- **Rule L1.** Within one transaction, stream rows are locked once, in ascending `stream_id COLLATE "C"` order, by `lock_streams`, before any other row lock.
- **Rule L2.** Projection rows are locked only after L1, and only through their own stream. A projection row is keyed 1:1 by its stream, for example `proj.wallets.stream_id UNIQUE`, `proj.claims.stream_id UNIQUE`. Holding the stream lock therefore serialises writers of that projection row.
- **Rule L3.** Unique-index entries (artifacts, `command_dedup`, `events_money_idem`) may be contended. Their waits are bounded: the conflicting transaction is either committing or aborting, and it never waits on a stream lock the current transaction holds. That is because L1 is complete before step 7.

A wait-for cycle needs two transactions that each hold a lock the other wants. Under L1, every stream lock is acquired in one global order, and by L2 and L3 nothing acquired later waits on a stream lock. So no cycle exists. Money transfers lock the two wallets in `stream_id` order. That is the same rule SPEC 36 states as "uuid order" [M R1.1]: both

`wallet:` IDs share a prefix, so bytewise order on the stream ID equals bytewise order on the UUID's text form.

3.2.3 Contention: two writers, same expected version, one loser



- **⟨3⟩.** B blocks on A's row lock and holds nothing yet. By L1, B has taken no other lock, so B cannot be part of a cycle.
- **⟨9⟩–⟨11⟩.** After A commits, B wakes, re-reads the row, finds `head = 3` and rolls back with a typed `Conflict`. No serialisation error is involved.

- **⟨12⟩.** B's retry is a **re-decision**, not a re-send. `decide` runs again on the new state, which may legitimately produce a different result, for example `402` because A spent the balance.
- Commands whose caller supplied an explicit expected version do not retry. They surface `409 Conflict { expected, actual }` to the client. These are claim resubmissions citing an attempt, and human decisions on a packet version.
- The same protocol without `lock_streams` (bare CAS) was also validated. Writer B updated **0 rows**, rolled back, and writer A's event is the only `seq 3` (Appendix D case 19).

3.2.4 Complexity

Operation	Cost
Append of `n` events to `k` streams	`O(k log S)` lock and CAS (B-tree on `streams`), `O(n log E)` inserts. The deferred gap-free check is an index range count of `n` rows per stream
Typical command, e.g. episode close	k = 2–4 streams, n = 5–40 events; 40–60 statements. Target p99 under 25 ms of database time excluding artifact upload
Fold	`O(tail)` with a snapshot, `O(head)` without

3.3 Reading and folding a stream

```
pub trait Aggregate: Default + Sized {
    const TYPE: &'static str;           // "claim", "wallet", "episode", ...
    const VERSION: u32;                 // bump when apply() semantics change;
invalidates snapshots
    fn apply(&mut self, e: &RecordedEvent) -> Result<(), FoldError>;   // total over
registered types
}
```

- **Fold** means `state = events.iter().try_fold(A::default(), apply)`. An unknown `(event_type, schema_version)` is a `FoldError`, never a skip.
- **Upcasting** happens *before* `apply`. `apply` sees only the latest schema version of each type, and upcasters are pure functions registered by `(event_type, from_version)`. Stored payloads are never rewritten.
- **Snapshots.**
  - For streams with more than `SNAPSHOT_EVERY` (200) events, the command path MAY write `aggregate_snapshots` after the fold, *in a separate transaction* after commit. A snapshot is a cache, so losing one is harmless.
  - A snapshot is valid only for its `(aggregate_type, aggregate_version)`.

    - A snapshot never feeds `evaluate()` (T11). The lattice re-reads raw events for anything it cites.
- **Read path for handlers.** Handlers read projections through `ReadDb`; they never fold. Folding is a ledger concern. The exception is `GET /api/claims/:cid/history`, which streams raw events through `ReadDb` with `SELECT … FROM ledger.events WHERE stream_id = $1 ORDER BY seq` and the tenancy check of §13.1.

3.4 Inline projections

3.4.1 Which projections are inline, and why

A projection is **inline** when a later decision must read it consistently in the same or the next transaction: a balance check, a state-machine guard, "is this claim `Proceeding`?". It is also inline when an external party could otherwise observe the event without its effect. Everything else is **async**.

Projection	Inline?	Why
`proj.wallets`, `proj.credit_holds`	**Inline, by trigger**	`wallet.hold` must see the current `balance − held`. A stale read would overdraw. Applied by `SECURITY DEFINER` trigger so the money projection cannot drift from the event (§9.2)
`proj.episodes`	Inline	Presence reads episode terminality. The reaper's predicate reads `status`
`proj.agent_versions`, `proj.agent_heads`	Inline	Run start stamps `card_sha` from the head. An edit and a run must not interleave ambiguously
`proj.claims`	Inline	Every claim command's `decide` reads the state machine. The endorsement queue must not show a claim already decided
`proj.gate_decisions`	Inline	The decision is written with its effect (T5). The read model and the event must agree at commit
`proj.field_evidence`	Inline	The lattice reads it in the same transaction (§5.11)
`proj.external_sources`, `prov.*`	Inline	The chain walk runs inside the evaluation transaction
`proj.publication_releases`, `proj.publication_tokens`	Inline	The public read path serves them immediately after release
Lattice metrics (inner, outer), calibration read model, gate rates, leaderboard, `/api/loops` views	**Async**	Derived aggregates. Staleness of seconds is acceptable. Rebuildable from events
CRUD mirrors (e.g. `public.teams.workspace_budget` display)	**Async**	Display only; never read by a trust decision

3.4.2 The database enforces T3

Every inline projection table carries a **justification trigger**. At commit, it verifies that each inserted or updated row cites an event appended **by this transaction**. The table must record the event position it was last updated from; for example `proj.claims.applied_pos` or `proj.gate_decisions.event_pos`.

```
CREATE FUNCTION proj.row_justified_by_event() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE pos bigint;
BEGIN
  IF TG_OP = 'DELETE' THEN
    IF NOT EXISTS (SELECT 1 FROM ledger.events WHERE txid = pg_current_xact_id())
THEN
      RAISE EXCEPTION 'ABW-T3: delete on %.% without an event in this transaction',
TG_TABLE_SCHEMA, TG_TABLE_NAME;
    END IF;
    RETURN NULL;
  END IF;
  pos := (to_jsonb(NEW) ->> TG_ARGV[0])::bigint;
  IF NOT EXISTS (SELECT 1 FROM ledger.events WHERE global_pos = pos AND txid =
pg_current_xact_id()) THEN
    RAISE EXCEPTION 'ABW-T3: %.% row cites event % that this transaction did not
append',
      TG_TABLE_SCHEMA, TG_TABLE_NAME, pos USING ERRCODE = 'insufficient_privilege';
  END IF;
  RETURN NULL;
END $$;

-- attached once per inline projection, naming the column that holds the justifying
event position, e.g.
--   CREATE CONSTRAINT TRIGGER t_claims_justified AFTER INSERT OR UPDATE OR DELETE
ON proj.claims
--     DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('applied_pos');
-- (each projection's DDL in sections 4 to 10 carries its own trigger)
```

Validated (Appendix D cases 22–24):

- an insert citing an old event is rejected;
- an insert citing the transaction's own event commits;
- a later `UPDATE` that keeps the old position is rejected, *even when run by the table owner*.

Combined with grants (`abw_app` has `INSERT, UPDATE` on `proj.*` but `abw_read` has `SELECT` only) and with T1 (only `abw-ledger` holds an `abw_app` connection), a projection row cannot change without its event.

Cost. The check is one primary-key probe on `events` per changed projection row, deferred to commit. An episode close changing 10 projection rows costs 10 index probes, about 50 µs.

3.4.3 Rebuild

Every projection, inline or async, MUST be rebuildable from `ledger.events` by its projector's `apply`.

- `abw-admin projection rebuild <name> --diff` replays into a shadow table, in the same transaction pattern but under `abw_owner` with the justification triggers disabled on the *shadow* table only, and reports a row diff [M R2.6].
- CI runs the rebuild against the fixture corpus. A non-empty diff fails the build (§14.6).

3.5 Read-side visibility for asynchronous consumers

The problem. `global_pos` is assigned at insert, but transactions commit in a different order. A reader that checkpoints on `global_pos` alone permanently skips any event whose transaction was still open at read time with a *lower* `global_pos`.

SPEC 36's fence is insufficient. SPEC 36 R2.6 checkpoints on `global_pos` and filters `txid < pg_snapshot_xmin(pg_current_snapshot())`. That still skips events, because `xid` order is not `global_pos` order either. A transaction can take its xid early (for example, by locking a stream row) and insert its event late, while a later-xid transaction inserts first. This was reproduced (Appendix D case 26):

```
stream_id | global_pos | txid
episode:H |          9 | 1075     <- later xid, lower position, committed second
episode:L |         10 | 1073     <- earlier xid, higher position, committed first
```

The sequence of events in that run:

1. With a third transaction (xid 1074) open, the fence admitted `L` (1073 < 1074). The checkpoint advanced to `global_pos = 10`.
2. When 1074 closed, `H` (pos 9) became visible.
3. `global_pos > 10` never returns `H`, so `H` was lost.

Target fence. The checkpoint is the pair `(fence_txid, fence_pos)`, and events are consumed in `(txid, global_pos)` order, below the snapshot's `xmin`:

```
-- run in its own READ COMMITTED transaction by the async projector
SELECT global_pos, stream_id, seq, event_type, schema_version, payload, txid
  FROM ledger.events
 WHERE (txid, global_pos) > ($fence_txid::xid8, $fence_pos::bigint)
   AND txid < pg_snapshot_xmin(pg_current_snapshot())
 ORDER BY txid, global_pos
 LIMIT $batch;                                  -- uses index events_fence (txid,
global_pos)
```

Why it is complete:

- Every transaction with `xid < xmin` has finished; that is the definition of `xmin`. So the set of events with `txid < xmin` is closed: no new row can ever join it.
- Ordering by `(txid, global_pos)` and checkpointing the last `(txid, global_pos)` consumed therefore never skips a committed row.
- Rows from aborted transactions never become visible and are simply absent.
- Applied to the validated scenario, the target fence returns `H` on the second round (Appendix D case 27).

Cost and liveness.

- The query is a range scan on `events_fence`. The plan is a bitmap index scan on `ROW(txid, global_pos) > ROW(…)` with `txid < xmin` as an index condition.
- **Lag** is bounded by the oldest open transaction in the cluster. `idle_in_transaction_session_timeout = 30s` on `abw_app` bounds it to 30 s for ledger transactions. A long `abw_read` report holds `xmin` too, so `abw_read` sessions carry `statement_timeout = 60s`. Projector lag is exported as `now() − recorded_at` of the last consumed event, with an alert above 60 s.
- **Checkpoint atomicity.** Each batch's projection writes and the checkpoint `UPDATE` commit in one transaction, so a crash re-delivers at most one batch. Async projectors MUST be idempotent per `(global_pos)`; each async projection row records the `global_pos` it last applied.
- **Leases.** Each projector holds a lease (`lease_owner` , `lease_until` , refreshed every `TTL/3`), so two workers never run the same projector. This replaces the consolidation lease pattern whose `extend` had no caller (G28).

Inline readers need no fence. An inline projection is written in the same transaction as its event, so ordinary MVCC visibility makes event and projection appear together.

3.6 JCS canonicalisation and the hash format

3.6.1 Byte rules

The canonical form of a JSON value is RFC 8785 (JCS). An implementation MUST follow these rules, which restate RFC 8785 §3.2 in implementable terms:

1. **Input domain is I-JSON** (RFC 7493).
   - Strings MUST be valid Unicode: no lone surrogates.
   - Numbers MUST be IEEE-754 binary64 values. An integer outside ±(2^53 − 1) is **rejected**, not rounded.
   - Object member names MUST be unique; duplicates are rejected at parse.
   - `NaN` and `±Infinity` are rejected.
2. **Whitespace.** None is emitted.
3. **Literals** are `null` , `true` and `false` .
4. **Strings.**
   - Output as UTF-8, enclosed in `"` .

- Escaped as exactly `\"`, `\\`, `\b`, `\f`, `\n`, `\r` and `\t`.
- All other code points below U+0020 are written `\u00xx` with **lowercase** hex.
- Everything else, including `/`, DEL and all non-ASCII, is emitted **raw**.

5. **Numbers** use the ECMAScript `Number.prototype.toString` algorithm (ECMA-262 §7.1.12.1). Let the shortest round-trip digit string be `d` (`k` digits) and the decimal exponent `n`, with value = 0.`d` × 10^n:
   - `-0` is written `0`.
   - If `k ≤ n ≤ 21`, write `d` followed by `n − k` zeros. Examples: `100`, `10000000000000000000`.
   - If `0 < n ≤ 21`, write `d[0..n] . d[n..]`. Example: `4.5`.
   - If `−6 < n ≤ 0`, write `0.` then `−n` zeros then `d`. Example: `0.000001`.
   - Otherwise use exponent form: `d[0]`, then `.d[1..]` if `k > 1`, then `e`, the sign (`+` or `-`), and `|n − 1|`. Examples: `1e+21`, `5e-7`, `1e-27`.
6. **Objects.** Members are sorted by their names compared as sequences of **UTF-16 code units**, not code points or UTF-8 bytes. This differs from code-point order for characters above U+FFFF. Values are canonicalised recursively.
7. **Arrays** keep their order. Elements are canonicalised recursively.
8. **Hash.** `sha256:` ‖ lowercase hex of SHA-256 over the UTF-8 bytes of the canonical text: 71 ASCII characters in total, matching `^sha256:[0-9a-f]{64}$`.

For `raw-utf8` artifacts the hash is over the UTF-8 bytes as given, with no normalisation, and for `raw-bytes` it is over the bytes as given. The canonicalisation tag is stored with the artifact, so a verifier knows which rule to apply.

This replaces `artifact_hash::canonical` (`src/artifact_hash.rs:118-133` [K: to be replaced]). That function sorts keys bytewise and emits `serde_json` compact output, which is not JCS. The two disagree on:

- floats, because `serde_json` emits `1e30` where JCS emits `1e+30`;
- key order above U+FFFF.

Every hash computed by the old function is recomputed in migration, with `legacy_sha` kept for one release [M R3.1].

Rust implementation constraint. The root crate enables `serde_json/preserve_order` (`Cargo.toml:61` [K]). JCS MUST NOT depend on map iteration order; it sorts explicitly. It MUST also NOT go through `serde_json`'s float formatter, which is ryu shortest *but formatted differently*; it MUST format with the ECMAScript algorithm above.

3.6.2 Test vectors

The conformance test `tests/claims/jcs_vectors.rs` MUST include the RFC 8785 vectors and these platform vectors. All values were computed with an independent JCS implementation and cross-checked against Postgres `sha256()` (Appendix D case 28).

#	Input
V1	RFC 8785 §3.2.2 sample: `{"numbers":[333333333.33333329,1E30,4.50,2e-3,0.000000000000000000000000001],"string":"\u20ac$\u000F\u000aA'\u0042\u0022\u005c\uaBcD","literals":[null,true,false]}`
V2	RFC 8785 §3.2.3 key-sorting sample (keys `€`, `\r`, `1`, `ד`, `😀`, `\u0080`, `ö`)
V3	`{}`
V4	`raw-utf8` artifact `hello, ABW\n` (11 bytes)
V5	A `lattice.evaluated` payload with integer-encoded scores (§5.7)
V6	Numbers `0.0, -0.0, 1.0, 1e21, 1e20, 5e-7, 1e-6, 0.1, 9007199254740991`
V7	`9007199254740992` (2^53)
V8	`{"a":1,"a":2}`
V9	A string containing U+0000

3.6.3 Floats in trust payloads

A verdict MUST be reproducible bit for bit (T11), so every score, θ and probability in a trust event is encoded **as integers**:

- **Fractions** are `(num, den)` pairs. For example, Truth is `matched/total` as `score_num` and `score_den`.
- **Thresholds** are basis points (`theta_bp`, 0–10000).
- **Probabilities** are parts per million (`p_ppm`, 0–1000000).
- **Brier scores** are millionths (`brier_micro`).

Comparisons are exact integer arithmetic: `score_num × 10000 ≥ theta_bp × score_den`. This avoids any dependence on float formatting, platform rounding or `jsonb`'s numeric normalisation [P OD-07]. The Lattice spec's JSON example uses decimals (`"score":0.81,"theta":0.70`) [S §Event schema]. The API read model renders decimals for display, computed from the stored integers.

3.7 Event envelope

```rust
/// Constructed only by `NewEvent::new`, which validates the payload against
`event_types.json_schema`
/// (cached at boot from the table; boot fails if a Rust event type is missing from
the table).
pub struct NewEvent {
    event_type: EventType,           // &'static str newtype, from the closed
`events!` registry
    schema_version: u16,
    payload: CanonicalJson,          // JCS-validated value plus its sha (computed
once)
    causation: Option<GlobalPos>,
    artifacts: Vec<ArtifactRef>,     // must be stored in the same tx (step 7)
    prov: Vec<ProvWrite>,            // nodes/edges caused by this event (step 10)
    outbox: Vec<OutboxJob>,          // side effects (step 11)
}

pub struct RecordedEvent {
    pub global_pos: GlobalPos, pub stream: StreamId, pub seq: u64,
    pub event_type: EventType, pub schema_version: u16, pub txid: u64,
    pub payload: serde_json::Value, pub payload_sha: Sha256Ref,
    pub command_id: Uuid, pub correlation_id: Uuid, pub causation:
Option<GlobalPos>,
    pub actor: Actor, pub recorded_at: DateTime<Utc>,
}
```

Each event type is declared once, in Rust, with a `serde` payload struct and a JSON Schema derived from it:

```rust
events! {
    "claim.submitted"   v1 on Claim   (Evidentiary)  => ClaimSubmittedV1,
    "lattice.evaluated" v1 on Claim   (Evidentiary)  => LatticeEvaluatedV1,
    "judge.reported"    v1 on Claim   (Interpretive) => JudgeReportedV1,
    "route.decided"     v1 on Claim   (Decision)     => RouteDecidedV1,
    // ... the full catalogue in 5.15
}
```

A migration registers the same rows in `ledger.event_types`. A boot check compares the Rust registry with the table, as `(event_type, schema_version, json_schema_sha)` sets, and **fails boot** on any difference [P OD-08].

3.8 Rust API surface

Signatures only. Bodies are implementation detail; the semantics are given in §3.2–§3.5.

```
// ---- handles ----------------------------------------------------------------
-----
pub struct Ledger { /* private: write pool, event registry, policy cache, object
store */ }
#[derive(Clone)] pub struct ReadDb(/* private: PgPool connected as abw_read */);

impl Ledger {
    pub async fn connect(cfg: &LedgerConfig, fence: &SingletonFence) ->
Result<(Ledger, ReadDb), BootError>;
    pub async fn dispatch<C: Command>(&self, cmd: C, ctx: &CommandCtx) ->
Result<Dispatched<C::Output>, DispatchError>;
    pub fn crud(&self) -> &Crud;                                    // non-trust
writes, 14.5
    pub fn reader(&self) -> ReadDb;
}

impl ReadDb {
    pub async fn acquire(&self) -> Result<ReadConn, sqlx::Error>; // read-only
session
    pub fn as_executor(&self) -> &sqlx::PgPool;                  // usable with
sqlx::query for SELECT
}

// ---- commands ---------------------------------------------------------------
----
pub trait Command: Send + Sync + 'static {
    const ID: CommandId;                                            // equals a
command_registry entry
    type Aggregate: Aggregate;
    type Output: Serialize + Send;
    fn streams(&self) -> StreamSet;                                 // write set W and
read set R
    fn idempotency_key(&self) -> Option<IdempotencyKey>;
    fn validate(&self) -> Result<(), Rejection>;                    // pure
    fn decide(&self, state: &Folded<Self::Aggregate>, ctx: &DecideCtx)
        -> Result<Decision<Self::Output>, Rejection>;               // pure
}
pub struct Decision<O> { pub appends: Vec<(StreamId, Expected, Vec<NewEvent>)>, pub
output: O }

pub enum Expected { NoStream, Exact(u64) }                          // no `Any` here
pub enum ExpectedCommutative { Any }                                // only accepted
by append_telemetry

pub enum DispatchError {
    Rejected(Rejection),                                            // typed domain
refusal; 4xx
```

```
    Conflict { stream: StreamId, expected: Expected, actual: u64 },
    IdempotencyMismatch { key: IdempotencyKey },
    Schema(EnvelopeError),                                  // payload failed
json_schema
    Integrity(IntegrityError),                              // a DB invariant
trigger fired: a bug
    Db(sqlx::Error),
}

// ---- inline projectors ------------------------------------------------------
-----
pub trait InlineProjector: Send + Sync {
    fn handles(&self) -> &'static [EventType];
    fn project<'t>(&self, tx: &mut ProjTx<'t>, e: &RecordedEvent) -> BoxFuture<'t,
Result<(), ProjError>>;
}
/// Only obtainable inside dispatch after step 8; exposes writer functions keyed by
GlobalPos.
pub struct ProjTx<'t> { /* private */ }

// ---- async projectors -------------------------------------------------------
-----
pub trait AsyncProjector: Send + Sync {
    const NAME: &'static str;
    fn handles(&self) -> &'static [EventType];
    fn apply<'t>(&self, tx: &mut ProjTx<'t>, batch: &[RecordedEvent]) ->
BoxFuture<'t, Result<(), ProjError>>;
}
pub async fn run_projector<P: AsyncProjector>(ledger: &Ledger, p: P, stop:
CancellationToken) -> Result<(), ProjError>;

// ---- artifacts --------------------------------------------------------------
-----
pub struct Sha256Ref(/* "sha256:" + 64 hex, validated */);
pub enum Canon { JcsRfc8785, RawUtf8, RawBytes }
pub struct ArtifactRef { pub sha: Sha256Ref, pub media_type: MediaType, pub canon:
Canon, pub byte_len: u64 }
pub fn artifact_json(v: &serde_json::Value, media_type: MediaType) ->
Result<PendingArtifact, JcsError>;
pub fn artifact_bytes(b: Bytes, canon: Canon, media_type: MediaType) ->
PendingArtifact;
impl ReadDb { pub async fn resolve(&self, sha: &Sha256Ref) -> Result<Option<Bytes>,
ArtifactError>; }

// ---- canonical JSON ---------------------------------------------------------
-------
pub fn jcs(v: &serde_json::Value) -> Result<Vec<u8>, JcsError>;
pub fn sha256_jcs(v: &serde_json::Value) -> Result<Sha256Ref, JcsError>;
```

```rust
pub fn sha256_raw(b: &[u8]) -> Sha256Ref;

// ---- outbox ----------------------------------------------------------------
-----
pub struct OutboxJob { pub topic: Topic, pub dedupe_key: String, pub payload:
serde_json::Value }
pub trait OutboxHandler: Send + Sync {
    const TOPIC: Topic;
    fn handle<'a>(&'a self, job: &'a ClaimedJob, ledger: &'a Ledger) ->
BoxFuture<'a, Result<(), JobError>>;
}

// ---- policy ----------------------------------------------------------------
-----
pub struct PolicyRef { pub policy_id: Uuid, pub version: u32, pub doc_sha: Sha256Ref
}
impl ReadDb { pub async fn policy(&self, r: &PolicyRef) -> Result<PolicyDoc,
PolicyError>;
              pub async fn active_policy(&self, kind: PolicyKind, tenant: &TenantId,
at: DateTime<Utc>)
                  -> Result<PolicyRef, PolicyError>; }
```

3.9 How the type system makes direct writes impossible

T1 requires that "a handler that tries to write directly fails to compile" [R §5 Phase 1 acceptance]. The mechanism has four layers. Each layer closes a hole the previous one leaves open.

1. **No write pool is nameable outside the crate.**
   - `Ledger` 's pool field is private.
   - `AppState` holds `ledger: Arc<Ledger>` and `db: ReadDb` , and no `PgPool` field.
   - The current `db: PgPool` (`src/api_server.rs:433` [K: removed]) is replaced, so handler code that passes `&state.db` to `sqlx::query(..).execute(..)` with a write statement no longer has a write-capable executor to pass.
2. **`ReadDb` is not a write executor.**
   - `ReadDb::as_executor` returns a pool connected as `abw_read` . Strictly, `sqlx` would still *compile* an `UPDATE` against it; the database refuses it at run time (`cannot execute INSERT in a read-only transaction` , and grants deny it even under `BEGIN READ WRITE` : Appendix D cases 12, 13, 18).
   - To reach compile time as well, `ReadDb` exposes only `fetch_one` , `fetch_optional` and `fetch_all` , each taking a `ReadQuery` .
   - `ReadQuery` is built only by the `read_query!` macro. The macro **parses the SQL literal at compile time** and emits `compile_error!` unless the first keyword is `SELECT` or `WITH` , it contains no data-modifying CTE

(`INSERT|UPDATE|DELETE|MERGE`), and it contains no `FOR UPDATE` or `FOR SHARE` [P OD-09].

3. **The crate graph forbids the bypass.**
   - Handler and executor crates do not depend on `sqlx` with the `postgres` feature *directly*. They see `sqlx` types only through `abw-ledger` 's re-exports, which include no `PgPool::connect` .
   - Concretely, the root `fermi` crate's handlers move to a `fermi-http` crate whose `Cargo.toml` has no `sqlx` dependency. `cargo deny` with a `bans` entry prevents adding one [P OD-10].
   - A test in `tests/claims/no_write_pool.rs` fails if any crate other than `abw-ledger` and the explicitly listed CRUD-free utility binaries calls `PgPoolOptions::connect*` .
4. **Commands are the only path to `ProjTx` .**
   - `ProjTx<'t>` has a private constructor and a lifetime tied to the dispatch transaction.
   - Inline projectors receive `&mut ProjTx` . They cannot store it, return it, or open their own transaction.
   - `Tx` is deliberately `!Send` (it holds a `PhantomData<*const ()>`). No future that holds a ledger transaction across an `.await` on a non-ledger future can be spawned onto the multi-thread runtime, so "LLM call inside a transaction" is a compile error at the `tokio::spawn` boundary. The only `.await` s inside dispatch are ledger-internal.

What remains outside the type system, and is caught elsewhere:

Hole	Caught by
`abw-ledger::crud` executing an `UPDATE` on a trust table	Grants (`abw_app` has no `UPDATE` on `ledger.*` beyond `streams.head` , and none on `prov.*`), justification triggers on `proj.*` , and `ratchet.py` metric `trust_table_write_outside_ledger` = 0
A future crate depending on `sqlx/postgres`	`cargo deny` `bans` + `tests/claims/no_write_pool.rs`
An operator connecting with `abw_owner`	`forbid_mutation` triggers fire for owners; nightly audit of the hash chain

3.10 Failure and crash semantics

Postgres commit is atomic and durable (`synchronous_commit = on` , required in production). The table gives the state after `SIGKILL` of the process, or a network partition, at each statement boundary of §3.2.1.

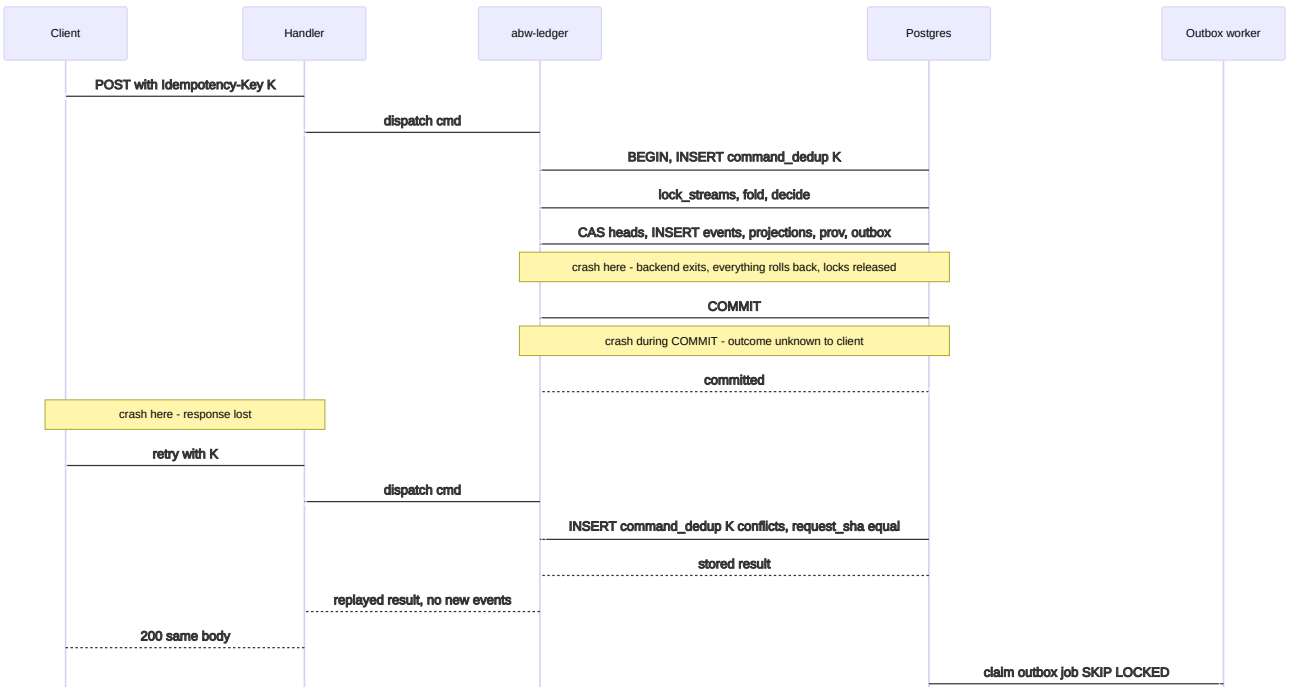
Crash after step	Durable state	Client sees	Recovery
0–1	Nothing	Connection reset / 5xx	Client retries. With an idempotency key the retry is the first attempt
2 (dedup row inserted, uncommitted)	Nothing (rolled back on disconnect)	5xx	Same
3–4 (locks held)	Nothing. Postgres releases locks at backend exit	5xx	Concurrent writers that were blocked proceed
6–11 (any partial writes)	Nothing. The transaction is aborted	5xx	Same
7, large artifact	Object uploaded, no `artifacts` row	5xx	Orphan object GC'd after 7 days (§2.5)
13, during `COMMIT`	Either the whole transaction or nothing; the client cannot tell which	Ambiguous: connection drops mid-commit	Client retries with the same idempotency key. If it committed, step 2 returns the stored result; if not, it runs
After 13, before the response	Committed	5xx / timeout	Same as above. Outbox jobs run regardless
After 13, outbox job mid-flight	Committed event; job claimed (`FOR UPDATE SKIP LOCKED`) by a worker that dies	—	Lock released at backend exit; job re-claimed. Side effects MUST be idempotent on `dedupe_key` (webhook carries `ABW-Idempotency-Key` ; git writes check the target tree first)
Async projector mid-batch	Batch transaction aborted; checkpoint unchanged	—	Batch re-delivered; per-row `applied_pos` makes it idempotent

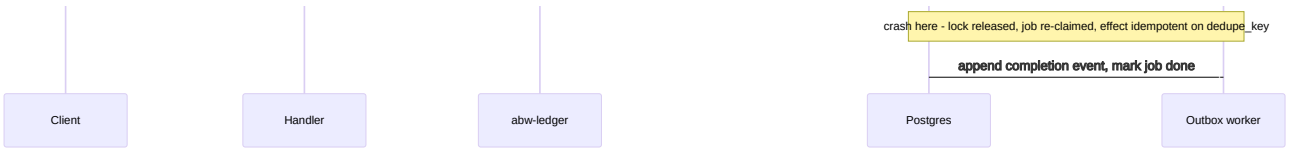
Invariant under every crash: no partial append exists. The deferred triggers (`t_streams_gap_free`, double-entry, justification) run inside `COMMIT`. Any state that would violate T2, T3 or T9 is rejected before durability, so a crash cannot produce it.

Integrity audit (nightly, `worker` role, `abw_read`):

- Re-derive `payload_sha` for every event since the last audit.
- Recompute every inline projection for every stream touched, and compare it with the stored projection.
- Verify that each wallet's `balance_after` chain is continuous.
- Hash large artifacts.
- A non-zero result emits `platform_health.observed { rung: rollup_consistent, state: violated }` and pages.

This is how a superuser bypass of triggers is detected after the fact.





4. What an agent is, in the target

4.1 Definition

An agent is an identity plus a **version stream of resolved cards**. Each run is an **episode stream** that cites exactly one version.

Element	Physical form (target)	Mutability
Identity	`public.agents` row: `agent_id`, `agent_name`, owner, tier, status	CRUD; configuration columns removed from it in Phase 3
Declaration	**Resolved card artifact**: `sha256:` of `JCS(resolved card)` = `card_sha`	Immutable (artifact)
History	Stream `agent:<agent_id>`: `agent.version_created` events; projections `proj.agent_versions`, `proj.agent_heads`, `proj.agent_field_origin`	Append-only
Behaviour record	Stream `episode:<episode_id>`; projection `proj.episodes`	Append-only
Memory	`semantic_rules`, `entities`, `facts`, `communities`	CRUD (memory domain), with provenance floors computed from `prov.edges` (§6.6)
Economics	`wallet:<wallet_id>` streams; `proj.wallets`	Append-only stream, projected balance
Runtime	`ToolAwareExecutor` (kept: `src/agent_backend/tool_executor.rs:906-913` [K]), bounded loop of `MAX_ITERATIONS = 5` (`:27` [K]) plus one flush turn (`FLUSH_TIMEOUT_SECS = 240`, `:60` [K])	Stateless per run

4.2 The resolved card and the version stream

Resolution is kept and its output becomes identity.

- The file×DB merge in `resolve_agent_card` (`src/api_server.rs:6834` [K]) and `agent_card_from_db` (`:6699` [K]) keeps its overlay rules (LLD §2.3).
- Its output is no longer a throwaway value. The resolved card is canonicalised and stored as an artifact (`canonicalization = 'jcs-rfc8785'`, `media_type = 'application/vnd.abw.agent-card+json'`), and its hash is the version identity [R §4 G14].
- **Excluded fields** are those that change at run time without a configuration change: `performance`, `usage`, `ontology_stats`, `wallet`, `declared_prompt_sha256`. The exclusion list is a `const` in `abw-agents` and is itself hashed into the card artifact as `"$excluded"`, so changing the list changes every `card_sha` visibly instead of silently.

- The in-memory `declared_prompt_sha256` (bare hex, `skip_serializing`; LLD §2.2) is removed. Its role passes to `card_sha` and `prompt_sha = sha256_raw(system_prompt)`, both stored [M R3.6].

The version stream `agent:<agent_id>`:

Event	Payload	Emitted by
`agent.version_created` v1	`{agent_id, version_number, card_sha, prompt_sha, reason: seed	owner_edit
`agent.retired` v1	`{agent_id, reason}`	`agent.retire` (soft delete; replaces `DELETE` cascades)

- **Numbering.** `version_number` **is** the stream `seq` of the `agent.version_created` event. Uniqueness and gap-freedom therefore come from the stream (§3.1.2), not from `MAX+1` (LLD §2.4).
- **No-op edits.** An edit whose resolved card hashes to the current head's `card_sha` is a no-op that appends nothing; the command returns the existing version. Identical configurations therefore cannot produce two versions.

```
CREATE TABLE proj.agent_versions (
  agent_id       uuid NOT NULL,
  version_number int  NOT NULL CHECK (version_number >= 1),
  card_sha       text NOT NULL REFERENCES ledger.artifacts ON DELETE RESTRICT,
  reason         text NOT NULL CHECK (reason IN
('seed','owner_edit','adopt_file','revert','backfill')),
  actor          text NOT NULL,
  event_pos      bigint NOT NULL UNIQUE REFERENCES ledger.events (global_pos),
  PRIMARY KEY (agent_id, version_number));
CREATE INDEX agent_versions_by_sha ON proj.agent_versions (card_sha);
CREATE TRIGGER t_agent_versions_immutable BEFORE UPDATE OR DELETE ON
proj.agent_versions
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
CREATE CONSTRAINT TRIGGER t_agent_versions_justified AFTER INSERT ON
proj.agent_versions
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('event_pos');

CREATE TABLE proj.agent_heads (
  agent_id       uuid PRIMARY KEY,
  version_number int  NOT NULL,
  card_sha       text NOT NULL,
  applied_pos    bigint NOT NULL REFERENCES ledger.events (global_pos),
  FOREIGN KEY (agent_id, version_number) REFERENCES proj.agent_versions ON DELETE
RESTRICT);
CREATE CONSTRAINT TRIGGER t_agent_heads_justified AFTER INSERT OR UPDATE OR DELETE
ON proj.agent_heads
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('applied_pos');

CREATE TABLE proj.agent_field_origin (
  agent_id    uuid NOT NULL,
  field_path  text NOT NULL,
  origin      text NOT NULL CHECK (origin IN ('file','owner')),
  applied_pos bigint NOT NULL REFERENCES ledger.events (global_pos),
  PRIMARY KEY (agent_id, field_path));
CREATE CONSTRAINT TRIGGER t_agent_field_origin_justified AFTER INSERT OR UPDATE OR
DELETE ON proj.agent_field_origin
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('applied_pos');
```

An edit is one command and one transaction [R §4 G14]:

- `PUT /api/agents/:id` dispatches `agent.update { patch }`.
- `W = {agent:<id>}`, `Expected::Exact(head)`, where the head comes from the client's `If-Match: "<version_number>"` header. Without the header, the head is read under

the stream lock.
- `decide` resolves `patch ∘ head card` and hashes it. If the hash is new, it emits `agent.version_created { reason: owner_edit, field_origins: {each patched path: owner} }`.
- The new card artifact, the event, and the three projections commit together.
- The legacy `public.agents` configuration columns are refreshed **asynchronously** from `proj.agent_heads`, for display only, until Phase 3 removes them. No run reads them.

Seed on insert only [R §4 G13]. The boot seeder (`seed_agents_to_database`, `src/api_server.rs:5747` [K: replaced]) dispatches `agent.seed { file_card }` per curated card:

Condition	Effect
No `agent:<id>` stream exists	`agent.version_created { reason: seed, version 1, every field origin = file }`
Stream exists and file `card_sha` equals the head's	No-op
Stream exists and they differ	For each changed path whose `proj.agent_field_origin.origin = 'file'`, take the file value. Paths with `origin = 'owner'` are **never** overwritten. If the merged card differs from the head, emit `agent.version_created { reason: adopt_file, changed_fields }`

The current `upsert_agent … system_prompt = EXCLUDED.system_prompt` overwrite (`agent-bestiary/memory/src/store.rs:642-669`) and the seeder's `mcp_servers: None` (`src/api_server.rs:5815`) are deleted. Every reseed is therefore a visible version with a reason.

4.3 Per-agent tool allowlist

G24 is "Do now": a per-agent tool allowlist from the card [R §4].

- **Declaration.** `capabilities.tools_allowed: Vec<ToolName>` is part of the resolved card and therefore of `card_sha`. Each element is one of:
  - a `PlatformTool::name()`;
  - `mcp:<server>__<tool>`, for a declared remote MCP tool;
  - `skill:<name>`, for a registered skill.
- **Defaults for existing cards.** Existing cards get the tool set they effectively have today (LLD §2.5: every LLM-visible platform tool) as a one-time `backfill` version with field

origin `file` [M App. A G24]. Owners and the outer loop narrow it from there. New cards default to the empty set.

- **Enforcement is a control (T7).** It applies at two points:
  1. Building the tool list. The LLM-visible filter at `src/agent_backend/tools/registry.rs:128` [K: location] additionally filters by `tools_allowed`, so the model is never offered a disallowed tool.
  2. Execution. `PlatformToolRegistry::execute` (`registry.rs:64` [K: location]) refuses any call whose name is not in the run's allowlist. This defeats a model that names a tool it was not offered.
- **Recording a refusal.** A refused call:
  - returns a tool error to the model;
  - appends `tool.invoked { allowed: false }` through the `tool.record` command;
  - appends `gate.decided { gate: tool_allowlist, decision: refused, reason_class: policy }` on the episode stream, in that tool call's append transaction (§4.5).
- **Delegation tools** (`is_delegation`) and workspace tools (`requires_workspace`) need both the allowlist entry and the existing registry variant (`standard()`, `all()`, `workspace_no_delegation()`, `registry.rs:34-46` [K]).

4.4 What is kept from the executor

The loop shape is kept exactly:

- the provider split into `execute_anthropic_loop` (`tool_executor.rs:147` [K]) and `execute_openai_loop` (`:545` [K]);
- `MAX_ITERATIONS`;
- the flush turn;
- sequential tool execution within a turn.

Three changes are required:

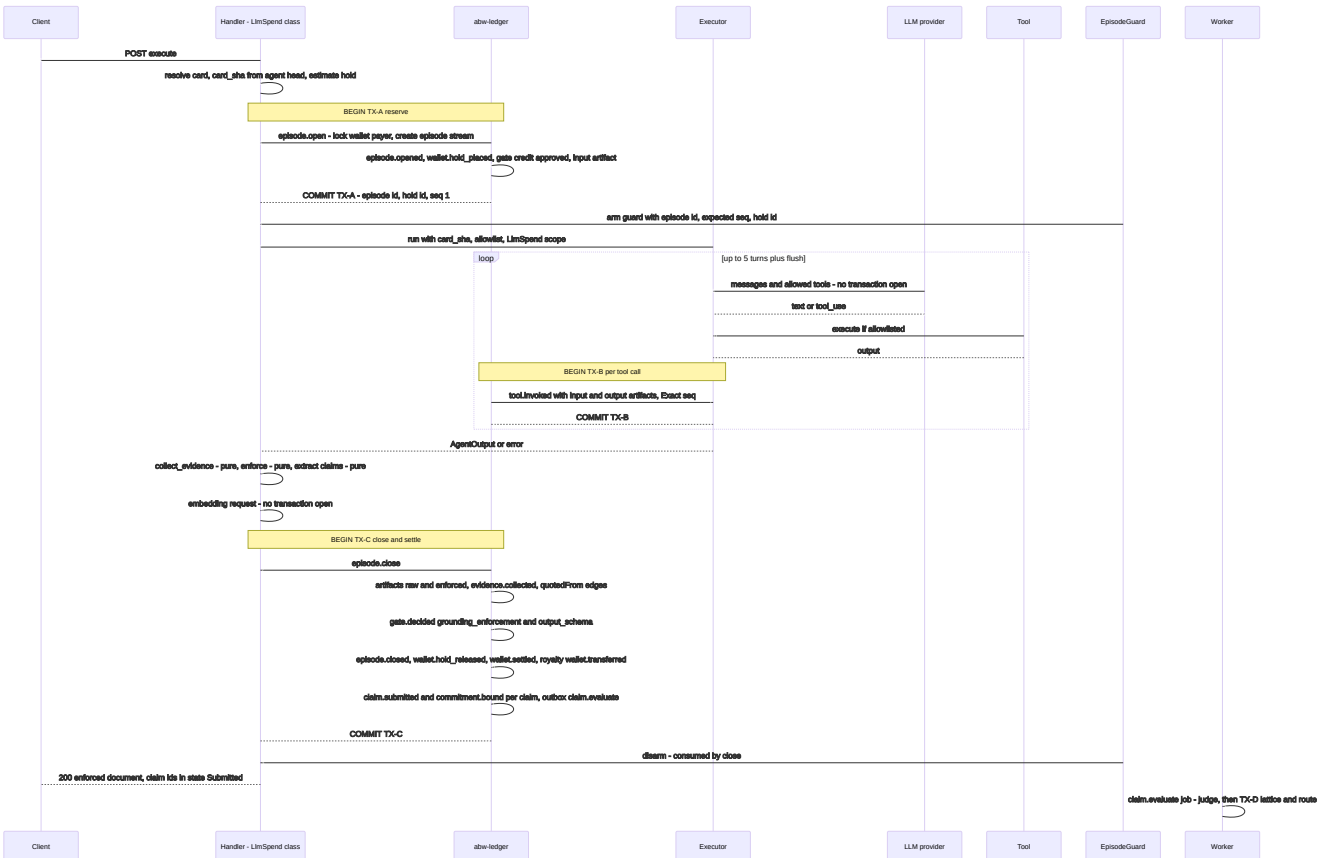
1. **UTF-8-safe truncation.** Tool results are truncated with `floor_char_boundary`, replacing the byte slice at the 32 000 caps (`:331`, `:690` [K: location]) [R §4 G06].
2. **Attachments** reach the first user message on the tool loop [R §4 G26].
3. **Durable tool invocations.** Every tool invocation becomes a stored artifact and a `tool.invoked` event *as it happens* (§4.5), not only a JSON array inside `episodes.context` (`src/episodes.rs:293-301` [K: current shape, retained as a projection field]).

4.5 Execution pipeline

An agent run is **three phases in separate transactions**, because no LLM or HTTP call may run inside a ledger transaction (§2.3).

Phase	Transaction	Isolation	Write set	Effect
Reserve	TX-A	READ COMMITTED	`episode:<eid>` (new), `wallet:<payer>`	Episode opened, card version stamped, input stored, credit held
Run	One short TX-B_i per tool call; none during LLM turns	READ COMMITTED	`episode:<eid>`	Each tool call's input and output stored as an artifact and recorded as an event
Close + settle	TX-C	READ COMMITTED	`episode:<eid>`, `wallet:<payer>`, `wallet:<platform>`, `wallet:<owner>` (royalty), `claim:<cid>` … (new), `agent:<id>` (read)	Outputs stored, evidence recorded, gates decided, claims submitted, money settled
Evaluate (per claim)	TX-D	REPEATABLE READ	`claim:<cid>`	Judge report, verdict vector and route in one transaction (§5.11)

The `episode:` stream has exactly one writer during a run: the request that opened it. The reaper is the only other writer, and it also uses expected-version. If the reaper wins, the run's next append gets `Conflict` and the executor stops.

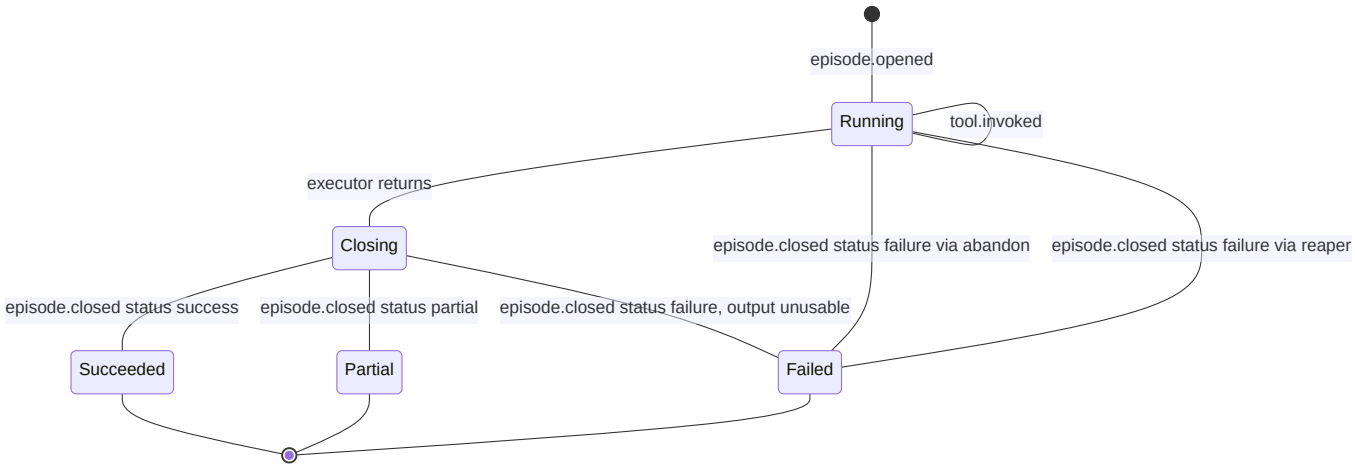

Client
Handler - LlmSpend class
abw-ledger
Executor
LLM provider
Tool
EpisodeGuard
Worker
POST execute
resolve card, card_sha from agent head, estimate hold
BEGIN TX-A reserve
episode.open - lock wallet payer, create episode stream
episode.opened, wallet.hold_placed, gate credit approved, input artifact
COMMIT TX-A - episode id, hold id, seq 1
arm guard with episode id, expected seq, hold id
run with card_sha, allowlist, LlmSpend scope
loop
[up to 5 turns plus flush]
messages and allowed tools - no transaction open
text or tool_use
execute if allowlisted
output
BEGIN TX-B per tool call
toolInvoked with input and output artifacts, Exact seq
COMMIT TX-B
AgentOutput or error
collect_evidence - pure, enforce - pure, extract claims - pure
embedding request - no transaction open
BEGIN TX-C close and settle
episode.close
artifacts raw and enforced, evidence.collected, quotedFrom edges
gate.decided grounding_enforcement and output_schema
episode.closed, wallet.hold_released, wallet.settled, royalty wallet.transferred
claim.submitted and commitment.bound per claim, outbox claim.evaluate
COMMIT TX-C
disarm - consumed by close
200 enforced document, claim ids in state Submitted
claim.evaluate job - judge, then TX-D lattice and route

Frame notes:

- ⟨2⟩ The card is resolved from `proj.agent_heads` and the head's card artifact, not from mutable columns.
  - `produced_by_version` is therefore the version **at run start**, carried in `episode.opened`. This replaces the correlated subquery at write time (`store.rs:339-344`, LLD §2.4).
  - The hold estimate is `rate(model) × max_tokens × (MAX_ITERATIONS + 1) + gas`, capped at `EXECUTION_HOLD_CAP` [M R1.3]. The cap value is OD-36.
- ⟨3⟩–⟨5⟩ Insufficient available balance (`balance − held < estimate`), or outstanding debt, is a `Rejection`:
  - A one-statement transaction appends `gate.decided { gate: credit, decision: refused, reason_class: policy }` on the stream `gate:credit:<tenant>:<yyyy-mm-dd>` [P OD-04].
  - The handler returns **402** before any LLM call.
  - The payer is the caller, or the workspace wallet on workspace paths.
- ⟨6⟩ `EpisodeGuard` is `#[must_use]`. Its consuming methods are `close(self, …)` and `abandon(self, reason)`, both `async` and awaited.
  - Every `?`-error path in the handler calls `abandon` explicitly.
  - `Drop` without consumption only covers panic and cancellation, because the request future was dropped. It does `try_send(AbandonRequest { episode_id,`

`expected_seq, hold_id, reason: dropped })` on a bounded channel owned by the supervisor's `episode_closer` loop [P OD-19].
    - The closer dispatches `episode.abandon` (episode closed with `failure`, hold released) in one transaction.
    - If the channel is full or the process dies, the `episode_reaper` closes the episode after `EPISODE_MAX_RUNTIME` (default 30 min) [M R0.5].
    - **The record and its effect are always atomic** (`episode.closed` with `wallet.hold_released`); only the *trigger* has three tiers. This closes G07 [R §4 G07].
- **⟨12⟩–⟨13⟩** Each tool call is one short transaction appending `tool.invoked { iteration, tool_name, allowed, input_sha, output_sha, duration_ms, origin, locators[] }` plus its two artifacts [P OD-18]. For `ToolOrigin::External` tools (§6.4) it also writes the `external_sources` rows and `hadPrimarySource` edges. A crash mid-run therefore keeps durable evidence of every completed tool call. That evidence is what Presence cites, and "tool invocations are events" is the Phase 2 requirement [R §5 Phase 2].
- **⟨15⟩** The evidence and enforcement functions are pure and run outside any transaction. They read the tool-call artifacts *from memory* (the same bytes just stored), not from the database. The split is in §7.
- **⟨17⟩–⟨22⟩** TX-C is one `dispatch` of `episode.close` (§3.2.1):
    - `W = {episode:<eid>, wallet:<payer>, wallet:<platform>, wallet:<owner>?, claim:<cid₁>…}`;
    - `R = {agent:<id>}`.
    - If the hold has been released by the reaper, `Conflict` on `wallet:` or `episode:` aborts the close. The run's output is still returned to the client, marked `persisted: false`, and the stored `tool.invoked` events remain.
    - Nothing is charged twice, because the settle key is `episode:<eid>:settle`.
- **⟨24⟩** The response carries claim IDs in state `Submitted`. Under [M R7.7], consumers that rely on a claim MUST wait for `Proceeding`. The handler MAY wait on `NOTIFY abw_events` for up to `ROUTE_WAIT_MS` (default 0 for `/execute`, 15 000 for A2A `message:send`) and include the route when it arrives.

4.6 The episode as a stream



`Closing` is an in-memory phase with no event. A closed episode stream accepts no further events: `decide` rejects any command on a closed episode. Consolidation bookkeeping moves to `episode_consolidation` [M R3.4], and the `context` patch at `src/handlers/forecasts.rs:1559` [M R3.4] becomes its own event.

Event	Payload (v1)
`episode.opened`	`{episode_id, tenant_id, agent_id, version_number, card_sha, input_sha, source_ref: {kind: execute
`tool.invoked`	`{iteration, call_index, tool_name, tool_version?, allowed, origin: external
`evidence.collected`	`{doc_sha, contract_sha, mode: shadow
`gate.decided`	§8.1
`observation.recorded`	§8.4
`episode.closed`	`{status, failure_reason?, raw_output_sha?, enforced_output_sha?, tool_call_shas: [..], tokens_in, tokens_out, cost, embedding_model?, claim_ids: [..]}`

The `proj.episodes` projection is inline:

```
CREATE TABLE proj.episodes (
  episode_id          uuid PRIMARY KEY,
  stream_id           text NOT NULL UNIQUE COLLATE "C" REFERENCES ledger.streams ON
DELETE RESTRICT,
  tenant_id           text NOT NULL,
  agent_id            uuid NOT NULL,
  card_sha            text NOT NULL REFERENCES ledger.artifacts ON DELETE RESTRICT,
  version_number      int  NOT NULL,
  parent_episode_id   uuid REFERENCES proj.episodes ON DELETE RESTRICT,
  workspace_id        uuid,
  hold_id             uuid,
  status              text NOT NULL CHECK (status IN
('running','success','partial','failure')),
  failure_reason      text,
  input_sha           text NOT NULL REFERENCES ledger.artifacts ON DELETE RESTRICT,
  raw_output_sha      text REFERENCES ledger.artifacts ON DELETE RESTRICT,
  enforced_output_sha text REFERENCES ledger.artifacts ON DELETE RESTRICT,
  tool_call_shas      text[] NOT NULL DEFAULT '{}',
  cost_charged        bigint,
  opened_at           timestamptz NOT NULL,
  opened_pos          bigint NOT NULL REFERENCES ledger.events (global_pos),
  closed_pos          bigint REFERENCES ledger.events (global_pos),
  applied_pos         bigint NOT NULL REFERENCES ledger.events (global_pos),
  CHECK ((status = 'running') = (closed_pos IS NULL)));
CREATE INDEX episodes_agent_recent  ON proj.episodes (agent_id, opened_pos DESC);
CREATE INDEX episodes_tenant_recent ON proj.episodes (tenant_id, opened_pos DESC);
CREATE INDEX episodes_running       ON proj.episodes (opened_at) WHERE status =
'running';
CREATE CONSTRAINT TRIGGER t_episodes_justified AFTER INSERT OR UPDATE OR DELETE ON
proj.episodes
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('applied_pos');
```

The legacy `public.episodes` table (ADM memory: embedding, `consolidated`, `context` and `response_text`) is handled in two steps:

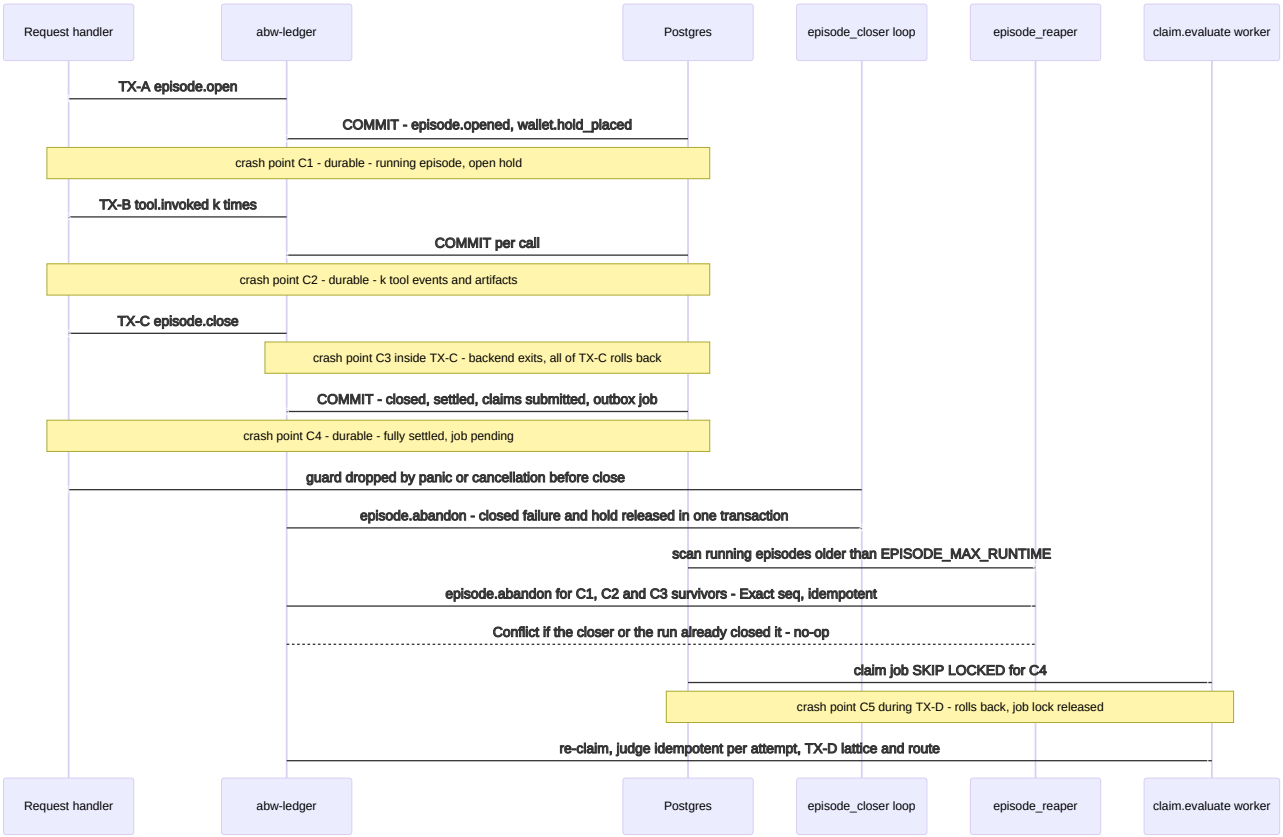
- **Phase 2.** It is written **only** by the `episode.close` inline projector, in TX-C, as a mirror keyed by `episode_id`, with FK to `proj.episodes`. Its embedding sidecar keeps the transactional pattern of `store.rs:305` [K] (the `embedding_provenance` row is inserted on the same connection). Its status enum gains `running` [R §4 G05]. No `.parse().unwrap()` remains on any column [R rule 3].
- **Phase 3.** Every reader moves to `proj.episodes` plus artifacts, and the mirrored trust columns are dropped (§14.3).

4.7 Crash-point analysis for one run

This is the target counterpart of LLD §7.3.

SIGKILL after	`episode:` stream	Money	Claims	Recovery
TX-A commit	`episode.opened`	Hold open	—	Reaper after 30 min: `episode.closed failure abandoned_timeout` + `wallet.hold_released`, one transaction
Mid-run, after k tool calls	opened + k × `tool.invoked`	Hold open	—	Same. The k tool artifacts remain citable
During TX-C	opened + k × tool	Hold open	None	TX-C rolled back entirely; reaper closes later. **No charge, no claim, no partial settle**
TX-C commit	Closed	Settled	`Submitted` + `commitment.bound` + outbox job	Worker evaluates
Worker mid-judge	Closed	Settled	`Submitted`	Job re-claimed. A second judge call costs money but is idempotent on `judge:<cid>:<attempt>`: only one `judge.reported` per attempt is accepted
During TX-D	Closed	Settled	`Submitted`	Rolled back; job re-claimed

The same crash points, frame by frame, with the recovery that follows each one:



Every recovery path appends through expected-version on the `episode:` or `claim:` stream:

- **⟨10⟩–⟨11⟩** are the closer path.
- **⟨12⟩–⟨14⟩** are the reaper path.
- **⟨15⟩–⟨16⟩** are the worker path.

When two recoverers race, the second gets `Conflict` and does nothing (⟨14⟩). The reaper, the closer and the run's own close are therefore mutually safe without any coordination

beyond the stream compare-and-swap.

---

5. Claims and the Verification Lattice

This section implements the Lattice spec exactly. Where the spec leaves a question open, the answer is in §5.16, taken from [M §15] when that agrees with [S] and [R], or marked [P].

Module layout. The crate `abw-lattice` has the modules below. The crate graph (`cargo` dependencies) enforces the separations of T8 and T15: an arrow `A -> B` means "A may depend on B"; no other edges are allowed.

Module	Responsibility	May depend on	MUST NOT depend on
`abw-evidence` (separate crate)	Collect evidence and write evidence events, artifacts and provenance edges	`abw-ledger`	`abw-lattice`, `abw-judge`
`abw-lattice::inputs`	Load `LatticeInputs` through `ReadDb` and hash them	`abw-ledger` (read), `abw-evidence` (types only)	`abw-judge`
`abw-lattice::evaluate`	Pure `evaluate()`: no I/O, no `async`, no clock, no RNG	`inputs` types	`abw-judge`, `abw-ledger`
`abw-judge` (separate crate)	Produce the interpretive report	`abw-ledger` (artifacts), provider clients	`abw-lattice::evaluate`
`abw-lattice::route`	Pure `route()`	`evaluate` output types, `abw-judge` report types, policy types	—
`abw-lattice::commands`	`claim.*` commands; the only writers of `lattice.evaluated` and `route.decided`	all of the above	—

`abw-evidence` cannot name a `VerdictVector` and `evaluate` cannot name a `JudgeReport`. These separations are therefore enforced by the compiler, not by convention.

5.1 The claim envelope

A claim is submitted into its own stream `claim:<cid>` by the `claim.submit` command. `cid` is a UUIDv7, so it is time-ordered and index-friendly. The canonical payload (`claim.submitted` v1) is shown in Rust form; the JSON Schema is derived from it.

```rust
pub struct ClaimSubmittedV1 {
    pub cid: Uuid,
    pub attempt: u32,                          // 1-based; resubmission increments
    pub tenant_id: TenantId,
    pub episode_id: Uuid,                      // the run that produced the claim
    pub agent: AgentRef,                       // { agent_id, version_number,
card_sha }
    pub proposer_model: ModelRef,              // { provider, model, family }  --
for judge independence
    pub task_ref: Option<Sha256Ref>,           // artifact: what was asked (input
artifact of the episode)
    pub task_issuer: Actor,                    // who issued the task (routing of
intent findings)
    pub content: Vec<ClaimField>,              // [{ pointer, value, match_method }]
    pub subject_artifact: Sha256Ref,           // enforced output artifact the
content points into
    pub artifact_refs: Vec<Sha256Ref>,         // every artifact the claim says it
is about
    pub cited_versions: Vec<CitedVersion>,     // [{ stream_id, seq }] state the
claim was derived from
    pub tool_call_refs: Vec<Sha256Ref>,        // tool-call artifacts the claim
cites as evidence
    pub binding: Option<BindingDeclV1>,        // { p_ppm, outcome_condition,
deadline }
    pub stakes: StakesV1,                      // { class: routine|high, reasons:
[StakesReason] }
    pub kind: ClaimKind,                       // assertion | forecast | publication
| assumption | description
    pub correction_of: Option<CorrectionRef>,  // { attempt, diagnoses_pos } on
resubmission
}
pub struct ClaimField { pub pointer: JsonPointer, pub value: serde_json::Value, pub
match_method: MatchMethod }
pub struct BindingDeclV1 { pub p_ppm: u32, pub outcome_condition: OutcomeCondition,
pub deadline: DateTime<Utc> }
pub enum OutcomeCondition {                    // closed set; each has a registered
resolver
    ForecastResolves { forecast_id: String, outcome: bool },        // Fermi
    MarketResolves { market_ref: String, outcome: String },         // Polymarket
    MetricWithin { metric_ref: String, lo: i64, hi: i64, scale: u32 }, // DPP /
carbon audit
    HumanAttests { role: String },
}
```

- **`content` restates the claimed fields.** It is the list of `(JSON pointer, value)` pairs the claim asserts, stated independently of the artifact. Truth diffs `content` against the

artifact. A claim whose `content` is empty has nothing to diff, so Truth is `not_evaluated` with class `no_claimed_fields`.

- **Every assertion becomes a claim.** The assertions an episode produces (`src/assertions.rs`, LLD §6.6) are submitted as claims in TX-C [M R7.2]:
  - `assertion_id` becomes `cid`;
  - `assertion_verifications` becomes an async read-model projection of `claim:` events (Phase 3, §14.3);
  - forecast multipliers become claims with `kind: forecast` (§10.1).
- **`kind: assumption`** is the Lattice spec's "declare the claim an assumption" corrective action for Grounding [S §Diagnosis]. An assumption claim's Grounding is evaluated as usual, so it will normally fail. The route table treats a failing Grounding on an `assumption` as acceptable at routine stakes (§5.10).
- **`kind: description`** is a non-binding description [S §Diagnosis, Binding row]. Binding is `not_evaluated` with class `non_binding_by_declaration`, and routing treats that as satisfying "Binding registered" only for `description` claims.

5.2 Dimensions: instruments, queries and cost

Every dimension is computed on every evaluation (T13). The physical instrument is given as the query `abw-lattice::inputs` runs. `evaluate()` then uses the loaded values without further I/O.

5.2.1 Presence: "Did the referenced work occur?"

Inputs	`artifact_refs`, `subject_artifact`, `tool_call_refs`, `episode_id`
Query	`SELECT sha FROM ledger.artifacts WHERE sha = ANY($refs)`. Episode status: `SELECT status, closed_pos FROM proj.episodes WHERE episode_id = $e`. Tool calls: `SELECT payload->>'output_sha' FROM ledger.events WHERE stream_id = 'episode:'
Index	`artifacts_pkey`, `episodes_pkey`, `events (stream_id, seq)`
Cost	`O(r log A)` for r refs; one range scan of the episode stream (≤ 40 rows)
Pass	Every ref resolves **and** the episode is closed with `status IN ('success','partial')` **and** every `tool_call_refs` SHA equals some `tool.invoked.output_sha` or `input_sha` on that episode stream
Fail classes	`artifact_missing` (diagnosis lists the unresolved SHAs); `reference_malformed` (not `^sha256:[0-9a-f]{64}$`, rejected earlier by schema, so listed for completeness); `episode_not_terminal`; `episode_failed`; `tool_call_not_in_episode`
`not_evaluated`	Never. Presence has no precondition
Deterministic	Yes

5.2.2 Liveness: "Is the claim still current?"

Inputs	`cited_versions` [(stream, seq)]
Query	Per cited stream: `SELECT head FROM ledger.streams WHERE stream_id = $s`. If `head > seq`: `SELECT global_pos, seq, event_type FROM ledger.events WHERE stream_id = $s AND seq > $seq AND seq <= $head AND event_type = ANY($conflicts)`, where `$conflicts = conflicts_with(kind)` is a declared table
Index	`streams_pkey`; `events (stream_id, seq)` range
Cost	`O(c log S + Σ (head − seq))`
Pass	For every cited stream, `head == seq` **or** no event of a conflicting type is in `(seq, head]`
Fail class	`superseded`. Diagnosis carries `{stream, cited_seq, head, conflicting: [{seq, event_type, global_pos}]}`. Suggested action: rebase on the current projection and re-derive [S §Diagnosis]
`not_evaluated`	`cited_versions` empty **and** the claim kind requires it (`assertion` over workspace state). Class `no_cited_version`. For kinds that cite nothing by construction (a forecast with no inputs), Liveness is `pass` on an empty set only when the claim kind's policy declares `liveness_vacuous_ok` [P OD-11]
Deterministic	Yes

`conflicts_with` is declared in `src/lattice/liveness.rs` [M R7.4]. For example: `agent → {agent.version_created}`; `wallet → {}`, because money appends do not invalidate a claim about history; `episode → {}`, because episodes are closed; `publication → {publication.released}`; `policy → {policy.adjusted}`. A cited CRUD-domain object is not a stream and **cannot** be cited: the claim must cite an artifact snapshot instead. A claim that needs Liveness over workspace state cites a `workspace_output:<id>` stream, which Phase 3 adds for authoritative workspace outputs [P OD-12].

5.2.3 Truth: "Does the content match the artifact's actual state?"

Inputs	`content` fields; bytes of `subject_artifact` (and any `artifact_refs` named by a field's pointer prefix `#<sha>/…`)
Query	`SELECT bytes, storage_ref, canonicalization FROM ledger.artifacts WHERE sha = $s`; object fetch if external
Cost	`O(f · d)` for f fields at pointer depth d, after `O(size)` parse
Per field	Resolve the JSON pointer (RFC 6901) in the artifact. Compare with the field's `MatchMethod`: `ExactJson` (JCS-equal), `NumericWithin { rel_bp }` (`
Score	`score_num = matched`, `score_den = total`
Pass	`score_num × 10000 ≥ theta_bp(truth) × score_den`
Fail class	`mismatch`. Diagnosis carries the field-level diff `[{pointer, claimed, observed, method}]`
`not_evaluated`	Presence did not pass (dependency, §5.3), class `presence_failed`; or `content` is empty, class `no_claimed_fields`; or the artifact is not JSON while fields have pointers, class `artifact_not_structured`
`unsupported`	Truth passed and Grounding failed (§5.3)
Result	Score against θ; per-field results are stored in the diagnosis and in `proj.claim_fields` [M §15]

5.2.4 Grounding: "Does it trace to a source outside the agent's say-so?"

Inputs	For each content field: the chain walk `origin_of(artifact: <subject_artifact>, pointer)` (§6.5)
Query	The recursive CTE of §6.5, run once per field. A batched variant runs one CTE seeded with all fields
Index	`edges_walk (subject, field_path, relation) INCLUDE (object, object_path, object_epistemic)` ; `nodes_origin` partial
Cost	`O(f · b^D)` worst case, for fan-out b and depth bound D = 16. In practice the chains of §6.3 are 2–4 edges with fan-out ≤ 3, so about 10 rows per field
Per field	The fold of the walk rows into `Origin { kind: External
Score	`score_num = fields with origin External` (+ `HumanAttested` when the policy for this stakes class sets `accept_human_attested`) / `score_den = total`
Pass	`score_num × 10000 ≥ theta_bp(grounding) × score_den`
Fail classes	`ungrounded` (no field reached an origin; diagnosis carries the node where each chain ended); `chain_broken` (a chain ended at an evidentiary node with no outgoing source edge after at least one hop)
`not_evaluated`	The producing agent has no field contract, so no `evidence.collected` exists for the episode (class `no_contract`), carried over from [M R4.4]; or `content` is empty (`no_claimed_fields`)
`unsupported`	Never. Grounding depends on nothing in the declared table
Interpretive	The judge report cannot appear: `edges_interpretive_unreachable` (§6.2) means no evidentiary node has an outgoing walkable edge to an interpretive node

5.2.5 Binding: "Does it create an obligation that can be checked later?"

Inputs	The claim stream's `commitment.bound` event, if any; the registered resolvers
Query	`SELECT payload FROM ledger.events WHERE stream_id = 'claim:'
Pass (at evaluation)	A `commitment.bound` exists with `p_ppm ∈ [0, 10⁶]`, an `outcome_condition` of a kind with a registered resolver, and `deadline > evaluated_at`, where `evaluated_at` is a recorded input, not the wall clock (§5.6)
Fail class	`unbound`. Diagnosis names the missing parts: `forecast`, `outcome_condition` or `deadline`. Suggested action: add them, or reclassify as `description` [S §Diagnosis]
`not_evaluated`	`kind = description` (class `non_binding_by_declaration`)
At outcome	Brier is computed when `outcome.observed` is appended (§5.12). It is a separate event, not part of the vector

5.3 Dependencies

Dependencies are declared in one table in `abw-lattice::evaluate`, applied **after** every dimension's raw result is computed. This preserves "lattice, not ladder" [S P2]:

```
pub const DEPENDENCIES: &[Dependency] = &[
    // "Truth requires Presence. With no artifact there is nothing to diff, so Truth
is not evaluated."
    Dependency { dim: Truth, on: Presence, when_on: NotPass, becomes: NotEvaluated,
class: "presence_failed" },
    // "If Truth passes and Grounding fails, Truth is unsupported"
    Dependency { dim: Truth, on: Grounding, when_on: Fail, becomes_if_pass:
Unsupported, class: "grounding_failed" },
];
// Liveness and Binding: independent.
```

The evaluation order is:

1. Compute all five raw results.
2. Apply the Presence → Truth rule.
3. Apply the Grounding → Truth rule, which only changes a `pass` to `unsupported`.

The raw Truth score is kept even when Truth is `unsupported` or `not_evaluated`. In the `not_evaluated` case it is `null`, because no diff could run. Routing and the outer loop can therefore see the dimension's own result. This is also what the spec's example shows: `{"dim":"truth","state":"unsupported","score":0.81,"theta":0.70}` [S §Event schema].

5.4 Verdict states

```
pub enum DimState { Pass, Fail, Unsupported, NotEvaluated }       // closed: T12
```

There is no other state. In particular:

- there is no `Undetermined` in the vector (that is gate vocabulary, §8);
- there is no `Skipped`;
- there is no `Error`.

An infrastructure error while loading inputs (an object store timeout, a walk that exceeds its statement timeout) is **not** a verdict. The evaluation transaction aborts and the job retries. After `max_attempts`, the claim receives `lattice.evaluated` with the affected dimension `not_evaluated` and class `infra_unavailable`. That evaluation is marked `infra_degraded: true`, which routing treats as not-pass (§5.10).

T12 is enforced in three places:

- the type (a four-variant enum);
- the event JSON Schema (`"enum": ["pass","fail","unsupported","not_evaluated"]`);
- the conformance test `lattice_missing_substrate_never_passes` (§15).

5.5 The verdict vector

```rust
pub struct VerdictVector {
    pub cid: Uuid,
    pub attempt: u32,
    pub dims: [DimVerdict; 5],                  // fixed order: presence, liveness,
truth, grounding, binding
    pub residual_risk: ResidualRisk,            // per dimension, 5.8
    pub theta_version: String,                  // "<policy_id>@<version>"
    pub policy_doc_sha: Sha256Ref,
    pub inputs_sha: Sha256Ref,                  // sha256(JCS(LatticeInputs)), 5.6
    pub evaluator_version: String,              // abw-lattice crate version + git
sha of evaluate.rs
    pub evaluated_at: DateTime<Utc>,            // an INPUT (recorded), not read from
the clock inside evaluate
    pub infra_degraded: bool,
}
pub struct DimVerdict {
    pub dim: Dimension, pub state: DimState,
    pub score: Option<Ratio>,                   // { num: u32, den: u32 }
    pub theta_bp: Option<u16>,
    pub raw_state: DimState,                    // before dependency rules
    pub diagnosis: Option<Diagnosis>,           // Some iff state != Pass
    pub fields: Vec<FieldResult>,               // per-field detail for Truth and
Grounding
}
pub struct Diagnosis {
    pub dim: Dimension, pub class: FailureClass, pub evidence_ref: Option<String>,
// "sha256:…" | "ξ:<node_id>" | "ev:<global_pos>"
    pub expected: serde_json::Value, pub observed: serde_json::Value,
    pub suggested_action: SuggestedAction, pub budget_left: u32,
}
```

`Diagnosis` holds exactly the fields the spec requires: "dimension, failure class, evidence reference, expected, observed, suggested action, and remaining correction budget" [S §Diagnosis]. A non-pass without a diagnosis is a type error, because the event constructor takes a `NonPass { state, diagnosis }` variant (T14).

5.6 Reproducibility

```rust
pub struct LatticeInputs {
    pub claim: ClaimSubmittedV1,
    pub artifacts: BTreeMap<Sha256Ref, ArtifactPresence>,     // presence + bytes
sha (bytes fetched by sha on replay)
    pub episode: EpisodeFacts,                                // status, closed_pos,
tool output shas
    pub stream_heads: BTreeMap<StreamId, StreamFacts>,        // head + conflicting
events in (seq, head]
    pub provenance: BTreeMap<JsonPointer, WalkRows>,          // raw rows of the
chain walk per field, with edge_ids
    pub commitment: Option<CommitmentBoundV1>,
    pub resolvers: BTreeSet<OutcomeConditionKind>,
    pub policy: PolicyRef,                                    // { policy_id,
version, doc_sha }
    pub evaluated_at: DateTime<Utc>,
    pub prov_high_water: GlobalPos,                           // max event_pos of
any prov edge read
}
pub fn evaluate(inputs: &LatticeInputs, policy: &LatticePolicyDoc) -> VerdictVector;
// pure, total
```

The reproducibility contract (T11):

1. `abw-lattice::inputs::load` reads every input in **one `REPEATABLE READ` transaction** (TX-D), so the inputs form one consistent snapshot. All reads are through the same transaction that later appends (§5.11).

2. `inputs_sha = sha256_jcs(LatticeInputs)` is stored in `lattice.evaluated`, and the full `LatticeInputs` is stored as an artifact (`media_type = application/vnd.abw.lattice-inputs+json`) referenced by `inputs_sha`.

3. **Replay** (`GET /api/claims/:cid/replay`, [M R7.12]) loads the inputs artifact by SHA and the policy document by `(policy_id, version)`, verifies `doc_sha`, runs `evaluate`, and asserts `sha256_jcs(vector) == lattice.evaluated.payload_sha`. Identity of the payload hash is the bit-for-bit criterion.

4. **Re-derivation** (`?mode=rederive`) re-runs `inputs::load` *as of* the original evaluation and compares the result with the stored inputs:

   - Stream heads are taken from `ledger.events` with `global_pos ≤ the evaluation's own global_pos`.
   - Provenance is taken from `prov.edges` with `event_pos ≤ prov_high_water`.

   Because events, artifacts and edges are immutable (T6), re-derivation reproduces the inputs. A mismatch proves a defect in `load`. This is the stronger form of the directive's acceptance test "Replay: the verdict vector is reproduced exactly from (claim, artifacts, stream heads, provenance graph, θ version)" [R §5 Phase 2].

5. **Purity of `evaluate`.**
    - `evaluate` is not `async`, takes no `ReadDb`, and uses only integer arithmetic for scores and comparisons (§3.6.3).
    - It is compiled with `#![forbid(unsafe_code)]`.
    - A `clippy.toml` `disallowed-methods` list bans `SystemTime::now`, `Utc::now`, `rand::*`, `HashMap` iteration in output position, and `f64` comparisons in `evaluate.rs` [P OD-13].

5.7 The `lattice.evaluated` event

There is **one event per evaluation, carrying the whole vector** [S §Event schema]. Per-dimension spans MAY be emitted to tracing as metadata only, never as events.


```
{"type":"lattice.evaluated","schema_version":1,
 "cid":"0192d6c8-7e1a-7b3c-9a4e-5f6a7b8c9d0e","attempt":2,
 "theta_version":"5b9e0c7e-3d0f-4a51-9e8a-0f2d3c4b5a69@3",
 "policy_doc_sha":"sha256:…","inputs_sha":"sha256:…",
 "evaluator_version":"abw-lattice 1.0.0+3f2a91c","evaluated_at":"2027-02-
01T10:00:00Z",
 "infra_degraded":false,
 "vector":[
  {"dim":"presence","state":"pass","raw_state":"pass"},
  {"dim":"liveness","state":"pass","raw_state":"pass"},

{"dim":"truth","state":"unsupported","raw_state":"pass","score_num":17,"score_den":2
1,"theta_bp":7000,
   "diagnosis":
{"class":"grounding_failed","evidence_ref":"ev:88121","expected":"grounded content",
    "observed":"accurate but ungrounded","suggested_action":
{"kind":"attach_external_source"},"budget_left":2}},

{"dim":"grounding","state":"fail","raw_state":"fail","score_num":11,"score_den":21,"
theta_bp":6500,
   "diagnosis":{"class":"chain_broken","evidence_ref":"ξ:artifact:sha256:7f3a…",
    "expected":"external origin","observed":"agent output at step 2",
    "suggested_action":{"kind":"attach_source","hint":"attach sensor log for run
22"},"budget_left":2}},
  {"dim":"binding","state":"pass","raw_state":"pass"}],
 "residual_risk":{"presence":{"num":1,"den":500,"prior_only":false},"liveness":
{"num":1,"den":250,"prior_only":false},
                  "truth":{"num":3,"den":100,"prior_only":false},"grounding":
{"num":1,"den":20,"prior_only":false},
                  "binding":{"num":1,"den":2,"prior_only":true}}}
```

This is the spec's example [S §Event schema] re-encoded with integer scores and the reproducibility fields. `budget_left` in each diagnosis is the per-claim budget *after* this evaluation's correction, if routing chooses Correct.

5.8 Residual risk

"A fully passing vector carries a per-dimension false-pass rate, not a guarantee" [S P5].

- **Population.** Residual risk is computed by the outer-loop projection (§11.1) for each `(dimension, policy_id, stakes_class)`, over claims where that dimension was `pass` and an outcome or a human decision later established truth.
- **Formula.** It is the Beta posterior mean with a Beta(1,1) prior:
  - `false_pass = false_pass_count + 1`;
  - `denominator = checked_pass_count + 2`.

  It is stored as a ratio. With fewer than 30 checked passes it is labelled `prior_only: true` [M R7.4].
- **Reproducibility.** The value in a vector is read from the read model *as of* the evaluation transaction's snapshot and copied into `LatticeInputs`. It is therefore reproducible without recomputing the read model.

Async read models. The read models named here and in §11.1 (`proj.loop_inner`, `proj.calibration`, `proj.loop_outer`) are given as column lists, not DDL.

- They are async projections: rebuildable, and written only by their projector through `ProjTx` with a checkpoint (§3.5).
- Each row is keyed by its grouping columns plus `applied_pos`.
- No trust decision reads them, except through a value copied into `LatticeInputs`.

5.9 The judge report

The judge is mandatory, and its output is interpretive [S §Judge].

Findings have exactly the spec's kinds: `intent_adequacy`, `plausibility`, `truth_cross_check` (only when given the artifact), `binding_observability` and `draft_diagnosis`. Each finding carries `kind`, `severity ∈ {info, warn, block}`, `confidence_ppm`, `text` and, for `intent_adequacy`, `cause ∈ {proposer, task_ambiguous}` [M §15].

`judge.reported` v1:

```
{"cid":"…","attempt":2,"type":"interpretive",
 "findings":
[{"kind":"intent_adequacy","severity":"warn","confidence_ppm":720000,"cause":"propos
er","text":"…"}],
 "judge_model":{"provider":"openrouter","model":"…","family":"openai-gpt"},
 "prompt_sha":"sha256:…","prompt_lineage":"judge-v3",
 "inputs":{"task_sha":"sha256:…","claim_sha":"sha256:…","artifact_sha":"sha256:…"},
 "report_sha":"sha256:…","nonce_sha":"sha256:…"}
```


The `type` field is constant (`"const": "interpretive"` in the schema). The `event_types` row for `judge.reported` has `epistemic = 'interpretive'`. The report artifact is written with a `prov.nodes` row `epistemic = 'interpretive'` (§6.2).

Independence rules:

1. **Model family.**
   - Families are declared data in the lattice policy (`judge_policy.families`): `anthropic-claude` , `openai-gpt` , `mistral` , `qwen` , `glm` , … [M R7.6].
   - The judge for a claim MUST be from a different family than `claim.proposer_model.family` . That family is taken from `episode.opened.model_family` , which records what actually ran, not the card's declared model.
   - The current judge default, `claude-haiku-4-5-20251001` (`src/handlers/eval_judge.rs:37` [K: to be replaced]), is the same string as the default agent model, so the current default fails this rule.
2. **Prompt lineage.**
   - `prompt_lineage` is an ID declared in `abw-judge` , and `prompt_sha` hashes the full system prompt.
   - The lineage MUST NOT be derived from any proposer card: the judge prompt is not built from the proposer's `system_prompt` , and `prompt_sha ∉ {card prompt_sha of the proposer}` .
   - Both conditions are checked in `judge.report` command `decide` [M §15].
3. **Injection-safe delimiting.**
   - The task, claim and artifact enter the prompt only as JSON **string values** (`serde_json` escaping) inside a block delimited by `<<ABW-DATA-{nonce}>>` … `<<END-ABW-DATA-{nonce}>>` , where `nonce` is 128 bits from `OsRng` , generated per call.
   - The system prompt says that content inside the block is untrusted data and that instructions inside it are to be reported as findings, not followed.
   - A payload containing the nonce string is impossible by construction: the nonce is generated after the payload is fixed, and a collision is checked and regenerated.
   - The current direct interpolation `AGENT OUTPUT:\n{}` (`eval_judge.rs:99` [K: to be replaced]) is removed [R §4 G12].

- `nonce_sha` (not the nonce) is stored, so the prompt is reproducible from stored parts given the nonce, which is kept in the report artifact.

4. **Unavailable judge.** If no independent judge is configured or reachable within `JUDGE_TIMEOUT` (60 s) after `max_attempts` (3), the worker appends `judge.unavailable { cid, attempt, reason_class: infra_error|insufficient_input }`. Routing treats this as a `block` intent finding with `confidence_ppm = 0`, so the claim goes to a human rather than Proceed [M R7.6].
5. **No certifying authority.** `evaluate()` has no parameter of any judge type (§5 module table). Nothing in the report can raise a dimension.

Measured reliability. The middle loop scores findings against outcomes (§11.1). The subject of `calibration.updated` is `judge:<family>/<model>@<prompt_lineage>`.

Injection test. `tests/claims/judge_injection.rs` runs a fixed corpus of injection payloads with a fixed seed. It asserts that the finding severity flip rate relative to a clean control is at most the threshold in OD-14 [M §16.6].

5.10 Routing

```
pub enum Action { Proceed, SignOff, Correct, Endorse, Park }
pub struct RouteDecision {
    pub action: Action, pub policy_id: Uuid, pub policy_version: u32, pub rule_id:
String,
    pub reason: String, pub assignee: Option<Actor>, pub correction_target:
Option<CorrectionTarget>,
}
/// Pure, total.
pub fn route(v: &VerdictVector, judge: &JudgeOutcome, claim: &ClaimFacts, policy:
&LatticePolicyDoc) -> RouteDecision;
pub enum JudgeOutcome { Reported(JudgeReportedV1), Unavailable(JudgeUnavailableV1) }
```

Routing **waits for both** the vector and the judge outcome [S §Routing]. The rules are evaluated **in order**, and the first match wins. This is a policy table in `LatticePolicyDoc.routes`, versioned with θ.

Rule	Condition	Action
R0	`infra_degraded`	**Park** if budget exhausted, else **Correct** with target `platform` (re-evaluation job, no agent call
R1	Judge has a `block` finding of kind `intent_adequacy` (or `judge.unavailable`)	**Correct**; target = `task_issuer` `cause = task_ambiguous`, else proposer [M §15]. Park if budge exhausted. If `judge.unavailable`, **SignOff** to human instead of Correct, since no agent can act on it
R2	Budget exhausted and any dimension not `pass`	**Park**
R3	Any of Presence, Liveness, Truth, Binding is `fail` or `not_evaluated`, except Binding `not_evaluated(non_binding_by_declaration)`	**Correct**
R4	Grounding `fail` and `kind = assumption` and stakes routine	Treat Grounding as accepted; continue to R6/R7
R5	Grounding `fail` or `not_evaluated`	**Endorse** if the diagnosis's `suggested_action` is `no_external_source_obtainabl` or the correction history shows one prior Grounding correction already failed; otherwise **Corre**
R6	All pass (Truth `pass`, not `unsupported`) and stakes `high`	**SignOff**

Rule	Condition	Action
R7	All pass and Binding registered (or `description`) and stakes routine	**Proceed**

Truth `unsupported` always implies Grounding `fail`, so it is handled by R5.

Stakes classes. These are the platform defaults [M §15]:

- `irreversible_external_effect`;
- `money_transfer_above` `SIGNOFF_CREDIT_THRESHOLD`;
- `dpp_publication`;
- `tenant_flagged`.

Tenants extend the list in their policy's `stakes_rules`. Stakes are computed by `claim.submit`'s `decide` from the claim kind and the tenant policy, and recorded on `claim.submitted.stakes`, so they are an input, not a judgement made at route time.

`route.decided` v1: `{cid, attempt, action, policy_id, policy_version, rule_id, reason, assignee?, correction_target?, vector_pos, judge_pos}`.

`lattice.evaluated` and `route.decided` commit in **one transaction** [R §5 Phase 2]. The routing consequences commit in that same transaction: `correction.requested`, `endorsement.requested` or `claim.parked`.

5.11 Transaction boundaries

TX-D is the claim evaluation transaction: a `REPEATABLE READ` dispatch of `claim.evaluate`, run by the `claim.evaluate` outbox worker.

1. `BEGIN ISOLATION LEVEL REPEATABLE READ`.
2. `lock_streams(W = {claim:<cid>}, R = {episode:<eid>} ∪ cited streams)`.
3. `inputs::load`: every read of §5.2, in this snapshot.
4. Check that `judge.reported` or `judge.unavailable` exists for `(cid, attempt)`. If it does not, `ROLLBACK`; the judge job will re-enqueue `claim.evaluate` when it finishes.
5. `evaluate` (pure), then `route` (pure).
6. Append `lattice.evaluated`, `route.decided`, and the consequence events. Store the `LatticeInputs` artifact and the packet artifact if routing chose Endorse or SignOff.
7. Inline projection `proj.claims`.
8. Outbox jobs: `claim.correct` (re-invoke the agent), `notify.assignee`, `notify.owner`.
9. `COMMIT`.

Why `REPEATABLE READ` here (and READ COMMITTED for other commands):

- The inputs span several tables. Liveness compares a cited seq with heads that may move during the load. A single snapshot makes the inputs self-consistent, so replay

reproduces them.
- The streams in `R` are share-locked, so a conflicting append to a cited stream waits until TX-D commits. The vector cannot be invalidated between load and commit.
- An append to an *uncited* stream is irrelevant by definition.

Judge before lattice. The judge runs **outside** any transaction (an LLM call; §2.3). It writes `judge.reported` in its own short transaction on `claim:<cid>` and then enqueues `claim.evaluate`. The lattice evaluation needs no LLM, so placing it after the judge costs nothing and satisfies "routing waits for both".

Inline fast path. When the judge is fast (≤ `ROUTE_WAIT_MS`), the request handler MAY run the judge itself after TX-C and dispatch `claim.evaluate` directly. The transaction shape is identical.

5.12 Binding and outcome

- `commitment.bound` is appended in TX-C, when the claim carries `binding`, with payload `{cid, attempt, p_ppm, outcome_condition, deadline, conditional}` [S §Event schema].
  - `conditional = true` when the vector later fails. This is decided at TX-D: if the route is not Proceed or SignOff, TX-D appends `commitment.conditioned { cid, attempt }` [P OD-16].
  - Conditional commitments are scored but excluded from calibration until the claim reaches `Proceeding` [M §15].
- **Outcome.** `outcome.observed { cid, ok: bool|null, evidence_ref, resolver, observed_at, class? }` is appended by the `outcome_observer` worker. The same transaction appends:
  - `calibration.updated` for the proposer, every endorser of the claim, and the judge (§11.1);
  - `false_pass.detected` when `ok = false` and some dimension was `pass` (one event per such dimension, `source: outcome`).
- **Brier per claim.** `brier_micro = (p_ppm − 1e6·ok)² / 1e6`, computed with integer arithmetic from this claim's own `p_ppm`. This replaces the current copy of the forecast's Brier to every contributing agent (`src/outcome_trust.rs:24-36` [K: defect described there]) [M R7.11].
- **Deadline passed, unresolved.** `outcome.observed { ok: null, class: deadline_passed_unresolved }`. The claim is scored as unresolved (no Brier), and Binding's residual risk counts it as a binding failure.

5.13 Correction budget, endorsement and sign-off packets

Correction budget. It is **per claim**, with default 3, set in policy (`correction_budget`) [M §15]. `budget_left` is folded from the claim stream: `budget − count(correction.requested)`. When it reaches 0, R2 parks the claim.

- **`claim.parked`** `{cid, diagnosis_ref, owner}` commits with `notify.owner` in the outbox. The claim is never dropped [S §Diagnosis].
- **Resuming a parked claim.** The owner resubmits with a new budget (`claim.resubmit { budget_grant }` , actor human). The state machine below allows only that transition out of `Parked` .

Correction loop. `correction.requested { cid, attempt, dims, diagnoses_pos, budget_left, target }` :

- For `target = proposer` , the outbox job `claim.correct` re-invokes the agent:
  - with the diagnoses and the judge's `draft_diagnosis` appended as structured feedback;
  - under a new episode with `parent_episode_id = <original>` .
- The new episode's TX-C submits `claim.resubmit { cid, attempt: n+1, correction_of }` into the **same** claim stream.
- For `target = task_issuer` , the job notifies the issuer and waits for a human `claim.resubmit` .

Packets. `endorsement.requested { cid, attempt, kind: endorse|sign_off, packet_sha, assignee }` . The packet is an artifact (`application/vnd.abw.claim-packet+json`) containing exactly what the spec lists [S §Endorsement]:

- the claim;
- the vector with scores and θ;
- the diagnoses;
- the correction history (all prior `lattice.evaluated` and `correction.requested` events);
- the residual risk;
- the proposing agent's calibration (middle-loop posterior, as of the snapshot);
- the judge report;
- the action requested.

A packet is a value, so a human decides on a specific `packet_sha` .

Assignee selection. The assignee is the tenant's endorser pool for the claim's stakes class, with the least open packets. It MUST NOT be the claim's proposer's owner when the stakes are `high` (four-eyes) [P OD-17]. The selection respects the `human_load_ceiling` (§11.3). If every eligible assignee is at the ceiling, the claim goes to `Parked` with class `human_capacity` .

`human.decided` `{cid, attempt, kind, decision: endorse|sign|return|decline, dims, note, actor, packet_sha}` is a command with:

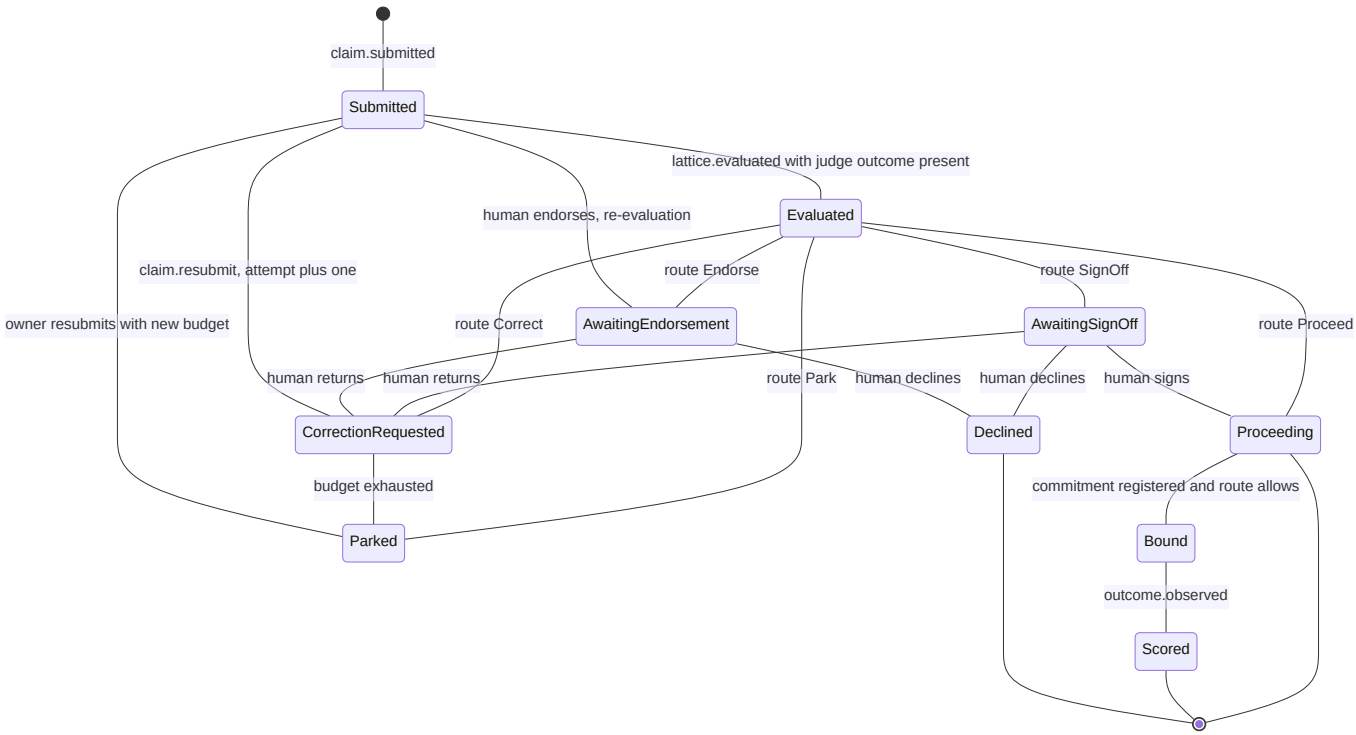
- `Expected::Exact(seq)` , where `seq` is the claim stream head at packet creation;
- the human's `packet_sha` MUST equal the packet in `endorsement.requested` .

A claim that changed under the human therefore returns `409`. A note is required for `return` and `decline`.

Effects of a human decision (same transaction):

Decision	Effects
`endorse` (kind endorse)	`prov.nodes` row `source:<uuid>` with `origin_kind = 'human_attested'`, plus `quotedFrom` / `hadPrimarySource` edges from the claim's subject fields to it, plus `wasAttributedTo` to `human:<actor>`. The claim's Grounding is re-evaluated in a follow-on TX-D, so the new origin is visible, and routes under the policy's `accept_human_attested` rule [M §15]
`sign` (kind sign_off)	Route to `Proceeding`. `human.decided` is the authorisation; the downstream command (e.g. `publication.release`) cites its `global_pos`
`return`	`correction.requested { target: proposer, from_human: true }`
`decline`	Claim to `Declined`. If the vector was clean: `false_pass.detected { dim: *, source: human }` as a **candidate** (`confirmed: false`). Endorsing a failed vector appends `false_fail.detected { dims, source: human, confirmed: false }` [S §Endorsement]

Claim lifecycle:



`Evaluated` is transient: it has no stored state and appears only inside TX-D, between `lattice.evaluated` and `route.decided`. The projection never shows it. `Bound` is reached at TX-D when Binding passed and the route is Proceed or SignOff. Illegal transitions are `Rejection`s in `decide` [M R7.7].

Claim projections (inline):

```
CREATE TABLE proj.claims (
  cid             uuid PRIMARY KEY,
  stream_id       text NOT NULL UNIQUE COLLATE "C" REFERENCES ledger.streams ON
DELETE RESTRICT,
  tenant_id       text NOT NULL,
  episode_id      uuid NOT NULL REFERENCES proj.episodes ON DELETE RESTRICT,
  agent_id        uuid NOT NULL,
  card_sha        text NOT NULL,
  kind            text NOT NULL CHECK (kind IN
('assertion','forecast','publication','assumption','description')),
  attempt         int  NOT NULL CHECK (attempt >= 1),
  state           text NOT NULL CHECK (state IN
('Submitted','Proceeding','AwaitingSignOff','CorrectionRequested',
                    'AwaitingEndorsement','Parked','Declined','Bound','Scored')),
  stakes          text NOT NULL CHECK (stakes IN ('routine','high')),
  budget_left     int  NOT NULL CHECK (budget_left >= 0),
  last_vector_pos bigint REFERENCES ledger.events (global_pos),
  last_route      text CHECK (last_route IN
('Proceed','SignOff','Correct','Endorse','Park')),
  theta_version   text,
  assignee        text,
  deadline        timestamptz,
  applied_pos     bigint NOT NULL REFERENCES ledger.events (global_pos));
CREATE INDEX claims_queue ON proj.claims (tenant_id, state, assignee)
  WHERE state IN ('AwaitingEndorsement','AwaitingSignOff','Parked');
CREATE INDEX claims_due ON proj.claims (deadline) WHERE state = 'Bound';
CREATE INDEX claims_episode ON proj.claims (episode_id);
CREATE CONSTRAINT TRIGGER t_claims_justified AFTER INSERT OR UPDATE OR DELETE ON
proj.claims
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('applied_pos');

CREATE TABLE proj.claim_fields (                    -- per-field Truth and
Grounding results, per evaluation
  cid         uuid NOT NULL REFERENCES proj.claims ON DELETE RESTRICT,
  attempt     int  NOT NULL,
  vector_pos  bigint NOT NULL REFERENCES ledger.events (global_pos),
  pointer     text NOT NULL,
  truth       text NOT NULL CHECK (truth IN
('match','mismatch','absent','not_evaluated')),
  origin_kind text NOT NULL CHECK (origin_kind IN
('external','human_attested','platform_record','none')),
  origin_node text,
  PRIMARY KEY (vector_pos, pointer));
CREATE TRIGGER t_claim_fields_immutable BEFORE UPDATE OR DELETE ON proj.claim_fields
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
CREATE CONSTRAINT TRIGGER t_claim_fields_justified AFTER INSERT ON proj.claim_fields
```

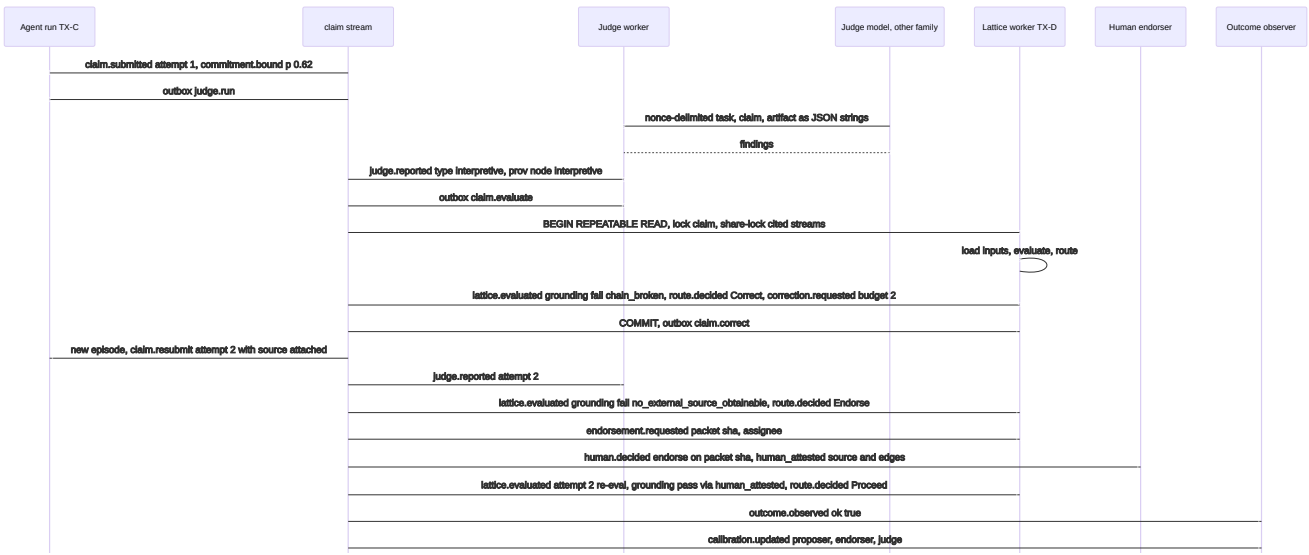
```
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('vector_pos');
```

What Proceed gates. Downstream consumers of a claim MUST check, in their own `decide`, that the claim is `Proceeding` or `Bound`, with a `FOR SHARE` lock on the `claim:` stream [M R7.7]. The consumers are:

- publication;
- cascade application;
- forecast multipliers;
- authoritative workspace outputs.

This is what makes routing a control (T7), not a record.

5.14 One claim end to end



In frame ⟨15⟩ the human attestation is evidentiary, because the human is a source by the spec's definition. The judge report written at ⟨5⟩ is not: the Grounding walks at ⟨8⟩ and ⟨16⟩ can never reach it (§6.2).

5.15 Event catalogue

These are all the events of the Lattice spec [S §Event schema], plus the events this design adds (marked **+**). Payloads are v1. Common envelope fields (`command_id` , `correlation_id` , `causation_pos` , `actor` , `actor_kind`) are in the `events` columns, not the payload. All probabilities are `*_ppm` , all thresholds `*_bp` , all scores ratios (§3.6.3).

Event	Stream	Epistemic	Payload (v1)
`claim.submitted`	`claim`	evidentiary	§5.1 `ClaimSubmittedV1` (spec: cid, agent, attempt, claim, refs, cited stream version forecast p, outcome condition, deadline)
+ `claim.resubmitted`	`claim`	evidentiary	Same shape as `claim.submitted`, `attempt = n+1`, `correction_of`, optional `budget_grant` (own only)
`lattice.evaluated`	`claim`	evidentiary	§5.7
`judge.reported`	`claim`	**interpretive**	§5.9
+ `judge.unavailable`	`claim`	observation	`{cid, attempt, reason_class, attempts}`
`route.decided`	`claim`	decision	§5.10
`correction.requested`	`claim`	decision	`{cid, attempt, dims, diagnoses_pos, budget_left, target: proposer
`endorsement.requested`	`claim`	decision	`{cid, attempt, kind: endorse
`human.decided`	`claim`	decision	`{cid, attempt, kind, decision: endorse
`claim.parked`	`claim`	decision	`{cid, attempt, diagnosis_ref, owner class: budget_exhausted
`commitment.bound`	`claim`	evidentiary	`{cid, attempt, p_ppm, outcome_condition, deadline, conditional}`
+ `commitment.conditioned`	`claim`	decision	`{cid, attempt}`

Event	Stream	Epistemic	Payload (v1)
`outcome.observed`	`claim`	evidentiary	`{cid, ok: bool
`calibration.updated`	`calibration`	evidentiary	`{subject: agent:
`false_pass.detected`	`claim`	evidentiary	`{cid, dim, source: outcome
`false_fail.detected`	`claim`	evidentiary	`{cid, dim, source: human
+ `policy.proposed`	`policy`	observation	`{policy_id, from_version, diff, cause, metrics_snapshot_sl ceiling_check: {ok, detail}}`
`policy.adjusted`	`policy`	decision	`{policy_id, versio from, to, cause, doc_sha, accepted_l rationale, proposal_pos?}`. Spe fields: `dim or route from, to, cause`
+ `episode.opened`, `tool.invoked`, `evidence.collected`, `episode.closed`	`episode`	evidentiary	§4.6
+ `gate.decided`	subject stream	decision	§8.1
+ `observation.recorded`	subject stream	observation	§8.4
+ `agent.version_created`, `agent.retired`	`agent`	structural	§4.2
+ `wallet.*`	`wallet`	evidentiary	§9.1
+ `publication.released`, `publication.token_issued`	`publication`	evidentiary	§10.2
+ `platform_health.observed`	`telemetry`	observation	`{rung: schema_present
+ `a2a.task_state`, `a2a.webhook_delivered`	`episode`	evidentiary	§12.4

Event	Stream	Epistemic	Payload (v1)
+ `source.resolved`	`episode`	evidentiary	`{source_id, status content_sha?, titl http_status}` (§6.4)
+ `forecast.revised`, `forecast.multiplier_applied`, `forecast.resolved`	`forecast`	evidentiary	`{forecast_id, revision_sha}`; `{forecast_id, cid, driver, value, claim_state_pos}`; `{forecast_id, outcome, resolver, brier_micro}` (§10.1
+ `workspace.spawned`, `workspace.git_committed`, `workspace.action_recorded`	`workspace`	structural / evidentiary	`{app_slug, manifest_sha, template_sha, funding_pos}`; `{commit, paths: [{path, sha}]}`; `{action_id, episode_id, kind, result_sha}` (§9.6, §10.2)
+ `rule.extracted`	`episode` (of the consolidation run)	evidentiary	`{rule_id, source_episodes: [..], provenance_floor}` (§6.6)
+ `hitl.intervention_requested`, `hitl.intervention_executed`	`intervention`	decision	`{intervention_id, encoded_sha, first_decision_pos` `{intervention_id, decision_positions [replay, reeval]}` (§8.3)
+ `gate.reviewed`	subject stream	decision	`{decision_pos, verdi upheld

Additional stream kinds are used by these events: `calibration`, `forecast`, `workspace` and `intervention`. All have exact concurrency and are registered with the kinds of §3.1.2.

Commands and outbox topics are not events. They share the dotted naming but live in different namespaces:

- `Command::ID` values, used by `Ledger::dispatch`;
- `ledger.outbox.topic` values.

Commands (`Command::ID`)	Outbox topics
`episode.open`, `episode.close`, `episode.abandon`, `tool.record`, `claim.submit`, `claim.resubmit`, `claim.evaluate`, `judge.report`, `human.decide`, `outcome.record`, `policy.propose`, `policy.accept`, `agent.update`, `agent.seed`, `agent.revert`, `agent.retire`, `agent.publish`, `wallet.hold`, `wallet.settle`, `wallet.deposit`, `wallet.transfer`, `app.spawn`, `publication.release`, `forecast.apply_claim`, `gate.review`, `hitl.intervene`, `hitl.confirm`	`judge.run`, `claim.evaluate`, `claim.correct`, `source.resolve`, `git.commit`, `a2a.webhook`, `a2a.execute` (deferred), `notify.assignee`, `notify.owner`

Every trust command is registered in `src/command_registry.rs`. The registry becomes the dispatcher's table, not a declaration [M R2.4]; `tests/claims/trust_commands_dispatch.rs` asserts that every registry entry has a `Command` impl.

The `calibration` stream kind is `calibration:<subject>`, with exact concurrency, one stream per subject. It is a derived stream written in the `outcome.observed` transaction, so the middle-loop posterior is append-only and replayable.

5.16 Answers to the Lattice spec's open questions

#	Open question [S §Open questions]	Answer	Source
1	Should Truth take an `unsupported` state, or should routing read Truth and Grounding together and leave both raw?	**Both.** `state` applies the dependency (`unsupported`), and `raw_state` keeps the raw result. Routing reads `state`	[M §15] for `unsupported`; `raw_state` is [P OD-20]
2	May Binding register on a vector with failures, as a conditional commitment?	**Yes.** `commitment.bound` at submit, `commitment.conditioned` when the route is not Proceed or SignOff. Scored, but excluded from calibration until `Proceeding`	[M §15]; event shape [P OD-16]
3	Is the correction budget per claim or per dimension?	**Per claim**, default 3	[M §15]
4	Which stakes classes trigger sign-off, and who defines them per tenant?	Platform defaults: `irreversible_external_effect`, `money_transfer_above`, `dpp_publication`, `tenant_flagged`. Tenants extend through `stakes_rules` in their policy version (human-accepted)	[M §15]
5	Does a human attestation count as a Grounding pass with source human, or as a separate attested state?	**A separate origin kind,** **`HumanAttested`**. Grounding passes on it only where policy sets `accept_human_attested` for the stakes class. The verdict vocabulary stays closed (T12): no new dimension state	[M §15]
6	Is Truth one continuous score or a pass/fail per claimed field?	**Both.** Per-field results are stored; the score is the matched fraction, compared with θ	[M §15]
7	What ceiling on human load does the outer loop respect, and who sets it?	`human_load_ceiling` in policy: default 50 open packets per assignee, 200 per tenant. Set by the tenant's policy owner, accepted as a policy version	[M §15]; §11.3

#	Open question [S §Open questions]	Answer	Source
8	Does a blocking intent finding go back to the proposer, or to whoever issued the task?	To the **task issuer** when `cause = task_ambiguous`, else to the proposer	[M §15]
9	Which judge model and prompt lineage count as independent of each proposer?	A **different model family** and a different prompt lineage, neither derived from the proposer's card	[M §15]

---

6. Provenance (ΞPROV)

ΞPROV is the Lattice spec's provenance trail [S §Dimensions, Grounding row]. The target implements it with the W3C PROV-DM vocabulary [M §8], stored in `prov.nodes` and `prov.edges`, and written by `abw-evidence` (and by the ledger for human acts) in the transaction of the event that causes it (T5).

6.1 Node and edge model

Nodes are typed references. The `node_id` prefix determines the PROV type:

`node_id` form	`prov_type`	`epistemic`	`origin_kind`	Create
`artifact:sha256:…`	entity	evidentiary, or **interpretive** for judge reports and LLM evaluator narratives	NULL	Any artifact ci event
`episode:<uuid>`	activity	evidentiary	NULL	`episode.oper`
`toolcall:sha256:…` (the tool-call artifact SHA)	activity	evidentiary	NULL	`tool.invoked`
`source:<uuid>`	entity	evidentiary	`external` or `human_attested`	`tool.invoked` tools), `human` (endorse)
`record:<table>:<id>` (canonical platform record snapshot)	entity	evidentiary	`platform_record`	Canonical rec inputs (e.g. th profiler's can record, LLD §
`agent:<uuid>@sha256:…`	agent	structural	NULL	`agent.versi`
`human:<user_id>`	agent	structural	NULL	First human a
`derivation:<name>@<version>`	activity	evidentiary	NULL	Platform `DER` (kept: the car arithmetic in `grounding_tr` §5.7)
`judge:<cid>/<attempt>`	activity	**interpretive**	NULL	`judge.report`
`claim:<uuid>`	entity	structural	NULL	`claim.submit`
`rule:<uuid>`	entity	evidentiary	NULL	Consolidation extraction (§6
`publication:<uuid>/<release>`	entity	evidentiary	NULL	`publication`

Edges use the PROV-DM relations `used`, `wasGeneratedBy`, `wasDerivedFrom`, `wasAttributedTo`, `wasAssociatedWith`, `wasInformedBy` and `hadPrimarySource`, plus `quotedFrom`, which PROV-DM defines as a subtype of `wasDerivedFrom`. An edge is field-

level when `field_path` (the pointer in the subject) or `object_path` (the pointer in the object) is non-empty.

The `epistemic` class is a property of the node and is copied onto each edge as `subject_epistemic` and `object_epistemic`. Composite foreign keys `(node_id, epistemic)` make the copies impossible to falsify (Appendix D case 31):

- `evidentiary`: bytes or acts that can support a claim;
- `interpretive`: model judgement, which never supports;
- `structural`: identity and attribution (agents, humans, claims), which is not a source of content.

6.2 The database makes interpretive nodes unreachable

```
CREATE TABLE prov.nodes (
  node_id     text PRIMARY KEY COLLATE "C"
              CHECK (node_id ~
'^(artifact|episode|toolcall|source|record|human|agent|derivation|judge|claim|rule|p
ublication):'),
  prov_type   text NOT NULL CHECK (prov_type IN ('entity','activity','agent')),
  epistemic   text NOT NULL CHECK (epistemic IN
('evidentiary','interpretive','structural')),
  origin_kind text CHECK (origin_kind IN
('external','human_attested','platform_record')),
  tenant_id   text NOT NULL,
  event_pos   bigint NOT NULL REFERENCES ledger.events (global_pos) ON DELETE
RESTRICT,
  UNIQUE (node_id, epistemic),
  CHECK (origin_kind IS NULL OR (epistemic = 'evidentiary' AND prov_type =
'entity')),
  CHECK (epistemic <> 'interpretive' OR split_part(node_id, ':', 1) IN
('judge','artifact')));
CREATE INDEX nodes_origin ON prov.nodes (node_id) WHERE origin_kind IS NOT NULL;

CREATE TABLE prov.edges (
  edge_id           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  relation          text NOT NULL CHECK (relation IN
('used','wasGeneratedBy','wasDerivedFrom','wasAttributedTo',

'wasAssociatedWith','wasInformedBy','hadPrimarySource','quotedFrom')),
  subject           text NOT NULL COLLATE "C",
  subject_epistemic text NOT NULL,
  field_path        text NOT NULL DEFAULT '',
  object            text NOT NULL COLLATE "C",
  object_epistemic  text NOT NULL,
  object_path       text NOT NULL DEFAULT '',
  event_pos         bigint NOT NULL REFERENCES ledger.events (global_pos) ON DELETE
RESTRICT,
  FOREIGN KEY (subject, subject_epistemic) REFERENCES prov.nodes (node_id,
epistemic) ON DELETE RESTRICT,
  FOREIGN KEY (object,  object_epistemic)  REFERENCES prov.nodes (node_id,
epistemic) ON DELETE RESTRICT,
  -- (a) an interpretive node can never be a source
  CONSTRAINT edges_interpretive_never_source CHECK (
    object_epistemic <> 'interpretive' OR relation NOT IN
('wasDerivedFrom','hadPrimarySource','quotedFrom')),
  -- (b) no evidentiary or structural node may point INTO an interpretive node,
except by informational relations
  CONSTRAINT edges_interpretive_unreachable CHECK (
    object_epistemic <> 'interpretive' OR subject_epistemic = 'interpretive'
    OR relation IN ('wasInformedBy','wasAttributedTo','wasAssociatedWith')),
```

```
  CONSTRAINT edges_quote_is_field_level CHECK (relation <> 'quotedFrom' OR
(field_path <> '' AND object_path <> '')),
  CHECK (subject <> object OR field_path <> object_path),
  UNIQUE (relation, subject, field_path, object, object_path));
CREATE INDEX edges_walk    ON prov.edges (subject, field_path, relation) INCLUDE
(object, object_path, object_epistemic);
CREATE INDEX edges_reverse ON prov.edges (object, relation);
CREATE TRIGGER t_nodes_immutable BEFORE UPDATE OR DELETE ON prov.nodes FOR EACH ROW
EXECUTE FUNCTION ledger.forbid_mutation();
CREATE TRIGGER t_edges_immutable BEFORE UPDATE OR DELETE ON prov.edges FOR EACH ROW
EXECUTE FUNCTION ledger.forbid_mutation();
```

Why this makes an interpretive node unreachable in any Grounding walk (T10).

1. The walk (§6.5) starts at an evidentiary artifact: the claim's subject artifact. It follows only `quotedFrom`, `wasDerivedFrom` and `hadPrimarySource` edges with `object_epistemic = 'evidentiary'`.
2. That filter alone suffices, but the database guarantees more, so a buggy walk cannot leak either:
   - By (a), no edge whose relation is walkable has an interpretive object.
   - By (b), no evidentiary node has *any* edge into an interpretive node except `wasInformedBy`, `wasAttributedTo` and `wasAssociatedWith`, none of which a source walk follows.
3. By induction over path length, every node reachable from an evidentiary start by walkable edges is evidentiary.
4. The composite FKs mean an edge cannot lie about an endpoint's class. Node classes are immutable (`t_nodes_immutable`), so a node cannot be reclassified after the fact.

The judge's own subgraph is allowed:

- `artifact:<report> --wasGeneratedBy--> judge:<cid>/<attempt>` is interpretive to interpretive.
- `judge:… --used--> artifact:<enforced>` is interpretive to evidentiary, which is permitted.

It is a dead end in the evidentiary direction. A query that starts from the claim can find the report only through `edges_reverse` (`object = artifact:<enforced>`, `relation = used`), and that is how the packet assembles the report for humans. ΞPROV therefore records that the report exists and where it came from (model, prompt version, inputs), exactly as the spec requires [S §Judge].

The four rejection cases were validated (Appendix D cases 30–33):

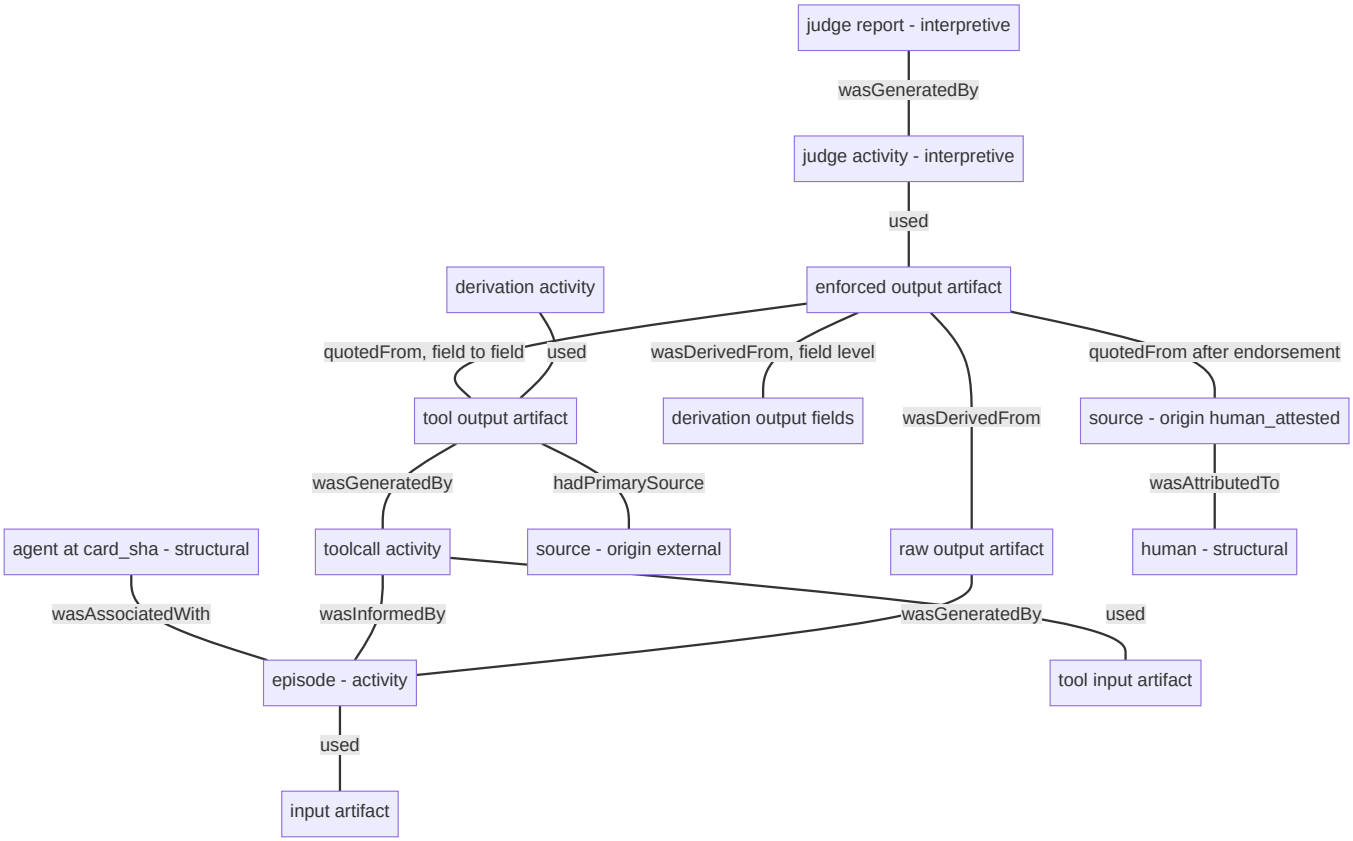
- a judge report as `quotedFrom` source;
- a mislabelled object class;
- an evidentiary artifact `wasGeneratedBy` a judge;
- an episode that `used` a judge report.

The last two matter: they stop an agent from consuming a judge report as input and laundering it into evidence.

Claims can never cite a judge report as evidence. `claim.submit` rejects any `artifact_refs` or `tool_call_refs` element whose artifact has a `prov.nodes` row with `epistemic = 'interpretive'`. The check is one index probe; the rejection class is `interpretive_not_citable`.

6.3 The edges every run writes

Edges are written in the transactions of §4.5.


judge report - interpretive
wasGeneratedBy
judge activity - interpretive
used
derivation activity
enforced output artifact
quotedFrom, field to field
used
wasDerivedFrom, field level
quotedFrom after endorsement
tool output artifact
derivation output fields
wasDerivedFrom
source - origin human_attested
wasGeneratedBy
hadPrimarySource
wasAttributedTo
agent at card_sha - structural
toolcall activity
source - origin external
raw output artifact
human - structural
wasAssociatedWith
wasInformedBy
wasGeneratedBy
used
episode - activity
tool input artifact
used
input artifact

Edge	Transaction	Producer
`episode --used--> input`, `agent@card --wasAssociatedWith--> episode` (reverse PROV direction: activity `wasAssociatedWith` agent is stored as subject `episode`, object `agent:…`)	TX-A	ledger (`episode.open`)
`toolcall --wasInformedBy--> episode`, `toolcall --used--> tool input`, `tool output --wasGeneratedBy--> toolcall`, `tool output --hadPrimarySource--> source`	TX-B_i	`abw-evidence::record_tool_call`
`raw --wasGeneratedBy--> episode`, `enforced --wasDerivedFrom--> raw`, `enforced#/p --quotedFrom--> tooloutput#/q`, `enforced#/p --wasDerivedFrom--> tooloutput#/q` (derivation inputs)	TX-C	`abw-evidence::record_evidence`
`judge --used--> enforced`, `report --wasGeneratedBy--> judge`	judge transaction	`abw-judge`
`source(human_attested) --wasAttributedTo--> human`, `enforced#/p --quotedFrom--> source#`	`human.decided` transaction	ledger (`human.decide`)

The `agent` relation follows PROV-DM's direction: `wasAssociatedWith(activity, agent)`. It is stored as subject = episode, object = `agent:…`.

6.4 External sources and resolution

```
CREATE TABLE proj.external_sources (
  source_id       uuid PRIMARY KEY,
  node_id         text NOT NULL UNIQUE COLLATE "C",          -- 'source:<source_id>'
  kind            text NOT NULL CHECK (kind IN
('url','doi','dataset','sensor','registry','human_attestation')),
  locator         text NOT NULL,
  retrieved_at    timestamptz NOT NULL,
  content_sha     text REFERENCES ledger.artifacts ON DELETE RESTRICT,
  resolver_status text NOT NULL CHECK (resolver_status IN
('resolved','unreachable','mismatch','not_attempted')),
  event_pos       bigint NOT NULL REFERENCES ledger.events (global_pos),
  UNIQUE (kind, locator, content_sha));
CREATE TRIGGER t_external_sources_immutable BEFORE UPDATE OR DELETE ON
proj.external_sources
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
CREATE CONSTRAINT TRIGGER t_external_sources_justified AFTER INSERT ON
proj.external_sources
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('event_pos');
```

- **Tools declare their origin.** Each tool declares `fn origin(&self) -> ToolOrigin`, which is **required, with no default** [M R5.3] (§12.1).
  - A tool declaring `External(kind)` returns a `ToolOutput { body, locators: Vec<Locator> }`, where the locators are the URLs fetched, DOIs resolved and dataset IDs read.
  - In TX-B_i the ledger writes one `proj.external_sources` row and one `source:` node per distinct `(kind, locator, content_sha)`, plus the `hadPrimarySource` edge.
  - `content_sha` is the SHA of the bytes the tool actually received, stored as an artifact when the policy's `retain_source_bytes` is on [P OD-21].
- **A source is retrievable, not a string.** Grounding treats `source:` nodes with `resolver_status IN ('resolved','not_attempted')` as `External` origins. A `mismatch` or `unreachable` node is an origin of kind `None` with class `source_unverifiable`. `not_attempted` counts only until the resolver runs.
- **DOI and URL resolution** is an outbox job `source.resolve`, enqueued in TX-B_i. Every fetch goes through `net_guard` (§12.5).
  - A `doi` locator is resolved through `https://doi.org/<doi>` with redirects **disabled at the client**. Each `Location` hop is re-validated by `net_guard::check` and followed manually, up to 5 hops.
  - A `url` locator is fetched once with `If-None-Match` when an ETag is known.
  - The result appends `source.resolved { source_id, status, content_sha?, title?, http_status }` on the episode stream.
  - The resolver never mutates `proj.external_sources`, which is immutable. It writes a new row with the new status, keyed by `(kind, locator, content_sha)`, and a

`wasDerivedFrom` edge from the new source node to the old one. The walk takes the newest node by `event_pos`.

- `mismatch` means the DOI resolves to a different title than the tool reported, compared with `NormalisedText` [M R5.3].

- **Human attestation** is a source of kind `human_attestation`, `locator` = the citation the human supplied (required), `origin_kind = 'human_attested'` [M R5.8b].

6.5 The chain walk

The walk runs in TX-D through the transaction's connection, against the snapshot. It returns every reachable row up to the depth bound, with cycle marking. The fold into an `Origin` is done in Rust (`abw-lattice::inputs`), because `wasDerivedFrom` at field level is **all-of** (a derived value is only as grounded as its weakest input) while `quotedFrom` and `hadPrimarySource` are **any-of**.

```
-- origin_of($1 = start node, $2 = JSON pointer in start node, $3 = depth bound,
default 16)
WITH RECURSIVE walk(node, path, depth, parent_edge, visited, is_cycle) AS (
    SELECT $1::text COLLATE "C", $2::text, 0, NULL::bigint,
           ARRAY[($1::text || '#' || $2::text) COLLATE "C"], false
  UNION ALL
    SELECT e.object,
           CASE WHEN e.relation = 'hadPrimarySource' THEN '' ELSE e.object_path END,
           w.depth + 1, e.edge_id,
           w.visited || (e.object || '#' || e.object_path),
           (e.object || '#' || e.object_path) = ANY (w.visited)
      FROM walk w
      JOIN prov.edges e
        ON e.subject = w.node
       AND e.object_epistemic = 'evidentiary'
       AND (   (e.relation IN ('quotedFrom','wasDerivedFrom') AND e.field_path =
w.path AND w.path <> '')
            OR (e.relation = 'hadPrimarySource' AND e.field_path IN ('', w.path)))
     WHERE NOT w.is_cycle
       AND w.depth < $3
       AND NOT EXISTS (SELECT 1 FROM prov.nodes t
                       WHERE t.node_id = w.node AND t.origin_kind IN
('external','human_attested'))
)
SELECT w.depth, w.node, w.path, w.parent_edge, n.origin_kind, w.is_cycle,
       NOT EXISTS (SELECT 1 FROM walk c
                   WHERE c.depth = w.depth + 1 AND c.visited[1:w.depth+1] =
w.visited) AS frontier
  FROM walk w JOIN prov.nodes n ON n.node_id = w.node
 ORDER BY w.depth, w.parent_edge;
```

Semantics:

- **Field-scoped.** At every hop the walk follows only edges whose `field_path` is the field it is tracing. A `quotedFrom` edge moves the traced pointer to `object_path` in the source artifact. `hadPrimarySource` applies to the whole artifact (`field_path = ''`) or to that field.
- **Recursion stops at origins.** Expansion stops at a node with `origin_kind IN ('external','human_attested')` , so the walk never explores beyond a source.
- **Cycle-safe.** The visited array holds `(node#path)` pairs. A revisit is emitted once with `is_cycle = true` and not expanded.
- **Depth-bounded** by `PROV_MAX_DEPTH = 16` [M R5.5]. A frontier row at `depth = 16` folds to `None` with class `depth_exceeded` .
- **Fold (Rust).**
  - `origin(node, path)` is defined case by case:

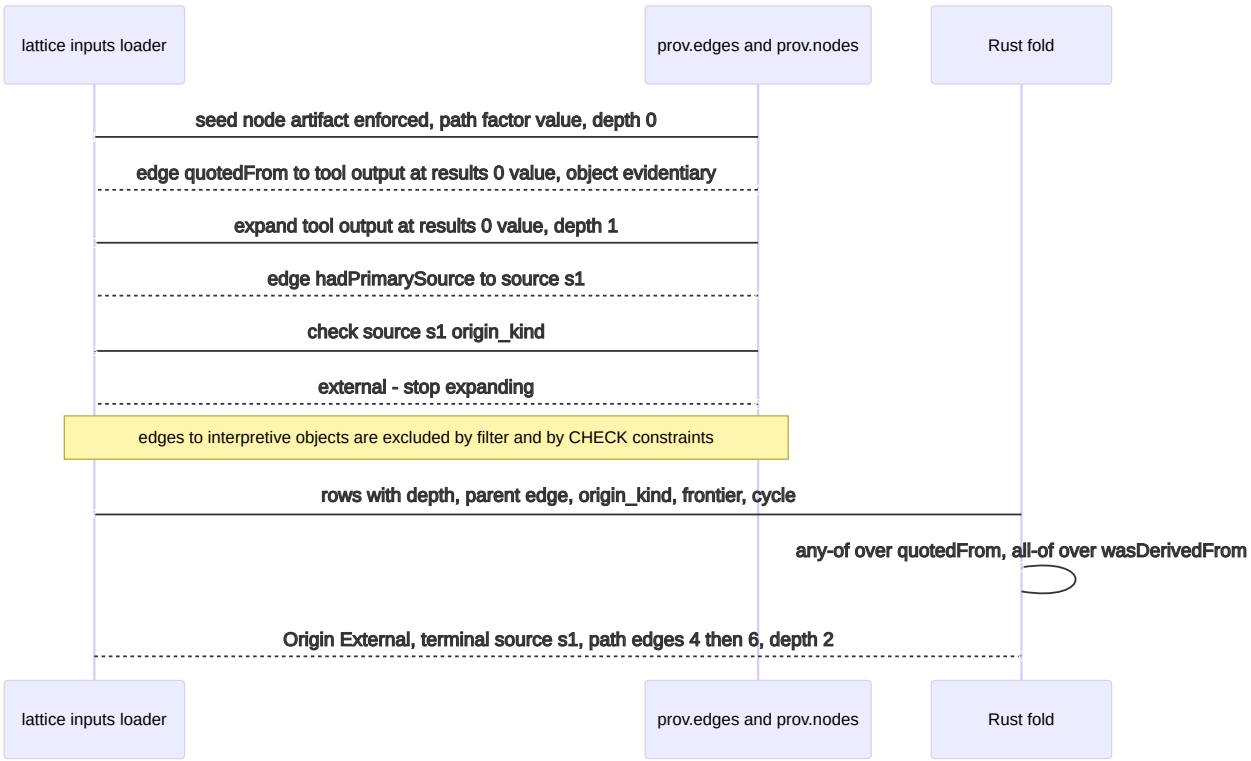
Node	**`origin(node, path)`**
Terminal with `origin_kind`	That kind
Frontier, not terminal	`None { ended_at: node }`
Has `wasDerivedFrom` children	The **minimum** over those children, with the order `External > HumanAttested > PlatformRecord > None`
Has only `quotedFrom` or `hadPrimarySource` children	The **maximum** over those children

  - A `record:` node (`platform_record`) is not a recursion stop. It is a recognised terminal that folds to `PlatformRecord` . That kind satisfies Truth cross-checks, **not** Grounding, unless the policy lists the record table as external (e.g. a regulatory registry).

Results on the validation fixture (Appendix D case 34):

Start	Walk	Fold
`artifact:enf#/factor/value`	→ `artifact:tco#/results/0/value` → `source:s1` (external), depth 2	`External`
`artifact:enf#/factor/name` (fabricated)	Frontier at depth 0	`None { ended_at: artifact:enf }`: the spec's "agent output at step N" diagnosis
`artifact:enf2#/co2e` (derived from two inputs, one grounded)	Two children: `tcoA` → `source:s2` (external), `tcoB` (frontier)	min(External, None) = `None { ended_at: tcoB }`, class `chain_broken`
Cycle `loop1#/a` ↔ `loop2#/b`	Revisit flagged `is_cycle` at depth 2	`None`, class `cycle`
Depth bound 1	Frontier at `tco`	`None`, class `depth_exceeded`

The walk sequence for one field:



Cost.

- Each recursion step is an index lookup on `edges_walk (subject, field_path, relation)`, with the object columns in `INCLUDE`, so the edge table needs no heap fetch. Run-written chains are 2–4 hops, so a typical field costs 3–5 index probes.
- Worst case is `O(b^D)` rows for fan-out b. `statement_timeout = 2s` on the walk bounds it in time, and exceeding the timeout is `infra_unavailable`, not `pass` (T12).
- A batched form seeds the CTE with all fields of the claim (`unnest($pointers)`) so one round trip serves every field. On the validation fixture's small tables the planner chose bitmap scans. At production cardinalities `edges_walk` is expected to drive a nested loop; `EXPLAIN` belongs in the Phase 2 performance test (§15, `prov_walk_plan_uses_index`).

6.6 Derived knowledge inherits a provenance floor

Consolidation extracts `semantic_rules` from episode clusters (`src/provenance_oracle.rs:155-251` [K: the floor logic is kept; its input changes]).

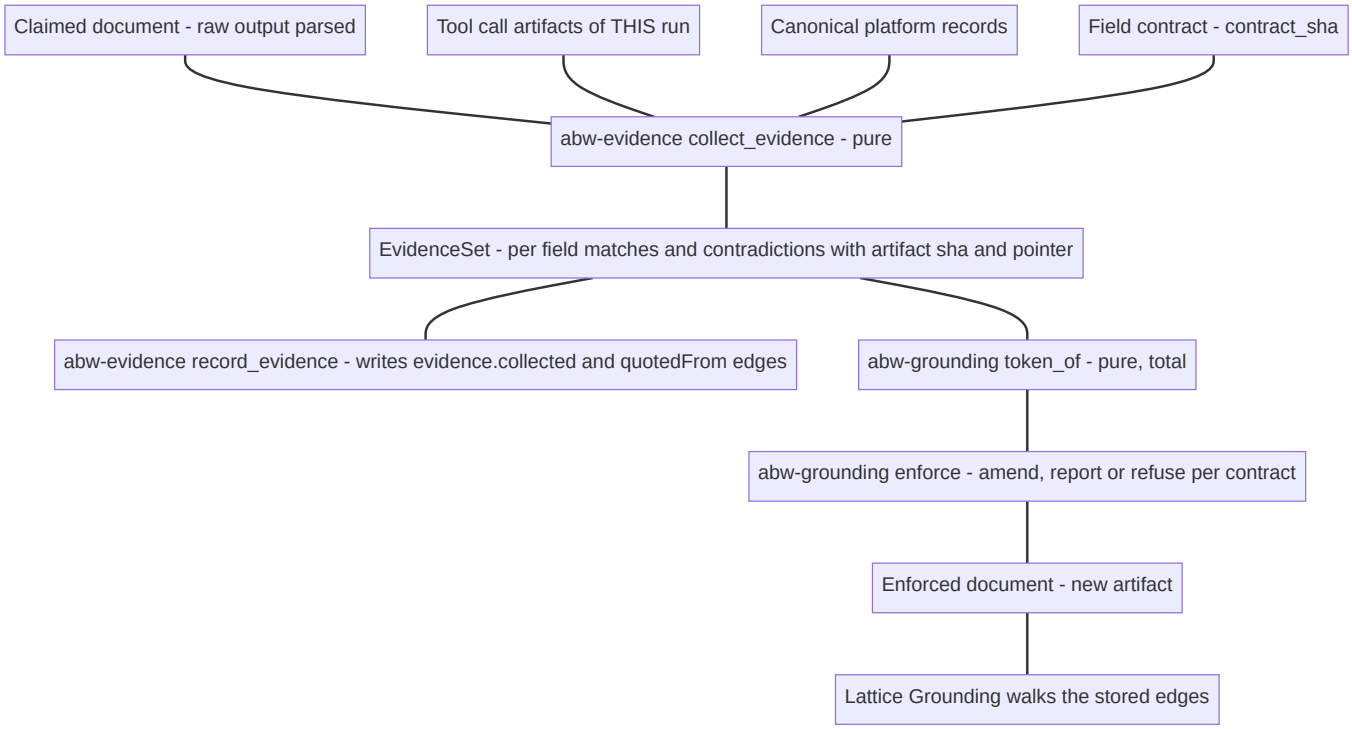
- **Edges.** Extraction writes a `rule:<id>` node and one `rule --wasDerivedFrom--> artifact:<enforced output of episode>` edge per source episode, in the transaction that inserts the rule. In that transaction `abw-ledger::crud` writes the CRUD `semantic_rules` row, and a `rule.extracted` event justifies the prov rows.
- **Floor.** `provenance_floor = min over source edges of strength(origin_of(source))`, capped at `EXTRACTION_CEILING = model_inference` (`src/grounding_trust.rs:1401` [K]). The kept `FloorAccumulator` folds it. It is computed **once, at extraction**, from stored edges, instead of re-grading raw text on every call (LLD §5.5).
- **Strength mapping.** `External` and `HumanAttested` give strength 2, `PlatformRecord` gives 2, and `None` gives 0. The ceiling then caps the result at 1.
- **Interpretive inputs.** A rule derived from an interpretive artifact is impossible by (a) and (b) of §6.2, because the edge insert fails. Consolidation MUST therefore exclude judge-report artifacts from its input set; the check is the same index probe as in `claim.submit`.
- **Dead columns.** `semantic_rules.origin` and `rule_retrievals.episode_id` are written by this same change [M R5.6]. G29 is Defer for the old model [R §4 G29]: "Do not write these columns in the old model". They are written only once extraction runs through the ledger, in Phase 3.

---

7. Evidence-based grounding

7.1 Two stages, two modules

Under T8, evidence and verdict are separate:



Function	Module	Pure	Writes
`collect_evidence(contract, doc, calls, canon) -> EvidenceSet`	`abw-evidence`	yes	—
`record_evidence(&EvidenceSet, &mut ProjTx)`	`abw-evidence`	no	`evidence.collected` event, `proj.field_evidence` rows, `quotedFrom` edges
`token_of(&FieldEvidence) -> ProvenanceToken`	`abw-grounding`	yes	—
`enforce(policy, doc, tokens) -> (EnforcedDoc, Vec<Violation>)`	`abw-grounding`	yes	— (the caller stores the artifact)

No function both stamps provenance and amends the claim:

- `record_evidence` writes evidence and edges; it never sees the enforced document.
- `enforce` produces the amended document; it never writes.

The current fused `grounding_trust::enforce(agent_id, &mut doc) -> Report` (`src/grounding_trust.rs:5309` [K: replaced]) is split along exactly this line. `tests/claims/evidence_verdict_separation.rs` asserts two things through `cargo metadata`:

- `abw-evidence` does not depend on `abw-grounding` or `abw-lattice`;
- no public function in either crate takes both `&mut ProjTx` and `&mut EnforcedDoc`.

7.2 Evidence collection

```
pub enum MatchMethod { ExactJson, NumericWithin { rel_bp: u32 }, NormalisedText,
Canonical(CanonId) }
pub struct EvidenceRef { pub artifact_sha: Sha256Ref, pub json_pointer: JsonPointer,
pub method: MatchMethod }
pub struct FieldEvidence {
    pub path: JsonPointer, pub claimed: serde_json::Value, pub claimed_sha:
Sha256Ref,
    pub contract: GroundingKind, pub tool_called: bool, pub populated: bool,
    pub matches: Vec<EvidenceRef>, pub contradicted_by: Vec<EvidenceRef>,
}
pub fn collect_evidence(contract: &AgentContract, doc: &serde_json::Value, calls: &
[ToolCallRef],
                        canon: &CanonicalRecords) -> EvidenceSet;
pub struct ToolCallRef<'a> { pub sha: &'a Sha256Ref, pub tool_name: &'a str, pub
input: &'a serde_json::Value,
                             pub output: &'a str, pub origin: ToolOrigin }
```

Rules for `Grounding::Sourced { tool, response_field }`, the contract variant kept from `src/grounding_trust.rs:1683` [K] [M R4.2]:

1. Consider only calls of *this run* with `tool_name == tool`, i.e. the `tool.invoked` artifacts of the episode. Pass them to `collect_evidence` from memory in TX-C, and from `ledger.artifacts` on re-collection.
2. Parse `output` as JSON if possible, and locate `response_field` by the JSON-pointer or path-glob declared in the contract.
3. A field **matches** when the contract's `MatchMethod` holds. The default is `ExactJson`. Carbon's 2% transcription band becomes `NumericWithin { rel_bp: 200 }`, declared on the contract.
4. A field is **contradicted** when the response contains the field with a value that does not match.
5. A contract naming a `response_field` that the tool's declared `response_shape` cannot produce is a **contract defect**. It fails CI (`tests/claims/contract_fields_exist.rs`), and

`PlatformTool::response_shape` (`platform_tool.rs:177` [K]) becomes a checked registry [M R4.2].

7.3 The total verdict table

`token_of` is total over the evidence space, with the first matching row winning [M R4.3]:

#	Evidence	Token	Strength
1	Field contract `Derived` and every input field's token has strength 2	`platform_derived`	2
2	Field contract `Derived` and some input below 2	`model_inference`	1
3	Human citation recorded (`human_attestation` source)	`human_sourced`	2
4	Human endorsement without citation	`human_endorsed`	1
5	`contradicted_by` non-empty	`tool_contradicted`	0
6	`matches` non-empty	`tool_verified`	2
7	Populated, named tool called, no match	`tool_unmatched`	0
8	Populated, named tool never called	`tool_not_called`	0
9	Empty, tool called	`tool_no_match`	0
10	Empty, tool not called	`unavailable_no_tool_source`	0
11	Contract `Unsourced` or `Narrative` with content	*(not a token)*: a violation, `UngroundedField` or `NarrativeLeak`	—
12	Agent has **no** contract	No tokens. Episode records `gate.decided { gate: grounding_enforcement, decision: undetermined, reason_class: insufficient_input }`. Claims enter the lattice with Grounding `not_evaluated(no_contract)` [M R4.4]	—

Persisted evidence is an inline projection of `evidence.collected`, written in TX-C [M R4.6]:

```
CREATE TABLE proj.field_evidence (
  episode_id      uuid NOT NULL REFERENCES proj.episodes ON DELETE RESTRICT,
  path            text NOT NULL,                                  -- JSON pointer
in the claimed document
  mode            text NOT NULL CHECK (mode IN ('shadow','enforce')),
  claimed_sha     text NOT NULL REFERENCES ledger.artifacts ON DELETE RESTRICT,
  token           text NOT NULL CHECK (token IN
('tool_verified','tool_contradicted','tool_unmatched','tool_not_called',

'tool_no_match','platform_derived','model_inference','human_sourced','human_endorsed
',

'unavailable_no_tool_source','pending_tool_check','pending_human_check','rejected'))
,
  matches         jsonb NOT NULL,                                 --
[{artifact_sha, json_pointer, method}]
  contradicted_by jsonb NOT NULL,
  contract_sha    text NOT NULL CHECK (contract_sha ~ '^sha256:[0-9a-f]{64}$'),
  event_pos       bigint NOT NULL REFERENCES ledger.events (global_pos),
  PRIMARY KEY (episode_id, path, mode));
CREATE TRIGGER t_field_evidence_immutable BEFORE UPDATE OR DELETE ON
proj.field_evidence
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
CREATE CONSTRAINT TRIGGER t_field_evidence_justified AFTER INSERT ON
proj.field_evidence
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('event_pos');
```

`contract_sha` makes contract drift a stored fact instead of something detected at read time (LLD §7.1) [M R4.6].

Row 5 precedes row 6 deliberately: one contradicting source outweighs a matching one. Rows 7 and 8 are the demotions G09 requires. **"Populated" never yields strength 2 on its own**: that is the defect at `src/grounding_trust.rs:5494-5496` [K: replaced], where `Some(true) => PROV_TOOL` is stamped on "has sourced fields, and at least one came back populated".

The three new tokens (`tool_contradicted`, `tool_unmatched`, `tool_not_called`) join `PROVENANCE_VALUES` (`grounding_trust.rs:136` [K: extended]), the `proj.field_evidence.token` CHECK, and `seam_vocabulary` [M R4.3].

7.4 Enforcement policy and the `<block>_provenance` rendering

Enforcement is declared per contract: `OnUnsupported { Amend, Report, Refuse }` [M R4.4].

- `Amend` nulls the field, which is today's behaviour. It is the default for tokens 5, 7 and 8.
- `Refuse` makes `grounding_enforcement` a **control** at the response boundary. The amended document is not returned; the client gets 422 with the violations. The run and its evidence are still persisted in TX-C, because the work happened and the record of it is not optional.
- The `<block>_provenance` keys written into documents are a **rendering** of the stored tokens, kept for card-schema compatibility [M R4.5]. A test asserts that a re-render from `proj.field_evidence` equals the keys in the stored enforced artifact.

7.5 The bridge from today's content proxy (G09)

G09 is "Do now: join sourced values against `context.tool_invocations`; demote unmatched values to `model_inference`. This is the bridge to the Grounding walk" [R §4].

The bridge has three steps, and each produces the next step's input:

Step	Phase	What	Output
B1	0	Join against `episodes.context.tool_invocations` (persisted verbatim, `src/episodes.rs:293-301` [K]) inside the current grader. Unmatched populated sourced fields are demoted to `model_inference`	Same document shape, honest strength
B2	2	`collect_evidence` + `token_of` over `tool.invoked` artifacts, persisted as `evidence.collected` and `quotedFrom` edges	Stored evidence that the walk can follow
B3	2	The Lattice Grounding dimension walks those edges to `source:` nodes	Grounding = origin, not population

B1's demotion target (`model_inference`, strength 1) is what the directive specifies. B2 refines it into `tool_unmatched` and `tool_not_called` (strength 0) once those tokens exist. That is a further demotion, introduced under shadow mode (§7.6), so the change from B1 to B2 is measured before it is enforced.

7.6 Measure → shadow → enforce

Directive rule 5 says that if enforcing a control breaks things, the breakage is reported, never hidden by downgrading the control [R rule 5]. The rollout therefore has three stages, and each needs the previous one's report.

1. **Measure.**

- `abw-admin grounding backfill-report` re-collects evidence for every historical episode that has `context.tool_invocations`. For each agent it reports how many `tool_verified` fields become `tool_unmatched`, `tool_not_called` or `tool_contradicted`, with 20 samples [M §13.1].
- The report is committed as `docs/reports/GROUNDING_BACKFILL_<date>.md`.
- **If enforcement would refuse or demote outputs at a rate that breaks cards, the report lists them, and work stops for the owner's decision.**

2. **Shadow.**
   - With `ABW_GROUNDING=shadow`, TX-C runs both graders.
   - The served document uses the legacy tokens. `evidence.collected { mode: shadow }` and `proj.field_evidence` rows with `mode = 'shadow'` hold the new verdicts.
   - The lattice reads **only** enforce-mode evidence. In shadow, Grounding is therefore `not_evaluated(grounding_shadow)`, not a shadow `pass` (T12).
   - Divergence is served at `/api/admin/grounding/divergence`.
3. **Enforce.**
   - `ABW_GROUNDING=enforce` is set only after the owner accepts the report (OD-22) [M §16.3].
   - It is recorded as a `policy.adjusted` on the platform gate policy, so the switch itself is an event with an actor.
   - The flag is deleted one release later.

The Lattice rolls out the same way with `ABW_LATTICE=off|shadow|route`. In `shadow`, vectors and routes are computed and stored, but `route.decided` carries `enforced: false`, and consumers do not gate on claim state.

---

8. Gates as events

8.1 Every gate decision is an event in its effect's transaction

`gate.decided` v1 has this payload:

```
{"gate":"coherence","decision":"refused","reason_class":"policy",
 "subject":"intervention:9d1…","episode_id":"…",
 "policy_ref":"<policy_id>@<version>",
 "detail":{"gamma_ppm":431000,"theta_ppm":500000,"correction_activation_ppm":-12000,
           "tensions":[…],"minimum_update_set":[…],"world_model_sha":"sha256:…"},
 "replay_of":null}
```

The `detail` object depends on the gate: γ and θ for coherence, schema errors for `output_schema`, the violated fields for `grounding_enforcement`, the estimate and available balance for `credit` [M R6.2].

The decision is appended **to the stream of the effect it gates**, in the effect's transaction:

Gate	Effect transaction	Stream	Refusal path
`credit`	TX-A (hold)	`episode:<eid>` when approved	Refusal: one-statement transaction on `gate:credit:<tenant>:<day>` **before** the 402 is sent
`tool_allowlist`	TX-B_i	`episode:<eid>`	Same transaction, with `tool.invoked { allowed: false }`
`grounding_enforcement`	TX-C	`episode:<eid>`	`Refuse` policy: TX-C still commits (the run happened); response 422
`output_schema`	TX-C	`episode:<eid>`	Report (status in response) or Refuse per card
`coherence`	`hitl.intervene` / `hitl.confirm` command	`intervention:<id>`	Refusal: the intervention command's transaction appends only the decision
`admission`	`agent.publish` command	`agent:<id>`	Same
`publication_claims`	`publication.release`	`publication:<id>`	Same
`net_guard`	The command that wrote the URL	`agent:<id>` or the push-config stream	422 at config write

The current write-behind machinery is deleted for Recorded gates [M R6.1] [R §4 G08]: the in-memory `QUEUE` (`src/gate_trust.rs:480` [K: deleted]), `QUEUE_CAP = 4096` (`:478`), and

the 15 s `spawn_gate_recorder` (`:818`). G08's interim fix ("per-row fallback on a poison batch") applies only until Phase 3, and nothing new is built on it.

`proj.gate_decisions` is an inline, immutable projection of `gate.decided` :

```
CREATE TABLE proj.gate_decisions (
  event_pos    bigint PRIMARY KEY REFERENCES ledger.events (global_pos) ON DELETE
RESTRICT,
  stream_id    text NOT NULL COLLATE "C",
  tenant_id    text NOT NULL,
  gate         text NOT NULL CHECK (gate IN
('coherence','grounding_enforcement','admission','output_schema',

'credit','tool_allowlist','publication_claims','net_guard','completeness')),
  decision     text NOT NULL CHECK (decision IN
('approved','refused','undetermined')),
  reason_class text CHECK (reason_class IN
('policy','infra_error','advisory','insufficient_input')),
  subject      text NOT NULL,
  episode_id   uuid,
  policy_ref   text,
  detail       jsonb NOT NULL,
  replay_of    bigint REFERENCES proj.gate_decisions (event_pos),
  CHECK ((decision = 'approved') = (reason_class IS NULL)),
  CHECK (decision <> 'refused' OR reason_class = 'policy'));
CREATE INDEX gate_decisions_episode     ON proj.gate_decisions (episode_id) WHERE
episode_id IS NOT NULL;
CREATE INDEX gate_decisions_gate_recent ON proj.gate_decisions (gate, event_pos
DESC);
CREATE TRIGGER t_gate_decisions_immutable BEFORE UPDATE OR DELETE ON
proj.gate_decisions
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
CREATE CONSTRAINT TRIGGER t_gate_decisions_justified AFTER INSERT ON
proj.gate_decisions
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('event_pos');
```

The last CHECK encodes T7 at the database: **only `policy` may produce `refused`** . An infrastructure error, an advisory check or missing input can only produce `undetermined` . The gate set is the `CHECK` , and `tests/claims/gate_ids_match_check.rs` asserts that the Rust enum equals it [M R6.5]. `Completeness` enters the CHECK in the same migration that makes it Recorded.

8.2 Reason classes

Class	Meaning	Allowed decisions
`policy`	The gate evaluated and the policy says no	`refused`
`infra_error`	Could not evaluate: provider error, timeout, `SettlingFailed`	`undetermined`
`advisory`	The check is an observation, not a control. Never a gate decision in the target; kept only for legacy rows mapped from today's advisory gates	`undetermined`
`insufficient_input`	A precondition is missing: no contract, empty world model, no schema	`undetermined`

`undetermined` is never a pass. Every consumer of a gate decision treats it as not-approved: it blocks, or it sends the effect to a human where a human path exists. The coherence gate's AgentWide human-only path is kept by design (LLD §6.5), and the human act is itself a `human.decided` event. Each richer verdict enum keeps its variants and exposes `fn as_decision(&self) -> GateDecision`, mapping "could not evaluate" to `undetermined` with a reason class [M R6.7].

8.3 Replay versus re-evaluation for two-reviewer consensus (G10)

Today, `confirm_two_reviewer_handler` calls `gate.check(&encoded)` (`src/handlers/observatory.rs:751` [K: replaced]), which uses an empty `WorldModel` and therefore always returns `Undetermined`. The first reviewer's Γ is never stored (LLD §6.5).

The target separates the two acts:

1. **First reviewer.**
   - `hitl.intervene` runs `check_against(encoded, &world)` (`agent-bestiary/coherence-gate/src/gate.rs:199` [K]).
   - In the command transaction it appends `gate.decided { gate: coherence, detail: {gamma_ppm, theta_ppm, correction_activation_ppm, tensions, minimum_update_set, world_model_sha} }`.
   - The world model is stored as an artifact, so `world_model_sha` resolves.
   - The two-reviewer request records the decision's `event_pos` [M R6.3].
2. **Confirmation** (`hitl.confirm`, second reviewer) appends **two** decisions in one transaction:
   - **Replay:** `gate.decided { …, replay_of: <first event_pos> }`. Its `detail` is copied from the first decision and verified by re-running `check_against` on the *stored* world model artifact. The replay must reproduce Γ exactly, or the command fails with `IntegrityError`, which is a bug.

- **Re-evaluation:** `gate.decided { …, replay_of: null }` against the *current* world model, with `detail.world_model_drift = (current_sha ≠ stored_sha)`.

3. **The intervention executes only if both are `approved`.** The correction record carries both `event_pos` values [M R6.3].
4. `CoherenceGate::check`, the no-world-model entry (`gate.rs:190` [K]), becomes `#[cfg(test)]`, so it cannot be called on a live path.

8.4 Observations

A control refuses or it does not exist (T7). Anything that only records is an **observation**:

- `observation.recorded { observer, subject, state: undetermined, reason_class, detail }` on the subject's stream, in the subject's transaction;
- `event_types.epistemic = 'observation'`.

Observations never appear in `proj.gate_decisions`, have no `refused` and no `approved`, and are read by metrics only.

Today	Target
`port_trust::bind_input` (`src/port_trust.rs:128` [K]) records `Refused` and refuses nothing (G30)	`observation.recorded { observer: input_port_match, state: undetermined, reason_class: advisory, detail: {matched_port?} }`. The enforcement decision is reported per directive rule 5: making it a control would refuse 47 of 102 cards (LLD §4.8). That is **reported, not enforced and not hidden**, and enforcement waits for the owner's decision (OD-23)
`schema_validate` on outputs records instead of refusing (`src/schema_validate.rs:125` [K])	Per card: `output_schema` gate with `Report` (observation) or `Refuse` (control), declared on the card's `output_contract`. Declared means part of `card_sha`
`InputSchema` (Counted, Metric)	Observation
`liveness_trust`, `schema_trust`, `rollup_trust`, `route_trust`, `outcome_trust`	`platform_health.observed` events with the rung names of §0.3; `state ∈ {ok, violated, undetermined}`
`write_accounting` counters (`src/write_accounting.rs:86-105`)	Deleted when the ratchet for direct writes reaches 0 (Phase 3). Counters are not a fix [R §2 freeze 3]

8.5 Gate reviews feed the outer loop

A `gate.review` command records a human's review of a past decision as a `gate.reviewed` event. When the review overturns the decision, the same transaction appends `false_pass.detected` or `false_fail.detected { source: gate_review }` [M R6.6]. A review never re-runs or reverses a past effect: reversal is a separate human command with its own events.

---

9. Money

9.1 Ledger events

Every wallet is a stream `wallet:<wallet_id>` (exact concurrency). Every event on it carries:

- `delta` (signed, `bigint`);
- `balance_after`;
- the split deltas `granted_delta`, `purchased_delta`, `held_delta`, `debt_delta`;
- `idempotency_key`;
- `related { kind, id }`.

Event	`delta`	`held_delta`	Typical counter-leg
`wallet.opened`	0	0	—
`wallet.granted`	+n (granted)	0	issuer −n
`wallet.deposited` (Stripe)	+n (purchased)	0	issuer −n (external money in)
`wallet.hold_placed`	0	+h	—
`wallet.hold_released`	0	−h	—
`wallet.settled` (execution charge)	−c	0 (paired with `hold_released`)	platform +c
`wallet.transferred`	±n	0	the other wallet ∓n
`wallet.royalty_paid`	−r (platform) / +r (owner)	0	—
`wallet.debt_recorded`	0	0	`debt_delta = +d` (actual cost exceeded hold and balance)
`wallet.sponsored`	+n	0	sponsor wallet −n (§9.6)
`wallet.reconciled`	±n	0	issuer ∓n (owner-approved backfill only, §14.4)

Double entry. Within each transaction, the sum of `delta` over all `wallet:` events is **zero** (T9). There is a single `issuer` wallet [P OD-24], whose balance is the negative of all credits ever issued. Grants and deposits are transfers from it. The total over all wallets is therefore always 0 (validated, Appendix D case 36). A mint like G01, which deposits with no counter-leg, is rejected **by the database** at commit (case 35).

9.2 DDL

```
-- per-stream chain: balance_after(seq) = balance_after(seq-1) + delta
CREATE FUNCTION ledger.wallet_chain_check() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE prev_after bigint; d bigint; after bigint;
BEGIN
  IF split_part(NEW.stream_id, ':', 1) <> 'wallet' THEN RETURN NEW; END IF;
  d     := (NEW.payload->>'delta')::bigint;
  after := (NEW.payload->>'balance_after')::bigint;
  IF d IS NULL OR after IS NULL THEN
    RAISE EXCEPTION 'ABW-T9: wallet event % lacks delta/balance_after',
NEW.event_type;
  END IF;
  IF NEW.seq = 1 THEN prev_after := 0;
  ELSE
    SELECT (payload->>'balance_after')::bigint INTO prev_after
      FROM ledger.events WHERE stream_id = NEW.stream_id AND seq = NEW.seq - 1;
    IF NOT FOUND THEN RAISE EXCEPTION 'ABW-T2: wallet % seq % has no predecessor',
NEW.stream_id, NEW.seq; END IF;
  END IF;
  IF prev_after + d <> after THEN
    RAISE EXCEPTION 'ABW-T9: wallet % chain broken at seq %: % + % <> %',
NEW.stream_id, NEW.seq, prev_after, d, after;
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER t_events_wallet_chain BEFORE INSERT ON ledger.events
  FOR EACH ROW EXECUTE FUNCTION ledger.wallet_chain_check();

-- double entry: per transaction, wallet deltas sum to zero; checked at COMMIT
CREATE FUNCTION ledger.wallet_tx_balanced() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE s numeric;
BEGIN
  SELECT coalesce(sum((payload->>'delta')::bigint), 0) INTO s
    FROM ledger.events WHERE txid = NEW.txid AND stream_id LIKE 'wallet:%';
  IF s <> 0 THEN
    RAISE EXCEPTION 'ABW-T9: transaction % moves % credits net; money is conserved',
NEW.txid, s;
  END IF;
  RETURN NULL;
END $$;
CREATE CONSTRAINT TRIGGER t_events_wallet_balanced AFTER INSERT ON ledger.events
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW WHEN (NEW.stream_id LIKE 'wallet:%')
  EXECUTE FUNCTION ledger.wallet_tx_balanced();

CREATE TABLE proj.wallets (
  wallet_id         uuid PRIMARY KEY,
  stream_id         text NOT NULL UNIQUE COLLATE "C" REFERENCES ledger.streams ON
DELETE RESTRICT,
```

```
  owner_type        text NOT NULL CHECK (owner_type IN
('issuer','platform','user','agent','workspace')),
  owner_id          text NOT NULL,
  balance           bigint NOT NULL,
  granted_balance   bigint NOT NULL,
  purchased_balance bigint NOT NULL,
  held              bigint NOT NULL CHECK (held >= 0),
  debt              bigint NOT NULL DEFAULT 0 CHECK (debt >= 0),
  applied_seq       bigint NOT NULL,
  CHECK (balance = granted_balance + purchased_balance),
  CHECK (owner_type = 'issuer' OR (granted_balance >= 0 AND purchased_balance >= 0
AND held <= balance)),
  UNIQUE (owner_type, owner_id));
CREATE UNIQUE INDEX wallets_one_issuer ON proj.wallets ((true)) WHERE owner_type =
'issuer';

CREATE TABLE proj.credit_holds (
  hold_id     uuid PRIMARY KEY,
  wallet_id   uuid NOT NULL REFERENCES proj.wallets ON DELETE RESTRICT,
  episode_id  uuid UNIQUE,
  amount      bigint NOT NULL CHECK (amount > 0),
  state       text NOT NULL CHECK (state IN ('open','settled','released')),
  opened_pos  bigint NOT NULL REFERENCES ledger.events (global_pos),
  closed_pos  bigint REFERENCES ledger.events (global_pos),
  expires_at  timestamptz NOT NULL,
  CHECK ((state = 'open') = (closed_pos IS NULL)));
CREATE INDEX holds_open_expiry ON proj.credit_holds (expires_at) WHERE state =
'open';

-- the wallet projection is applied BY THE DATABASE from the event; abw_app has
SELECT only on proj.wallets
CREATE FUNCTION proj.apply_wallet_event() RETURNS trigger LANGUAGE plpgsql SECURITY
DEFINER
SET search_path = pg_catalog, ledger, proj AS $$
DECLARE p jsonb := NEW.payload;
BEGIN
  IF split_part(NEW.stream_id, ':', 1) <> 'wallet' THEN RETURN NULL; END IF;
  IF NEW.event_type = 'wallet.opened' THEN
    INSERT INTO proj.wallets (wallet_id, stream_id, owner_type, owner_id, balance,
granted_balance,
                              purchased_balance, held, applied_seq)
    VALUES ((p->>'wallet_id')::uuid, NEW.stream_id, p->>'owner_type', p-
>>'owner_id', 0, 0, 0, 0, NEW.seq);
    RETURN NULL;
  END IF;
  UPDATE proj.wallets
     SET balance           = (p->>'balance_after')::bigint,
         granted_balance   = granted_balance   + coalesce((p-
```

```
>>'granted_delta')::bigint, 0),
         purchased_balance = purchased_balance + coalesce((p-
>>'purchased_delta')::bigint, 0),
         held              = held              + coalesce((p-
>>'held_delta')::bigint, 0),
         debt              = debt              + coalesce((p-
>>'debt_delta')::bigint, 0),
         applied_seq       = NEW.seq
   WHERE stream_id = NEW.stream_id AND applied_seq = NEW.seq - 1;
  IF NOT FOUND THEN RAISE EXCEPTION 'ABW-T3: wallet projection for % not at seq %',
NEW.stream_id, NEW.seq - 1; END IF;
  RETURN NULL;
END $$;
CREATE TRIGGER t_events_apply_wallet AFTER INSERT ON ledger.events
  FOR EACH ROW EXECUTE FUNCTION proj.apply_wallet_event();

-- belt and braces: at commit, the projection equals the stream head
CREATE FUNCTION proj.wallet_matches_head() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE h bigint; after bigint;
BEGIN
  SELECT s.head, (e.payload->>'balance_after')::bigint INTO h, after
    FROM ledger.streams s JOIN ledger.events e ON e.stream_id = s.stream_id AND
e.seq = s.head
   WHERE s.stream_id = NEW.stream_id;
  IF NEW.applied_seq <> h OR NEW.balance <> after THEN
    RAISE EXCEPTION 'ABW-T9: wallet % projection (seq %, bal %) <> ledger head (seq
%, bal %)',
      NEW.wallet_id, NEW.applied_seq, NEW.balance, h, after;
  END IF;
  RETURN NULL;
END $$;
CREATE CONSTRAINT TRIGGER t_wallet_matches_head AFTER INSERT OR UPDATE ON
proj.wallets
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.wallet_matches_head();
```

Why the balance can never drift from the ledger.

1. `abw_app` has no `UPDATE` on `proj.wallets` (Appendix D case 5). The only writer is `apply_wallet_event`, a `SECURITY DEFINER` trigger fired by the event insert.
2. The trigger applies exactly the event's deltas, guarded by `applied_seq = seq - 1`, so out-of-order application is impossible.
3. `wallet_chain_check` makes `balance_after` a running sum of `delta` over the stream (case 7). The fold of the stream therefore *is* `balance_after` at head.
4. `wallet_matches_head` re-checks the equality at commit (T3).

5. `CHECK (held <= balance)` and `granted, purchased ≥ 0` reject an overdraw at the statement that would cause it (case 9). There is no time-of-check/time-of-use gap, because the wallet stream row is locked for the whole transaction (L1).
6. Even `abw_owner` cannot change the projection without an event (case 6, the T3 guard).

The current balance is read from `proj.wallets` (`O(1)`). The ledger identity `balance = Σ delta` is a nightly audit query and a conformance test, not a runtime cost.

The legacy `public.wallets` and `public.credit_ledger` (`migrations/012_credit_ledger.sql`) become async read-model projections in Phase 3 and stop being written by any other path (§14.3). Until then, Phase 0's `*_tx` primitives write the wallet row and its ledger row on one connection in one transaction [R §4 G04] [M R1.1].

9.3 Holds and settlement inside episode close

The sequence is §4.5 ⟨3⟩–⟨5⟩ and ⟨17⟩–⟨22⟩. In ledger terms:

TX-A (hold). `lock_streams({wallet:payer, episode:eid})`. Then:

1. Read `proj.wallets` for the payer.
2. If `balance − held < estimate` or `debt > 0`, the result is `Rejection(InsufficientCredits)`.
3. Otherwise append `wallet.hold_placed { hold_id, held_delta: +estimate, delta: 0, related: episode, idempotency_key: "episode:<eid>:hold" }`.

The `credit_holds` row is written by the same `SECURITY DEFINER` trigger family.

TX-C (settle), with `W` including `wallet:payer`, `wallet:platform` and `wallet:owner` if a royalty is due. The events, in order:

1. `wallet.hold_released { hold_id, held_delta: −estimate }`.
2. `wallet.settled { delta: −min(actual, available), idempotency_key: "episode:<eid>:settle" }` and the platform's counter-leg.
3. If `actual > available`: `wallet.debt_recorded { debt_delta: actual − available }`. Debt is recorded, never silently forgiven, and a wallet with debt cannot open a new hold [M R1.3].
4. Royalty: the platform-to-owner `wallet.royalty_paid` pair, `idempotency_key: "episode:<eid>:royalty"`. The percentage is kept from `execution_owner_royalty_pct` (`src/gas.rs:30` [K]). The current non-atomic royalty (`src/gas.rs:241` [K: replaced]) is removed with its comment [M R1.4].

What this removes:

- the `balance <= 0` pre-check;
- `charge_execution_with_royalty` downgrading errors to `(0,0)` (`src/handlers/execution.rs:783` [K: replaced]);

- the post-work `charge_gas` 402.

Workspace messages and A2A use the same three phases.

9.4 Idempotency keys

Caller	`idempotency_key`	Stream
Hold	`episode:<eid>:hold`	payer
Settle	`episode:<eid>:settle`	payer
Royalty	`episode:<eid>:royalty`	platform and owner (two events, key suffixed `:debit` and `:credit`)
Stripe	`stripe:<checkout_session_id>`	user
App spawn funding	`app_spawn:<workspace_id>`	caller and workspace
Sponsorship	`sponsor:<app_slug>:<workspace_id>`	sponsor and workspace
Onboarding grant	`grant:onboarding:<user_id>`	user
Reconciliation	`reconcile:<report_sha>:<wallet_id>`	wallet

The partial unique index `events_money_idem (stream_id, payload->>'idempotency_key')` rejects a second event with the same key on the same stream (case 10). A retried command first hits `command_dedup` (step 2) and returns the stored result. If a different command carries the same money key, the unique index raises at the insert, and the command fails with `Conflict(idempotent_replay)`. A retry therefore cannot double-charge or double-credit.

9.5 Stripe

The Stripe webhook dispatches `wallet.deposit { session_id, user, credits }` with `idempotency_key = stripe:<session>`. The deposit event and its issuer counter-leg commit in one transaction. `stripe_sessions_processed` becomes a projection of `wallet.deposited`. Two current behaviours are deleted:

- the separate `INSERT` and the compensating `DELETE` (`src/handlers/billing.rs:266, 304` [K: replaced]);
- the follow-up `UPDATE credit_ledger SET stripe_session_id` (`billing.rs:334` [K: deleted]).

The session ID is in the event payload [M R1.5].

9.6 App spawn funding and platform sponsorship for template budgets

Funding rule (owner decision, kept). On 2026-11-06 the owner chose option (c) for G01: the platform sponsors the template `initial_budget`, and the caller pays any `extra_budget`. This decision is recorded in the amended SPEC 36 R0.1.2 and in `migrations/244_platform_sponsor_wallet.sql:9-14`. The Phase 0 implementation is committed on the integrity branch:

- the template budget is transferred from the wallet `platform:app_sponsorship` (`src/apps/provision.rs:36` [K]);
- `extra_budget` is transferred from the caller, capped by `APP_SPAWN_MAX_EXTRA_BUDGET` with default 1000 (`src/apps/provision.rs:39-46` [K]);
- both transfers are in the same transaction as the workspace creation, funding last so the shared sponsor row lock is short (`src/apps/provision.rs:277-320` [K]);
- either payer being short returns 402, and nothing is created [R §4 G01];
- auto-hire goes through the single hire rule `may_place` (`src/apps/hire_policy.rs:39` [K]).

In the target, the same rule becomes one `app.spawn` command:

- **Write set:**
  - `wallet:<sponsor>` (if `initial_budget > 0`);
  - `wallet:<caller>` (if `extra_budget > 0`);
  - `wallet:<workspace>` (new);
  - `workspace:<id>` (new). That stream holds `workspace.spawned { app_slug, manifest_sha, template_sha, funding_pos }` and is the authority for the workspace's existence.
- **Lock order** is `stream_id COLLATE "C"` order (§3.2.2), which replaces "funding last" as the deadlock rule.
- **Events.** The sponsor leg is `wallet.sponsored` / `wallet.transferred`, keyed `sponsor:<app_slug>:<workspace_id>`. The caller leg is `wallet.transferred`, keyed `app_spawn:<workspace_id>`. Both preserve the granted/purchased split.
- **CRUD rows.** The `public.teams` row and memberships are written by `abw_ledger::crud` in the same transaction (§14.5).
- **Topping up.** The sponsor wallet is topped up only by a ledgered `wallet.granted` from the issuer, with the operator as actor. An empty sponsor yields 402, never a mint. That is checked by the database: double entry, §9.1.

Per-caller sponsorship quota [P OD-25]. SPEC 36 R0.1.5b leaves this open: without a quota, one caller can drain the sponsor wallet with repeated `extra_budget = 0` spawns (up to 200 per batch request). The target mechanism is a policy value `max_sponsored_budget_per_caller_per_day`. `app.spawn` takes the sponsor stream lock,

which serialises sponsored spawns. Its `decide` folds the caller's `wallet.sponsored` events for the day, through an index on `(stream_id, (payload->>'beneficiary_owner'), recorded_at)`, and refuses with 402 `sponsor_quota_exhausted` when the quota is exceeded. The quota value and whether per-App sponsor wallets replace the single platform wallet are owner decisions.

9.7 Money crash analysis

Crash point	Result
Mid-TX-A	No hold, no episode, no LLM call made
After TX-A, before TX-C	Hold open until `hold_reaper` or `episode_reaper` releases it with `episode.closed failure`. The same transaction appends the release and the close
Mid-TX-C	Nothing settled, nothing closed, hold still open. Reaper as above. The executed LLM spend is not charged; this is a platform loss bounded by one run, and it is visible as an `abandoned_timeout` episode with tool events
After TX-C	Fully settled. A client retry replays the dedup result

The invariants `Σ delta = 0` per transaction, `balance_after` chain continuity and `0 ≤ held ≤ balance` hold in every row above, because each is checked at statement or commit time.

---

10. Vertical slices

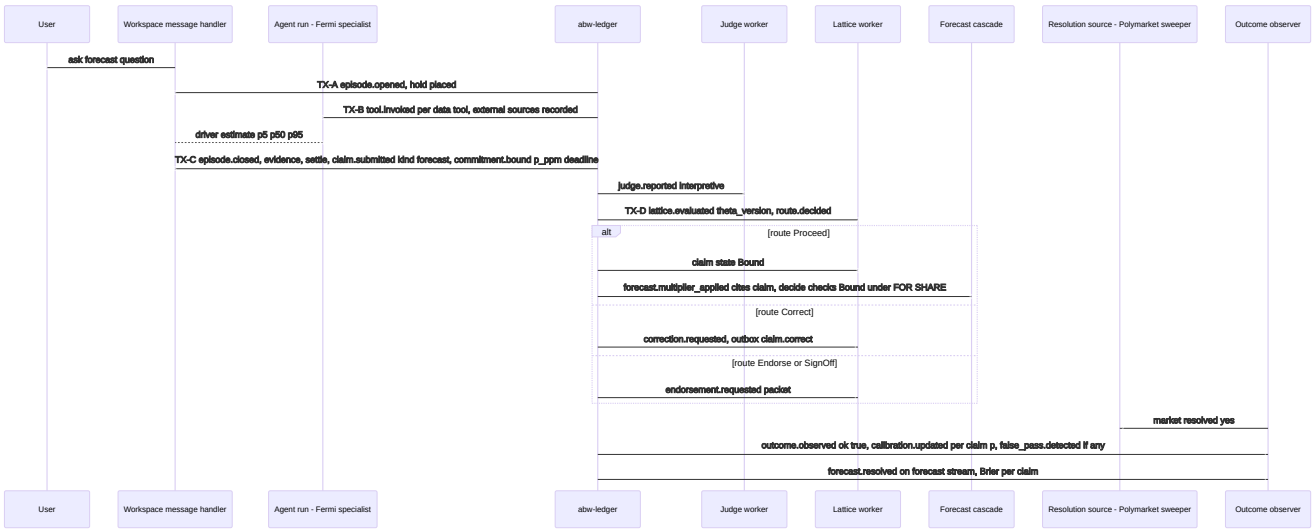
10.1 Fermi (Phase 2)

Fermi goes first. Binding and Brier scoring are native to forecasts, `forecast_agent_claims` is already append-only, and outcomes resolve, so the calibration loop can be tested for real [R §5 Phase 2].

What becomes a claim. Each agent's driver estimate becomes a claim of `kind: forecast`. Today such an estimate is a `forecast_agent_claims` row: `driver, p5, p50, p95` (`migrations/187_forecast_agent_claims.sql:67-99` [K: the table is kept as a projection]).

Claim field	Value
`content`	`[{"/driver": <name>}, {"/p50": <value>}, {"/p5": …}, {"/p95": …}]` over the agent's enforced output artifact
`cited_versions`	`[(forecast:<id>, seq), (agent:<id>, version)]`. The forecast's working state stays CRUD, but each *forecast revision that is scored* is captured as an artifact and appended to a `forecast:<id>` stream (`forecast.revised { revision_sha }`) [P OD-12]. Adding forecast resolution to the event-sourced scope is SPEC 36's owner question (OD-37)
`binding`	`{ p_ppm: P(outcome
`stakes`	`routine` ; `high` if the forecast is published to a market or used in a money transfer above threshold

Event flow [R §5 Phase 2]: `claim.submitted` → `lattice.evaluated` (one verdict vector) → `route.decided` → `outcome.observed` . Tool invocations are events, and θ lives in `ledger.policy_versions` , with `theta_version` part of the vector (G23).



Consistency boundaries:

Boundary	Transaction	Invariants guarded
Reserve	TX-A	T9 hold
Each data tool call	TX-B_i	T5 (evidence durable as it happens)
Close, claim and commitment	TX-C	T5, T9 settle, Binding registration (T19)
Evaluate and route	TX-D (`REPEATABLE READ`)	T11, T13, T15
Applying a multiplier to the forecast	`forecast.apply_claim` command: `W = {forecast:<id>}` , `R = {claim:<cid>}` share-locked	T7: Proceed gates the effect
Outcome	`outcome.record` command: `W = {claim:<cid>, calibration:<subject>…, forecast:<id>}`	T19: Brier from the claim's own `p`

Brier at outcome. Brier is computed per claim (§5.12). `select_agent` (`src/agent_backend/tools/domains/workspace.rs:165-247` [K: ranking kept]) switches its Brier input to the middle-loop posterior read model [M R7.11]. The resolved forecast's own Brier is still computed by the kept `compute_brier_score` path for the leaderboard. It is no longer copied to agents (`src/outcome_trust.rs:24-36` [K]).

Freeze trigger. `fermi_forecasts` ' freeze trigger currently *silently reverts* scoring changes with `RAISE WARNING` (`migrations/174` , LLD §4.4). In Phase 2 it becomes `RAISE EXCEPTION` [M R2.3]. Its nine callers that rely on the silent revert are listed by the Phase 2 rule-5 report, and the change is not made until they are fixed [R rule 5].

Calibration loop tested for real. The Phase 2 conformance suite replays resolved Polymarket questions from the fixture corpus (`scripts/brier_backtest_questions.yaml`). It asserts:

- `calibration.updated.brier_micro` equals the hand-computed $(p - o)^2$ per claim;
- two agents contributing different `p` to one forecast receive different Brier scores.

The second assertion is the defect `outcome_trust.rs` documents.

10.2 DPP / carbon (Phase 3)

This is the flagship lineage: carbon accounting into a Digital Product Passport (LLD §5.7). The target makes every published value traceable to the episode, the source commit and the artifact hashes that produced it [R §5 Phase 3 acceptance].

Changes from today:

1. `dispatch_rabble_action` (`src/handlers/rabble_workspace.rs:203` [K: signature changes]) returns `Dispatched { episode_id, reply, tool_calls }`, not `String` [M R5.7].
2. Carbon enforcement happens **once**, on the normalised document, inside the accountant's TX-C. The double enforcement (raw in `persist_opened`, normalised in `execute_carbon_run`, `src/handlers/workspace/carbon.rs:1617` [K: replaced]) is removed, so the gate decision records the verdict on the document that is published.
3. The platform's arithmetic stays platform-owned. `DERIVATIONS` (LLD §5.7) is kept and becomes `derivation:<name>@<version>` activities with field-level `wasDerivedFrom` edges from each derived value to its inputs (§6.3). A CO_2e total is therefore grounded only if every factor and quantity it was computed from is grounded (all-of fold).
4. Each emission factor is a claim (`kind: assertion`, Truth against the factor source artifact, Grounding to the `web_search` / dataset `source:` node). The `CROSS_CHECK_EXEMPTIONS` for `source_url`, `dataset` and `reference_year` (`src/grounding_trust.rs:516-560` [K: removed]) go, because their blocker no longer holds [M R4.8].
5. `carbon_emission_factors` and `workspace_action_log` gain `episode_id NOT NULL REFERENCES proj.episodes`. Legacy rows are marked `episode_id_unrecoverable` [M R5.7]. `workspace_action_log` splits into an immutable `workspace_actions` log and a mutable `workspace_action_state` (OD-26) [M §16.2].
6. Publication is one command, `publication.release`, and one transaction.

Release DDL:

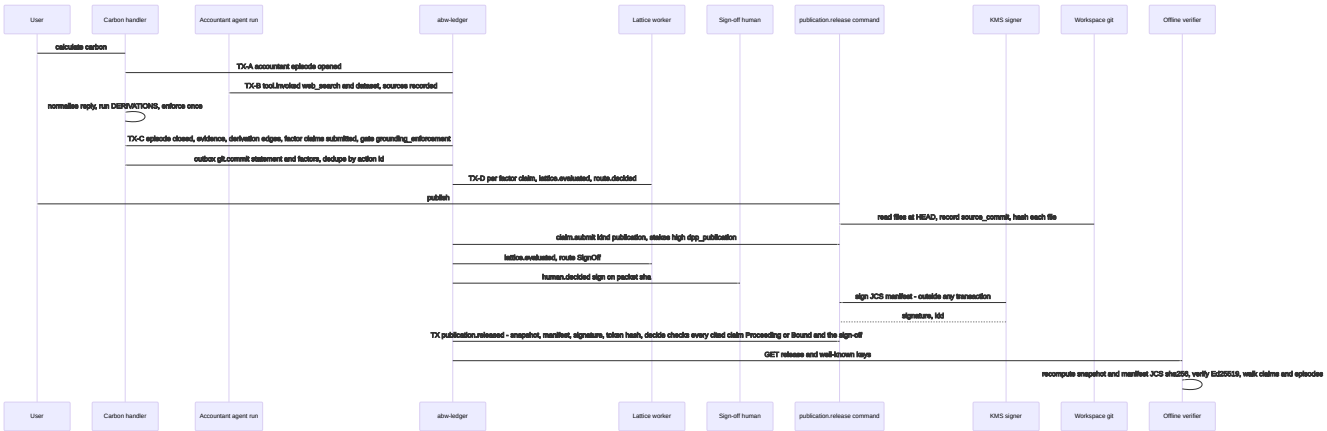
```
CREATE TABLE proj.publication_releases (
  publication_id uuid NOT NULL,
  release        int  NOT NULL CHECK (release >= 1),            -- = seq of
publication.released
  tenant_id      text NOT NULL,
  snapshot_sha   text NOT NULL REFERENCES ledger.artifacts ON DELETE RESTRICT,
  manifest_sha   text NOT NULL REFERENCES ledger.artifacts ON DELETE RESTRICT,
  source_commit  text NOT NULL CHECK (source_commit ~ '^[0-9a-f]{40}$'),
  kid            text NOT NULL,
  signature      text NOT NULL,                                 -- base64url
Ed25519, 64 bytes
  released_at    timestamptz NOT NULL,
  event_pos      bigint NOT NULL UNIQUE REFERENCES ledger.events (global_pos),
  PRIMARY KEY (publication_id, release));
CREATE TRIGGER t_releases_immutable BEFORE UPDATE OR DELETE ON
proj.publication_releases
  FOR EACH ROW EXECUTE FUNCTION ledger.forbid_mutation();
CREATE CONSTRAINT TRIGGER t_releases_justified AFTER INSERT ON
proj.publication_releases
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('event_pos');

CREATE TABLE proj.publication_tokens (
  token_sha      text PRIMARY KEY CHECK (token_sha ~ '^sha256:[0-9a-f]{64}$'),   --
plaintext shown once
  publication_id uuid NOT NULL,
  event_pos      bigint NOT NULL REFERENCES ledger.events (global_pos));
CREATE CONSTRAINT TRIGGER t_tokens_justified AFTER INSERT ON proj.publication_tokens
  DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
proj.row_justified_by_event('event_pos');
```

The release manifest is an artifact (`application/vnd.abw.dpp-manifest+json`, JCS). The signature covers its JCS bytes:

```
{"publication_id":"…","release":4,"released_at":"2027-05-02T09:00:00Z",
 "snapshot_sha":"sha256:…","source_commit":"3f2a91c…(40 hex)",
 "sources":
{"statement.yaml":"sha256:…","claims.json":"sha256:…","composition.json":"sha256:…",
            "evaluated/factors.json":"sha256:…"},
 "episodes":["…"],"claims":
[{"cid":"…","state":"Proceeding","vector_sha":"sha256:…"}],
 "signoff":{"human_decided_pos":918221,"actor":"user:…"},
 "kid":"abw-dpp-2027-01"}
```

The signature is `Ed25519(sk[kid], JCS(manifest))` [M R5.8]. The public keys are served at `/.well-known/abw-dpp-keys.json` as `{"keys": [{"kid","alg":"Ed25519","x","not_before","not_after"}]}`, and a rotated key stays listed for verification.



Consistency boundaries:

Boundary	Transaction	Notes
Accountant run	TX-A, TX-B_i, TX-C	Factors, derivations and claims commit with the episode
Git write of statement and factors	Outbox job after TX-C	`dedupe_key = git:<workspace>:<action_id>`. The job checks the target tree first, so a retry does not duplicate the commit
Factor claim evaluation	TX-D each	Independent claims; no ordering among them
Publication claim	TX-C-like `claim.submit` + TX-D	Stakes `high` (`dpp_publication`) forces SignOff [M §15]
Signing	**Outside** any transaction (KMS call)	The manifest is fixed before signing. The release transaction re-reads every cited file's artifact and fails if `source_commit` or any `sources` SHA no longer matches what was signed, so a race with a concurrent git write fails closed
Release	One transaction: `W = {publication:<id>}`, `R = {claim:<cid> for each cited claim, claim:<publication claim>}`	`decide` requires every cited claim to be `Proceeding` or `Bound` and the publication claim signed off. Snapshot, manifest, release row and token hash all commit together

Offline verifier (`abw dpp verify <url>`) [M R5.8]:

1. `GET` the release, the snapshot bytes, the manifest bytes and `/.well-known/abw-dpp-keys.json`.
2. Recompute `sha256(snapshot)` and `sha256(JCS(manifest))`, and check them against the manifest and the release.
3. Verify the Ed25519 signature with the key for `kid`, valid at `released_at`.
4. Optionally, `--deep` fetches each cited episode's lineage (`GET /api/episodes/:id/lineage`, tenant-authorised or public-token-authorised) and each

claim's replay result (`GET /api/claims/:cid/replay`). It then confirms that the vectors reproduce and that each published value walks to an external origin.

Steps 1–3 need only the public key: no database access, no trust in the server beyond bytes.

Publication tokens are stored as `sha256:` hashes. The plaintext is shown once, at creation [R §4 G20 "Hash the plaintext token now"].

---

11. Loops and policy

11.1 The three loops

[S §Loops]: each loop has an input, a thing it changes, and a metric.

Loop	Input events	Projection (async unless noted)	Changes
Inner, per claim	`correction.requested`, `claim.resubmitted`, `lattice.evaluated`	async read model `proj.loop_inner(dim, failure_class, policy_ref, attempts, recovered, n)`	The claim, on resubmission
Middle, per agent, endorser and judge	`outcome.observed`, `human.decided`, `judge.reported`	`calibration:<subject>` **streams** (inline, written in the outcome transaction) and async read model `proj.calibration(subject, a, b, brier_mean_micro, bias_ppm, n)`	Reliability posterior; forecast bias
Outer, per dimension and route	`false_pass.detected`, `false_fail.detected`, `outcome.observed`, `route.decided`, `endorsement.requested`	async read model `proj.loop_outer(policy_ref, dim, stakes, false_pass, false_fail, checked, escalations, open_packets)`	θ and the routing policy, **by proposal**

Reliability posterior (middle loop). The posterior for a subject is `Beta(a, b)`:

- `a` counts outcomes where the subject's forecast or endorsement held;
- `b` counts those where it did not.

It starts at `Beta(1, 1)`. Brier is the mean of per-claim `(p − o)²`. **Human endorsers are in the middle loop**: an endorsement is scored as a forecast that the claim will hold, with `p = 1` unless the endorser supplied a confidence [S §Loops].

For the **judge**, each finding with severity `block` or `warn` is scored as a forecast that the claim will fail, with `p = confidence_ppm`.

11.2 θ as versioned data

The policy document (`ledger.policy_versions.doc`) is JCS-hashed and immutable:

```
{"policy_kind":"lattice",
 "thetas_bp":{"truth":7000,"grounding":6500},
 "accept_human_attested":{"routine":true,"high":false},
 "stakes_rules":[{"class":"money_transfer_above","credits":500},
{"class":"dpp_publication"},
                 {"class":"irreversible_external_effect"},
{"class":"tenant_flagged"}],
 "correction_budget":3,
 "human_load_ceiling":{"per_assignee_open":50,"per_tenant_open":200},
 "judge_policy":{"families":{"anthropic-claude":["claude-"],"openai-gpt":["gpt-
","o1","o3"],
                 "mistral":["mistral-"],"qwen":["qwen"],"glm":["glm-"]},
                 "allowed_judges":[{"family":"openai-gpt","model":"…"},
{"family":"mistral","model":"…"}],
                 "prompt_lineage":"judge-v3","timeout_ms":60000},
 "routes":["R0","R1","R2","R3","R4","R5","R6","R7"],
 "liveness_vacuous_ok":["forecast"],
 "targets":{"false_pass_bp":{"truth":300,"grounding":500}}}
```

The gate constants listed in LLD §6.9 move to `policy_kind = 'gate'` documents with the same lifecycle [M R7.10]:

- coherence threshold 0.5 (`agent-bestiary/coherence-gate/src/gate.rs:66`);
- pre-filter 0.5 (`agent-bestiary/evaluators/src/registry.rs:78`);
- conflict 0.20;
- `UNIVERSAL_SHARE`;
- `CORROBORATION_BAND`.

Each gate decision records `policy_ref`. **Every verdict and route records `theta_version`** = `<policy_id>@<version>` (T11).

Resolution. The active policy for a claim is resolved at `claim.submit` as follows, and recorded in `claim.submitted`:

1. The tenant policy if one exists, else the platform default.
2. Within that policy, the highest version whose `active_from ≤ submitted_at`.

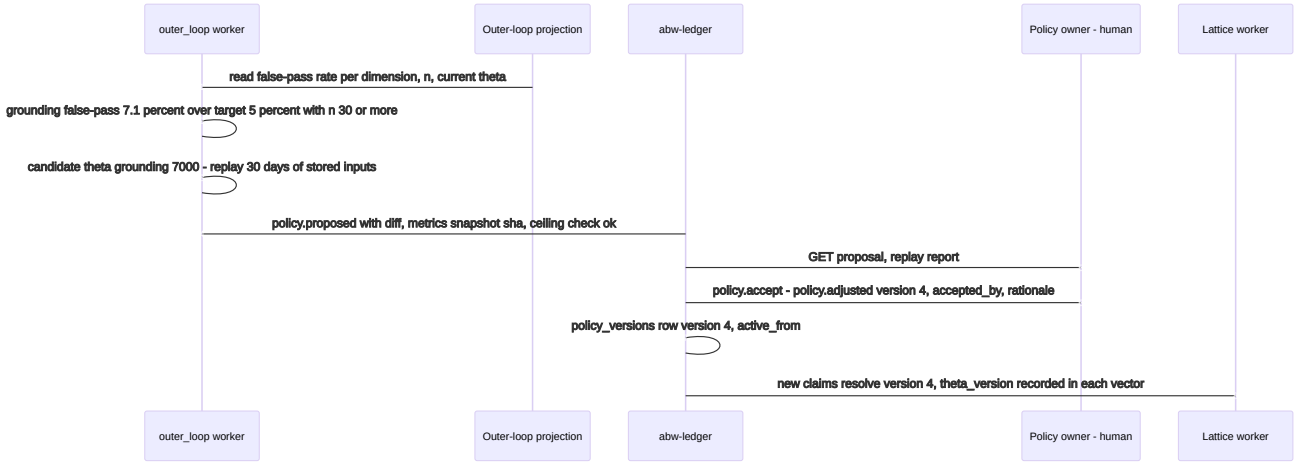
Re-evaluations of the same attempt use the same policy. A resubmission re-resolves.

11.3 Human-load ceilings

`human_load_ceiling` bounds open `AwaitingEndorsement` and `AwaitingSignOff` packets, per assignee and per tenant [M §15].

- **At routing time**, TX-D counts open packets in `proj.claims` through the `claims_queue` index, under the snapshot. If every eligible assignee is at the ceiling, R5/R6 become `Park { class: human_capacity }` with owner notification. The claim does not silently Proceed (T14).
- **At proposal time**, a proposed θ change is replayed over the last 30 days of claims, by re-running `evaluate` and `route` on stored `LatticeInputs` with the candidate policy. If the projected per-tenant open-packet peak exceeds the ceiling, the proposal is rejected *as a proposal* (`policy.proposed { ceiling_check: {ok: false} }`). It is shown, but cannot be accepted without also raising the ceiling, which is itself a human-accepted policy change.

11.4 The outer loop proposes; a human accepts



- **Triggering.** The outer loop emits `policy.proposed`, **never** `policy.adjusted` [M R7.11] (T20). The trigger is a dimension whose false-pass rate exceeds its target with at least 30 samples.
- **Acceptance.** `policy.accept` is a human-only command (`actor_kind = 'human'` enforced in `decide`). It appends `policy.adjusted`, whose payload holds the spec's "dim or route, from, to, cause" fields [S §Event schema], and writes the `policy_versions` row in the same transaction.
- **Constraint.** The spec's "improves one without breaking an agreed ceiling on the other" is the acceptance check: the replay report must show `false_pass` improved and the human-load projection within ceiling.
- **Automatic adjustment within ceilings** is out of scope for v1 (OD-27) [M §16.7].

---

12. Extension surfaces in the target

12.1 Tools

The kept trait (`src/agent_backend/tools/platform_tool.rs:126-182` [K]) gains two **required** methods:

```
#[async_trait] pub trait PlatformTool: Send + Sync {
    fn name(&self) -> &'static str;                          // kept: the stable
contract string
    fn version(&self) -> u32;                                // NEW, required (G33)
    fn origin(&self) -> ToolOrigin;                          // NEW, required, no
default (6.4)
    fn description(&self) -> &'static str;
    fn input_schema(&self) -> serde_json::Value;
    fn category(&self) -> ToolCategory;
    fn is_llm_visible(&self) -> bool { true }
    fn requires_workspace(&self) -> bool { false }
    fn is_delegation(&self) -> bool { false }
    fn required_credential(&self) -> Option<&'static str> { None }
    fn response_shape(&self) -> Option<&'static ToolResponse> { None }   // now a
checked registry (7.2)
    async fn execute(&self, input: &serde_json::Value, ctx: &ToolContext) ->
Result<ToolOutput, ToolError>;
}
pub enum ToolOrigin { External(SourceKind), Platform, Model }
pub struct ToolOutput { pub body: String, pub locators: Vec<Locator> }   // locators
required non-empty for External
```

- **Compile-time check.** Adding `origin()` without a default makes every one of the 107 tools (LLD §2.5) fail to compile until it declares one. That is the enforcement mechanism [M R5.3].
- **Narrowed context.** `ToolContext` (`tools/context.rs:9` [K: narrowed]) loses `db: PgPool` and gains `read: ReadDb` . A tool that must write trust state returns a `ToolEffect` value, which the executor dispatches as a command in TX-B_i. No tool holds a write handle (T1).
- **Allowlist** per §4.3. `tool.invoked` records `tool_version` .

12.2 Evaluators and feeds

- **`EvalModel`** (`agent-bestiary/evaluators/src/model.rs:21` [K]) is kept. Its outputs are written as `observation.recorded { observer: eval:<name>@<version> }` events, with `eval_signals` as their projection. LLM-backed evaluators' narrative outputs are artifacts with `epistemic = 'interpretive'` [M R7.6.5].
- **Pre-filters fail closed.** A pre-filter `Err` produces `Undetermined` and stops dimensional scoring [R §4 G11]. This replaces the `_ => false` arm at `agent-bestiary/evaluators/src/registry.rs:127` [K: replaced].

- **The evaluator registry** keeps its two registration sites until a single registry is introduced. Merging them is outside this design's scope.
- **`Feed`** (`crates/posterior/src/feeds.rs:282` [K]) is kept. A feed that supplies data an agent's claim relies on is a tool with `ToolOrigin::External(SourceKind::Dataset)` for provenance purposes. Its fetch results are recorded as artifacts with `source:` nodes.

12.3 Apps and MCP

- **Apps.**
  - Manifests gain `manifest_version` (G33).
  - Validation stays in the pure `abw-apps-core` crate (`crates/abw-apps-core/src/lib.rs:465` [K]).
  - Spawn is the `app.spawn` command (§9.6). Auto-hire goes through the same authorisation as hiring [M R0.1.5].
  - The manifest and template are stored as artifacts and their SHAs recorded in `workspace.spawned`, so a later manifest edit does not change what an old workspace was spawned from.
- **MCP client.**
  - The client is kept (`src/agent_backend/mcp_client.rs`, StreamableHttp only). Endpoints are validated by `net_guard` at configuration write and at call time.
  - One `reqwest::Client` is kept per `(server, pinned address)` instead of one per RPC (`mcp_client.rs:686` [K: replaced]). Each client is built by `net_guard::client_for` with redirects disabled and the resolved IP pinned.
  - Remote tool outputs are artifacts with `ToolOrigin::External(SourceKind::Registry)` unless the server is declared `platform`.
- **MCP server.** The HTTP server (`src/handlers/mcp.rs:208` [K]) is kept. `tools/call` for a published tool is a command with an `episode:` stream of its own (`source_ref.kind = mcp`), charged through the same hold and settle. The stdio server is kept as is; it is single-tenant and outside the trust domain.

12.4 A2A

The kept wire format is A2A 1.0, with routes at `src/api_server.rs:2864-2869` [K]. The behaviour changes as follows:

- **Task identity** is the `episode:` stream. Task states are `a2a.task_state { state: SUBMITTED|WORKING|COMPLETED|FAILED|CANCELED }` events on it.
- **Tenancy.** The poll (`GET /a2a/:slug/tasks/:id`) returns 404 unless `episode.tenant_id = caller`, `episode.agent_id = slug's agent` and `source_ref.kind = a2a` [R §4 G02].
- **Stored output.** The poll serves the **stored** enforced artifact, never a re-enforcement on read (`a2a.rs:608-623` behaviour removed) [M R0.2.3].
- **Deferred parts of G25.** `returnImmediately` and `CANCELED` are Deferred [R §4 G25]: "Defer. Except HMAC signing and credential encryption: do now". The target therefore

specifies them, but they are not part of any phase gate (OD-28). The design is an outbox `a2a.execute` job, and cancellation at the next turn boundary [M R2.9].

- **Webhooks (in scope now):**
  - Delivery is an outbox job `a2a.webhook`, enqueued in the transaction that appends the task-state event. This replaces `tokio::spawn` fire-and-forget (`src/a2a_webhook.rs:100-101` [K: replaced]).
  - Retries: 6 attempts, exponential backoff with jitter.
  - Each request is signed with `ABW-Signature: t=<unix>,v1=<hex(HMAC-SHA256(secret, t + "." + body))>` and carries `ABW-Idempotency-Key: <dedupe_key>`.
  - The per-config secret is returned once at creation and stored encrypted.
  - `auth_credentials` are encrypted with `SecretEncryptor` (AES-256-GCM, `fermi-auth/src/secrets.rs:11-30` [K]) [R §4 G25].
  - Each successful delivery appends `a2a.webhook_delivered { config_id, attempt, http_status }`.

12.5 Outbound SSRF guard

One module, `net_guard`, implements T18 [R §4 G03] [M R0.3]:

```
pub struct GuardedUrl { url: Url, pinned: SocketAddr }              // private fields; only check() builds it
pub enum NetGuardError { Scheme, Unresolvable, Forbidden { addr: IpAddr, class: &'static str }, Port, Redirect { location: String } }
pub async fn check(url: &str, policy: &OutboundPolicy) -> Result<GuardedUrl, NetGuardError>;
pub fn client_for(g: &GuardedUrl, timeout: Duration) -> reqwest::Client;   // resolve() pin, redirect::Policy::none()
```

- **Denied ranges:**
  - loopback; unspecified;
  - RFC 1918; CGNAT 100.64/10;
  - link-local 169.254/16 and fe80::/10;
  - ULA fc00::/7;
  - multicast; broadcast;
  - documentation ranges;
  - IPv4-mapped and IPv4-compatible IPv6 forms of any of these.
- **Ports.** Only 443 is allowed, plus 80 when `ABW_ALLOW_HTTP=1`.
- **When it runs.** URLs are checked at **config write and at call time**. The second check defeats DNS rebinding.
- **Redirects** are failures carrying `Location`. A caller that wants to follow one (the DOI resolver) re-checks each hop.

- **Dev override.** `ABW_NET_GUARD=off` is honoured only when `ABW_ENV=dev`. Boot refuses `off` in any other environment.
- **Refusals** are `gate.decided { gate: net_guard, decision: refused, reason_class: policy }` on the stream of the command that wrote the URL. For call-time refusals, the record goes on the episode stream, with `tool.invoked { error_class: net_guard }`.
- **Coverage test.** `tests/claims/ssrf_guard.rs` enumerates every `reqwest` call site and requires each to be classified `guarded` or `fixed_host`.

12.6 Contract versioning (G33)

G33 is Defer with "Versioning is designed into the ledger's event types in Phase 1" [R §4]. The target's contract-versioning mechanism is therefore the event type registry:

Contract	Version carrier	Rule
Events (the primary contract)	`(event_type, schema_version)` in `ledger.event_types`	A breaking change is a new `schema_version` with an upcaster from `n−1`. Old versions remain readable forever, and a version may be retired for *writing* (`event_type_retirements`) but never deleted
REST	`/api/v1/...` for every route introduced from Phase 1 on [M App. A G33]. Response bodies that embed events carry `schema_version`	Existing unversioned routes are frozen and served alongside
App manifests	`manifest_version`	`abw-apps-core` validates per version
Tools	`PlatformTool::version()`, recorded in `tool.invoked`	A card's allowlist MAY pin `name@version`
Judge	`prompt_lineage` + `prompt_sha` + model	Recorded in `judge.reported`
Lattice	`evaluator_version` in `lattice.evaluated`	A change in `evaluate` semantics bumps it. Replay of an old vector uses the old evaluator, kept as a versioned function table: `evaluate_v1`, `evaluate_v2`, … [P OD-29]
Policy	`theta_version`	§11.2

13. Security model

13.1 Tenancy

T17 states the rule: a read by opaque ID returns **404** unless the caller is authorised. It is 404, not 403, because 403 would be an existence oracle [R §4 G02] [M R0.2].

- **Where the tenant lives.** Every trust stream has `streams.tenant_id`, and every inline projection carries `tenant_id`.
- **Authorisation function.** Authorisation is one function per object kind: `authorize_read(principal, kind, id) -> Result<(), NotFound>` in `fermi-http::authz`. It reuses `fermi_auth::visibility` (`fermi-auth/src/lib.rs:13` [K]) for sharing rules. It is applied by an Axum extractor (`Authorized<Kind>`) that resolves the path ID, so a handler cannot receive an ID it has not authorised.
- **Coverage.** `tests/claims/idor_matrix.rs` enumerates every `GET` route with a `:…id` parameter from the route table (the existing parser at `tests/route_table.rs:31-40` [K]). For each one it asserts that a second user gets 404, with zero exemptions unless a written reason is in the test.
- **Provenance and lineage reads** (`/api/prov/:node_id`, `/api/episodes/:id/lineage`, `/api/claims/:cid/replay`) are tenant-scoped by the node's `tenant_id`. Public DPP releases are reachable through their publication token (hashed) or their public flag. The public flag exposes only the manifest, the snapshot and the claims the manifest lists.
- **Row-level security as defence in depth.** `abw_read` sessions set `SET LOCAL abw.tenant = $principal_tenant` per request transaction, and RLS policies on `proj.*` filter by it. This is enabled from Phase 3, once every read path sets the variable [P OD-30].

13.2 Database roles

The roles below were validated: Appendix D cases 1–13 and 37–40.

Role	Login	Owns	Privileges	Us
`abw_owner`	yes (migrations only)	Every table, function, trigger	DDL	Migration r `MIGRATION_`
`abw_app`	yes	nothing	`SELECT` on `ledger`, `proj`, `prov`. `INSERT` on `ledger.events`, `ledger.artifacts`, `ledger.streams`, `ledger.command_dedup`, `ledger.aggregate_snapshots`, `ledger.outbox`. `UPDATE (head)` on `ledger.streams`. `UPDATE` of outbox status columns and projector checkpoint columns. `INSERT` on `prov.*`. `INSERT, UPDATE` on non-money `proj.*` (guarded by justification triggers). **No `UPDATE` / `DELETE` / `TRUNCATE` on any history table, and no write on `proj.wallets` or `proj.credit_holds`.** May `SET ROLE abw_crud`	`abw-ledger`
`abw_crud`	**no**	nothing	`SELECT, INSERT, UPDATE, DELETE` on the CRUD tables in `public`. No access to `ledger`, `prov`, `proj`	`abw_ledger` `SET LOCAL` `abw_crud` i transaction
`abw_read`	yes	nothing	`SELECT` on `ledger`, `proj`, `prov`, `public`. Session default `default_transaction_read_only = on`; `statement_timeout = 60s`. Cannot `SET ROLE`	`ReadDb`

Boot checks [M R2.8]:

- `DATABASE_URL`'s role must not own any table, because ownership bypasses `REVOKE`. That is why the current `REVOKE … FROM PUBLIC` in migration 135 is very likely a no-op (LLD §4.4).
- `abw_read` must not be able to `INSERT` into `ledger.events`, which is probed with a rolled-back transaction.
- Boot fails if either check fails.

Why `abw_app` has `UPDATE (head)` on `streams`. The CAS needs it, and the monotonic trigger restricts it to forward moves of `head` only (§3.1.2; case 11).

13.3 Secrets and signing-key custody

Secret	Custody	Use
User secrets, A2A webhook secrets, A2A `auth_credentials`, remote MCP credentials	Encrypted at rest with AES-256-GCM (`SecretEncryptor`, `fermi-auth/src/secrets.rs:11-30` [K]). The data key comes from `SECRETS_ENCRYPTION_KEY`, wrapped by a KMS key-encryption key in production [P OD-31]	Decrypted in memory per request; never logged; never in event payloads (events carry a `secret_ref`)
DPP signing key	KMS asymmetric key (Ed25519) referenced by `DPP_SIGNING_KEY` = KMS key ID. The private key never enters the process. Rotation per OD-32 [M §16.5]	`publication.release` signs through the KMS API outside the transaction (§10.2)
JWT secret, OAuth client secrets, Stripe secret	Environment, from the platform secret store	Unchanged
API keys	Kept: `ferm_` + 64 hex from `OsRng`, argon2 hash. **Scopes intersect** with the role, never union [R §4 G27]	—
CLI login	OAuth-style code exchange with PKCE, and `state` from a CSPRNG. The API key is never in a redirect query string [R §4 G27]	—

Secrets never appear in artifacts or events. `NewEvent::new` runs a deny-list scanner over payload strings: provider key prefixes, `ferm_` keys, and JWT shape. A hit is rejected as `EnvelopeError::SecretLike`.

13.4 Rate limiting classified by handler

G17 is "Do now: classify LLM spend by handler, not by path allowlist" [R §4]. The current `LLM_SPEND_ROUTES` path list (`src/api_server.rs:244` [K: deleted]) is replaced by a **type-level permit** [M R0.8], in a form adjusted to the directive [P OD-33]:

```
pub struct LlmPermit { principal: PrincipalId, class: SpendClass, _private: () }
// only the limiter constructs it
pub enum SpendClass { Interactive, Workspace, Evaluation, Worker }
impl RateLimiter { pub async fn acquire_llm(&self, p: &AuthPrincipal, class:
SpendClass) -> Result<LlmPermit, RateLimited>; }
impl ToolAwareExecutor { pub async fn execute(&self, permit: LlmPermit, agent:
&AgentStmt, ctx: &ExecutionContext) -> Result<AgentOutput, ExecutionError>; }
impl JudgeClient        { pub async fn report(&self, permit: LlmPermit, input:
JudgeInput) -> Result<JudgeReportedV1, JudgeError>; }
```

- **Every handler is classified.** Each Axum handler is registered through `route_llm::<Class>(path, handler)` or `route_plain(path, handler)`.
  - Only `route_llm` handlers receive a `PermitSource` extractor.
  - Without a permit, `execute` cannot be called, so an unclassified LLM path does not compile.
  - This covers the workspace message path and `/coherence/evaluate`, which today run the tool loop unthrottled (LLD G17).
- **Worker LLM calls** (judge, `claim.correct`) acquire `SpendClass::Worker` permits from a per-tenant budget. This keeps the correction loop bounded even with no human in it.
- **Storage.** Until Phase 3 the limiter is the kept in-process sliding window (`src/api_server.rs:110-114` [K]), which is correct under the single-replica fence. In Phase 3 it moves to Postgres or Redis (§2.6).
- **Recording.** A rate-limit refusal is a Counted control. It returns 429 and appends nothing, since no trust effect occurred, and it increments a `telemetry` stream counter (`ExpectedCommutative::Any`).

13.5 Outbound guard

See §12.5. Every user-supplied or tenant-stored URL reaches the network only through `net_guard::client_for`. This covers:

- remote MCP endpoints;
- A2A webhooks;
- fetch-style tools;
- DOI and URL resolution.

---

14. Migration from current state

The migration is strangler-style. The new substrate is built clean, and existing code moves behind it one vertical slice at a time. Each phase ends at a gate, and the next phase does not start until every acceptance item holds [R §5].

14.1 Phase plan and acceptance gates as tests

Phase	Work (from [R §5] and [R §4] dispositions)	Acceptance item [R §5]	Ch
0	Spec tracked and invariants numbered (both now tracked at `docs/architecture/` on the branch); agent instruction file; ladder renamed to `platform_health`; CI ratchets (committed: §14.6); `clippy::let_underscore_must_use` and `#[must_use]` on write results; all "Do now" items (G01–G07, G09–G14 as re-scoped, G16, G17, G24, G25 partial, G26, G27, G30, G31)	Spec tracked; invariants numbered and referenced from the agent instructions	`doc_claims::lat` `ls-files` contain cites T1…T20 and
		`git grep` finds no dimension names outside Lattice code and the spec	`reserved_dimens` 'Presence
		Ratchet job runs in CI and fails on a deliberate increase	`ratchet_selftes` patch adding one `scripts/ratchet`
		G01–G04 fixed, each with a test that fails on the old code	`app_spawn_fundi` (G02), `ssrf_guar` `money_atomic_ch`
1	`abw-ledger`: streams, events, artifacts, JCS, one hash format, typed versioned envelopes; `ReadDb` in `AppState`; triggers, `ON DELETE RESTRICT`, roles; inline projections in the append transaction	A handler that tries to write directly fails to compile	`trybuild` test `handler_write_f` handler calling `s` `…").execute(&sta` `read_query!("UP` expected error
		An `UPDATE` on `events` as the app role fails at the database	`db_roles::app_c` `abw_app`, `UPDATE` `42501`
		Concurrent appends with the same expected version: exactly one succeeds	`ledger::concurr` tasks append to exactly one `Ok`, `n, actual: n+1`

Phase	Work (from [R §5] and [R §4] dispositions)	Acceptance item [R §5]	Ch
		Artifact round trip: store, resolve by hash, bytes identical	`ledger::artifac` object-store path bytes; a tampere CHECK
2	Fermi slice with all five dimensions; tool invocations as events; θ in `ledger.policy_versions`; evidence grounding in shadow then enforce; judge independence; G14 becomes a stream; G23	Replay reproduces the vector exactly	`lattice::replay` fixture claims, `sha256_jcs(eval` `lattice.evaluate` `rederive` mode
		An `interpretive` event is never reachable in a Grounding walk	`prov::interpret` test generates ra judge nodes; for walk row is interp forbidden edge r
		Evidence-writing and verdict-writing code are separate modules; no function writes both	`arch::evidence_` (`cargo metadata`
		Every dimension returns one of the four states; missing substrate never yields pass	`lattice::missin` for each dimensi artifact, no strea no commitment)
		`lattice.evaluated` and `route.decided` commit in one transaction	`lattice::vector` events have equa between them le
3	DPP and carbon on the ledger; money as ledger events with a ledger-derived balance; gate decisions as events with their effect; remaining writers migrated; old tables become projections	A published passport traces to the episode, source commit and artifact hashes	`dpp::passport_t` on a fixture relea snapshot SHA an walks every publi and an external c
		Ratchet count of direct `UPDATE` outside `abw-ledger` reaches 0	`ratchet.py` met crate, = 0 for eve

Phase	Work (from [R §5] and [R §4] dispositions)	Acceptance item [R §5]	Ch
		Every question in LLD §7.4 answerable at strength "Strong"	`lineage::llld_7` row of the table i question from ev

14.2 LLD §7.4 questions at strength "Strong" (Phase 3 gate)

Question	Answered by	Why it is Strong
Which agent?	`episode.opened.agent_id`	Immutable event
Which configuration ran?	`episode.opened.card_sha` → card artifact	Content-addressed resolved card, stamped at run start
What was asked and said?	`input_sha`, `raw_output_sha`, `enforced_output_sha` artifacts	Immutable, hash-checked
Which tools, with what I/O?	`tool.invoked` events + artifacts	Immutable; used by grounding
Which model produced the embedding?	`embedding_provenance` (kept) and `episode.closed.embedding_model`	Transactional with the episode
What did the grounding_enforcement gate decide?	`gate.decided { grounding_enforcement }` in TX-C	Same transaction as the effect
Which fields are pending or settled?	`proj.claims`, `proj.claim_fields`, claim streams	Events with sequence order, not timestamps
Was the caller charged for it?	`wallet.settled { idempotency_key: episode:<eid>:settle }` in TX-C	Same transaction; unique key
Did a downstream artifact come from it?	`prov.edges` + `publication.released.sources` + factor rows' `episode_id` FK	FK + edges + signed manifest
Were the bytes altered since?	Artifact SHA recomputed (database CHECK for inline bytes, audit for external), plus signature	Content addressing + Ed25519

14.3 Tables that become projections

Current table	Becomes	Phase	Writte
`wallets`	Async mirror of `proj.wallets`	3	Projector c role loses
`credit_ledger`	Async mirror of `wallet:` events	3	Projector c immutable
`agent_versions`	Mirror of `proj.agent_versions`	2	Projector c
`agents` (config columns)	Display mirror of `proj.agent_heads`	2 (mirror) / 3 (columns dropped)	Projector c
`episodes` (trust columns)	Mirror of `proj.episodes`	2	TX-C inline projector
`assertion_verifications`	Async read model of `claim:` events	2	Projector c immutable
`gate_decisions`	`proj.gate_decisions`	2 (Recorded gates on the Fermi path) / 3 (all)	Inline, in th transactio
`gate_decision_reviews`	Read model of `gate.reviewed` events	3	Projector
`forecast_agent_claims`	Read model of `claim:` events with `kind = forecast`	2	Projector; immutable
`episode_corrections`, `hitl_actions`	Read models of `hitl.intervention_*` and `gate.decided` events	3	Projector
`carbon_emission_factors`	Read model of factor claims	3	Projector
`dpp_publications`, `dpp_publication_releases`	`proj.publication_releases`, `proj.publication_tokens`	3	Inline
`stripe_sessions_processed`	Read model of `wallet.deposited`	3	Projector

Current table	Becomes	Phase	Writte
`workspace_action_log`	Split: `workspace_actions` (immutable read model of `workspace.action_recorded` events) + `workspace_action_state` (CRUD)	3	OD-26
`embedding_provenance`	Kept, transactional sidecar; registered migration 136; unconditional CHECK after quarantine [M R3.7]	2	`abw_ledge`

14.4 Backfills that cannot be perfect

Every backfilled event carries `actor_kind = 'platform'`, `actor = 'backfill:<migration>'` and a payload field `"backfill": {"source": "<table>", "fidelity": "exact|partial|unrecoverable", "note": "…"}`. Backfilled events are never strength 2 unless their fidelity is `exact` and their artifacts were recoverable.

Data	Action	Marking
Episodes before migration 199 (no `response_text`)	`episode.opened` + `episode.closed` with no output artifacts	`fidelity: unrecoverable`; `artifacts_unrecoverable`. Presence fails for any claim citing them [M §13.4]
Episodes with `context.tool_invocations`	`tool.invoked` events reconstructed from the JSON, with artifacts of the stored I/O	`fidelity: partial` (the JSON was truncated at 32 000 bytes)
`agent_versions` before migration 240 (no whole card)	`agent.version_created { reason: backfill, card_sha: null }`	`card_incomplete: true`. Claims from those episodes get Presence `fail(card_unrecoverable)` if they cite the card
Carbon factors without a recoverable episode	Read-model rows with `episode_id_unrecoverable = true`	Excluded from DPP releases unless re-derived [M §13.4]
Wallets where `balance ≠ Σ credit_ledger.amount`	Opening `wallet.opened` + `wallet.reconciled { delta: balance_now }` from the issuer, with a report artifact	**Owner-approved** report; `idempotency_key = reconcile:<report_sha>:<wallet>`. History is never rewritten (OD-34) [M §16.8]
Gate decisions lost to write-behind	Not backfilled	The gap is stated in the release notes; no synthetic events
Hashes computed by `artifact_hash::canonical`	Recomputed with JCS; old value kept as `legacy_sha` for one release	[M R3.1]

14.5 CRUD-domain rules

The CRUD domain keeps mutable tables. T1 still holds: CRUD writes go through `abw_ledger::crud`, which is the only holder of a connection able to `SET LOCAL ROLE abw_crud`.

- **Mixed transactions.** A command that writes both trust and CRUD state does so in **one** transaction. For example, `app.spawn` writes `public.teams` and the wallet transfer.
  - The command runs `SET LOCAL ROLE abw_crud` for the CRUD statements and `RESET ROLE` before ledger statements.
  - The roles are disjoint: `abw_crud` cannot touch `ledger.*`, and `abw_app` cannot write `public.*`. A statement in the wrong role fails (validated, Appendix D cases 38–40).

- **Rules for CRUD writes:**
  - no `let _ =` on a write [R rule 2];
  - two writes that must agree go in one transaction;
  - optimistic concurrency (`WHERE version = $v`) on any row the UI edits concurrently.
- **API.** `Crud::tx(&self) -> CrudTx` exposes a typed query builder plus `execute_crud!("…")` . The macro refuses at compile time to name any table in the `ledger` , `prov` or `proj` schemas, or any table listed in `TRUST_TABLES` [P OD-35].

14.6 CI ratchets

`scripts/ratchet.py` exists on the integrity branch and runs as a blocking CI job (`scripts/ratchet.py:1-12` , `.github/workflows/ci.yml:19-25` [K]).

- **Scope.** It counts six metrics over production Rust in the workspace members. Excluded:
  - `tests/` directories and `tests.rs` files;
  - `#[cfg(test)]` items;
  - comments.
- **Check.** `--check` compares the counts against `scripts/ratchet_baseline.txt` and requires **exact equality**:
  - a rise is a regression;
  - a fall must lower the baseline in the same change, so an improvement is locked in (`scripts/ratchet.py:7-9` [K]).

The target keeps this mechanism and extends the metric set. The required trend is the directive's: counts may not rise, and they must fall with every phase [R rule 1].

Metric	Exists	Pattern	Baseline at `326` (`scripts/ratchet_bas` [K])
`update_ident`	yes	`UPDATE <ident> SET` and `DO UPDATE`	330 (origin/main: 335)
`delete_from`	yes	`DELETE FROM`	63
`let_underscore`	yes	`let _ =` and bare `_ =`	279 (origin/main: 290)
`let_underscore_sqlx`	yes	the subset discarding a `sqlx::` call	108 (origin/main: 115)
`row_get`	yes	panicking `row.get(..)`	707
`db_decode_unwrap`	yes	`try_get(..)` followed by `unwrap` or `expect` (optionally via `.parse()`)	58 (origin/main: 76)
`tokio_spawn_trust_path`	**add**	`tokio::spawn` in trust-path files without an allowlist reason	measured when adde

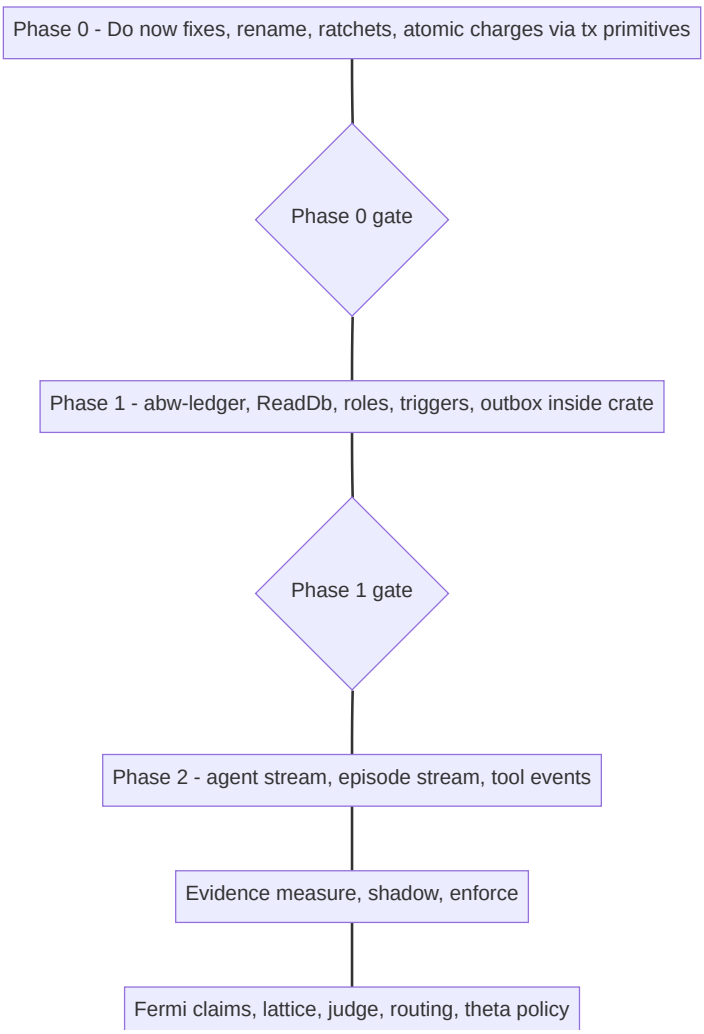
Metric	Exists	Pattern	Baseline at `326` (`scripts/ratchet_bas` [K])
`trust_table_write_outside_ledger`	add	SQL `INSERT`, `UPDATE` or `DELETE` naming a `TRUST_TABLES` entry outside `abw-ledger`	measured at Phase 1
`outbox_like_outside_ledger`	add	Tables or modules named `*outbox*`, `*queue*` or `*job*` outside `abw-ledger`	measured when adde
`advisory_named_as_control`	add	Types named `*Gate*`, `*Contract*`, `*Binding*` or `*Validation*` whose decision enum lacks a refusal path	measured when adde
`uncited_behaviour_claims`	add	Doc and comment sentences with “now”, “always”, “never” or “every” about behaviour, without `path:line` or `[I]`	measured when adde

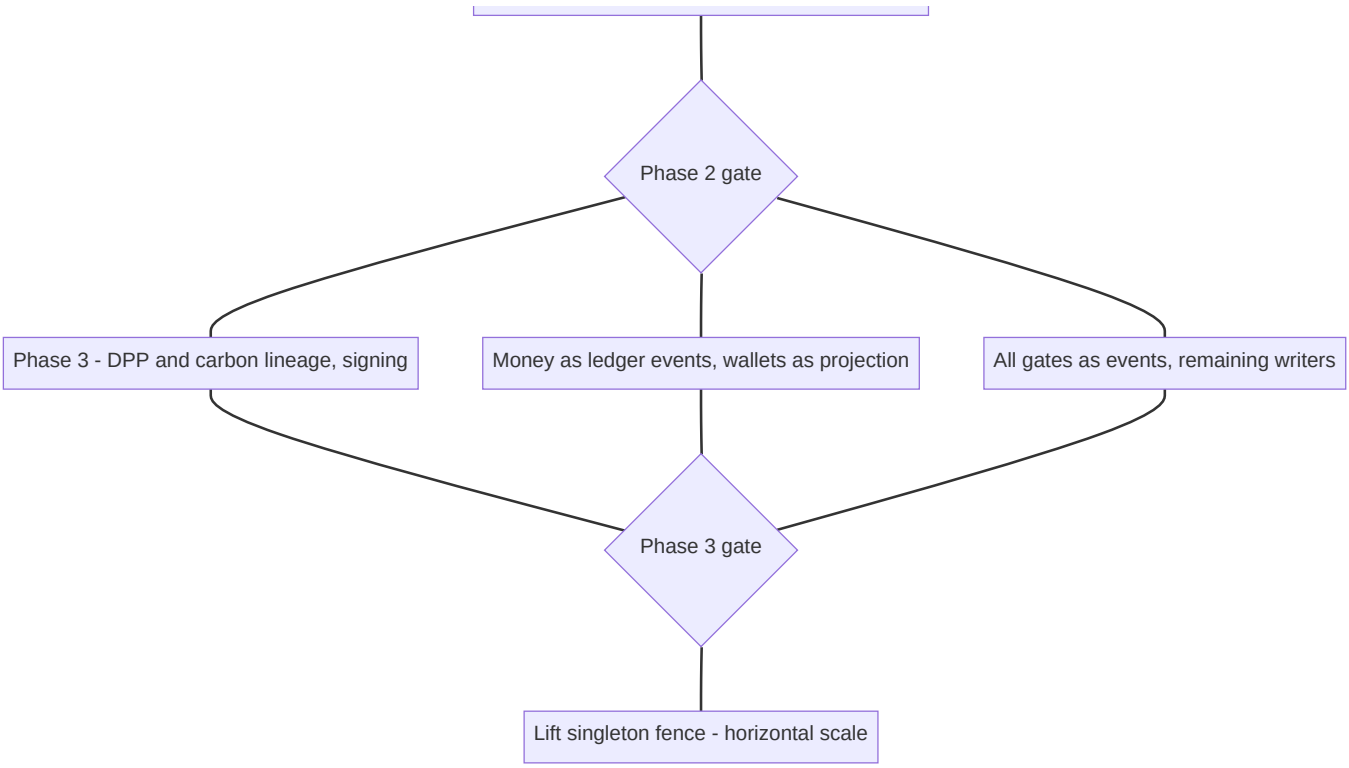
The Phase 0 acceptance item “ratchet job runs in CI and fails on a deliberate test increase” [R §5] is the test `ratchet_selftest`. It applies a fixture patch adding one `UPDATE foo SET` to a production file and asserts that `ratchet.py --check` exits non-zero.

The freeze list is enforced as review checks plus ratchets [R §2]:

- no new `const` classifier tables in `*_trust`;
- no `command_registry` entries without a `Command` impl (`tests/claims/trust_commands_dispatch.rs`);
- no counter sinks (`write_accounting` frozen);
- no outbox outside `abw-ledger` (metric above);
- no advisory check presented as a control (metric above).

14.7 Strangler sequence





---

15. Conformance test catalogue

Tests live under `tests/claims/` (an existing directory: `tests/claims/main.rs` [K]). Each test MUST exist and fail on the baseline before the change that makes it pass [M §Preamble]. "Fails today because" names the current defect the test detects.

15.1 Invariants

Inv.	Test	Asserts	
T1	`handler_write_fails_to_compile` (trybuild)	A handler writing through `state.db` or `read_query!("UPDATE…")` does not compile	`AppState.d` (`src/api_se` sites compil (`scripts/ra`
T1	`no_write_pool`	Only `abw-ledger` calls `PgPoolOptions::connect*` with the app URL	`src/api_se` pool, shared
T2	`concurrent_expected_version`	32 concurrent `Exact(n)` appends: one wins, 31 `Conflict`, head = n+1, gap-free	No streams §4.7: 0 optir
T2	`gap_free_streams_db`	Advancing `head` without events, or inserting `seq > head`, fails at commit	No `streams`
T3	`projection_requires_event`	As `abw_owner`, `UPDATE proj.claims` with no event in the transaction raises `ABW-T3`	No projectio mutable
T3	`projection_rebuild_zero_diff`	`abw-admin projection rebuild --diff` is empty for every projector on the fixture corpus	No projecto
T4	`jcs_vectors`	RFC 8785 vectors + V1–V9 of §3.6.2	`artifact_h` `1e+30` and (`src/artifa` hashes exis
T5	`one_run_one_transaction` (fault injection)	Abort at each statement of TX-C; afterwards no settle without close, no close without settle, no claim without close	Close, `work` gate decisio
T5	`gate_decision_atomic_with_effect`	For each Recorded gate, kill after the effect commit; the decision is present	15 s write-b (`src/gate_t`
T6	`app_cannot_update_events`, `owner_cannot_update_events`, `no_truncate`, `restrict_fk`	Each raises at the DB; `DELETE FROM agents` with history fails `23503`	No triggers `CASCADE` int (LLD §4.4)

Inv.	Test	Asserts	
T7	`observations_never_refuse`	No `gate.decided` row with `reason_class ∈ {advisory, infra_error, insufficient_input}` and `decision = refused`; DB CHECK rejects one	`InputBindi` (`src/handle` coherence i
T7	`guards_registry`	Every guard has a known-bad input that it refuses or marks `Undetermined`	Pre-filter `Er` `bestiary/ev`
T8	`evidence_verdict_separation`	Crate graph and signature scan (§7.1)	`grounding_` and amends (`src/ground`
T9	`wallet_ledger_identity`	For every wallet, the fold of its stream equals `proj.wallets`; Σ balance over all wallets = 0	`wallets.ba` recomputat
T9	`money_crash_injection`	Abort at each statement of hold, settle, transfer, deposit; identity holds	Two autoco the committ
T9	`no_mint`	A transaction with Σ delta ≠ 0 fails at commit	At `a3cea516` debit (LLD G uses paired rejects an u
T9	`idempotent_settle`	Settling the same episode twice yields one charge	No idempot
T10	`interpretive_unreachable`	Property test, §14.1	No `interpr`
T10	`claim_cannot_cite_judge`	`claim.submit` with a judge artifact in `artifact_refs` is rejected	No judge re
T11	`replay_bit_for_bit`, `rederive_inputs`	§5.6	No vector; θ `registry.r`
T12	`missing_substrate_never_passes`	§14.1	No lattice; `t` population (`src/ground`
T13	`no_short_circuit`	With Presence failing, Liveness, Grounding and Binding still carry computed raw states	The ladder a (`registry.r`

Inv.	Test	Asserts	
T13	`one_event_per_evaluation`	Exactly one `lattice.evaluated` per (cid, attempt, evaluation)	n/a: no ever
T14	`every_non_pass_has_diagnosis`, `never_dropped`	Schema requires diagnosis for non-pass; every claim reaches a terminal or waiting state with owner notification	`Automated` §6.6)
T15	`judge_cannot_certify`	`evaluate` has no judge parameter (compile-time); a `block` finding never yields Proceed	Judge score evaluators (
T15	`judge_independence`	Judge family ≠ proposer family; `judge.unavailable` routes to a human	Judge defau (`src/handle`
T15	`judge_injection`	Flip rate ≤ threshold (OD-14)	Output inte (`eval_judge`
T16	`reserved_dimension_names`	§14.1	`src/ladder` Presence...E
T17	`idor_matrix`	Every `GET …/:id` returns 404 for a second user	A2A poll rea (`src/handle`
T18	`ssrf_guard`	Rejects loopback, metadata, RFC 1918, mapped IPv6, rebinding stub, 302 to loopback	`reqwest::C` (`src/agent_`
T19	`brier_per_claim`	Two agents with different `p` on one forecast get different Brier	Forecast Bri (`src/outcom`
T20	`outer_loop_cannot_adjust`	The `outer_loop` worker's actor cannot dispatch `policy.accept`; `policy.adjusted` requires `actor_kind = human`	No policy ta

15.2 Phase acceptance items

All are listed in §14.1 with their tests. Two further tests guard design choices this document makes:

Test	Asserts	Fails today because
`async_fence_never_skips`	Reproduces §3.5's interleaving (early xid, late insert, a third open transaction); the projector consumes every committed event exactly once	No projector; and the SPEC 36 R2.6 fence would fail this test (Appendix B)
`prov_walk_plan_uses_index`	At 10^6 edges, `EXPLAIN` of the walk uses `edges_walk` and p99 < 20 ms per field	No provenance graph

---

16. Open decisions

Every [P] in this document is listed here, together with ambiguities in the sources. Each gives the options and a recommendation; none is silently decided. The document is written as if the recommendation were accepted, so rejecting one changes the cited sections.

ID	Decision	Options	Recommendati
OD-01	Worker topology before Phase 3	(a) `api+worker` in one process; (b) separate worker process	(a) until the singleton lifts: one process mea fence. (b) after
OD-02	Shared rate-limit store after Phase 3	(a) Postgres token bucket; (b) Redis	(a): no new infrastruc one `INSERT … ON CON` per LLM call is cheap to an LLM call
OD-03	JCS implementation (also [M §16.4])	(a) `serde_json_canonicalizer`; (b) in-tree, about 300 lines	(b): the number forma must be ECMAScript- and an in-tree implementation teste against V1–V9 and the vectors is auditable
OD-04	Stream for gate decisions with no effect stream (refusals before an effect exists)	(a) per `(gate, tenant, day)`; (b) per subject	(a): it bounds stream and avoids creating st for refused subjects
OD-05	Maximum inline event payload	16 / 64 / 256 KiB	64 KiB; larger content artifact
OD-06	Isolation for ordinary commands	(a) READ COMMITTED + explicit locks; (b) SERIALIZABLE	(a): conflicts become t `Conflict`s, not `4000` uses REPEATABLE REA snapshot inputs
OD-07	Numeric encoding in trust payloads	(a) integers (ppm, bp, ratios); (b) JCS floats	(a): exactness, and no dependence on float formatting across lan The spec's decimal ex is a rendering
OD-08	Event-schema source of truth	(a) Rust registry with the DB table checked at boot; (b) DB table generates Rust	(a)
OD-09	Compile-time read-only queries	(a) `read_query!` macro parsing the SQL literal; (b) runtime-only enforcement through the role	(a) **and** the role. The c requires a compile fa
OD-10	Crate split to remove `sqlx` from handlers	(a) `fermi-http` crate without `sqlx`; (b) lint only	(a), with `cargo deny`

ID	Decision	Options	Recommendati
OD-11	Liveness on claims that cite nothing	(a) `not_evaluated`; (b) `pass` when the kind declares `liveness_vacuous_ok`	(b) for `forecast` only `not_evaluated` other
OD-12	Streams for CRUD state that claims cite (`forecast:`, `workspace_output:`)	(a) add minimal streams that record revisions as artifacts; (b) forbid such claims	(a), limited to revision are scored or publish
OD-13	Mechanism for `evaluate` purity	(a) `clippy` disallowed-methods + no-`async` crate; (b) a WASM sandbox	(a)
OD-14	Judge injection flip-rate threshold ([M §16.6])	1% / 2% / 5% at a fixed seed	2%, as SPEC 36 sugge
OD-15	When Grounding failure routes to Endorse rather than Correct	(a) first failure is Correct unless the diagnosis says no source is obtainable; (b) always Endorse	(a): it gives the agent chance to attach a so the spec's first correc action
OD-16	Conditional commitment event shape	(a) `commitment.conditioned` appended at routing; (b) flag on `commitment.bound` decided at submit	(a): the condition is o known after routing
OD-17	Four-eyes on high-stakes endorsements	(a) assignee ≠ proposer's owner; (b) no rule	(a)
OD-18	Tool-call durability granularity	(a) one transaction per tool call; (b) buffer and write in TX-C	(a): evidence survives mid-run, and it match invocations are event
OD-19	Drop-guard mechanism	(a) `#[must_use]` guard with explicit `abandon` plus a channel to a closer loop, with the reaper as backstop; (b) `Drop` spawning a task	(a): no detached trust task; the record and e stay atomic
OD-20	Keep `raw_state` alongside `state`	(a) yes; (b) no	(a): it answers the spe open question 1 with losing information
OD-21	Retain source bytes for External tools	(a) always; (b) policy flag; (c) never	(b), default on for `dpp_publication` sta

ID	Decision	Options	Recommendati
OD-22	Evidence-grounding (`ABW_GROUNDING`) enforce switch ([M §16.3])	Owner approval after the backfill report	Required. No default
OD-23	`port_trust` enforcement: it would refuse 47 of 102 cards	(a) stay an observation; (b) enforce after cards are fixed; (c) enforce now	(b). Per [R rule 5] this reported, not decidec
OD-24	Issuer wallet for double entry	(a) single issuer wallet with negative balance; (b) single-sided grants exempt from double entry	(a): Σ = 0 globally is a checkable invariant
OD-25	Per-caller sponsorship quota, and sponsor-wallet granularity. The funding rule itself is **decided** by the owner (option c: platform sponsors the template budget)	Quota: (a) per-caller daily budget in policy; (b) per-caller spawn count; (c) none. Granularity: (i) one platform wallet `platform:app_sponsorship` , as built; (ii) one sponsor wallet per App	(a) with (i). This is SPE R0.1.5b's open owner question; the mechar in §9.6
OD-26	`workspace_action_log` ([M §16.2])	(a) split immutable log + mutable state; (b) drop the immutability claim	(a)
OD-27	Outer-loop autonomy ([M §16.7])	v1 proposes only; v2 auto-applies within ceilings	v1 only; revisit with 6 of `policy.proposed` acceptance data
OD-28	A2A `returnImmediately` and `CANCELED`	Directive: Defer	Specified in §12.4 but scheduled. Confirm th deferral
OD-29	Old evaluator versions for replay	(a) keep `evaluate_vN` functions forever; (b) re-evaluate history under the new version	(a): replay must be bit
OD-30	Postgres RLS as defence in depth	(a) Phase 3; (b) never	(a)
OD-31	Secrets KEK in KMS	(a) envelope encryption with KMS; (b) environment key only	(a) in production

ID	Decision	Options	Recommendati
OD-32	DPP signing key custody and rotation ([M §16.5])	KMS provider; rotation 6 or 12 months	KMS-held Ed25519, 1 rotation, old keys kep
OD-33	LLM permit form	SPEC 36 `LlmPermit` passed per call ([M R0.8]) vs a route-registration class	Both: the class is assig route registration and permit is required by `execute`
OD-34	Credit reconciliation sign-off ([M §16.8])	Who approves adjustment rows	Platform owner plus other admin
OD-35	Compile-time CRUD/trust separation	(a) `execute_crud!` macro rejecting trust schemas; (b) runtime only	(a) plus the role separ
OD-36	Hold caps ([M §16.9])	`EXECUTION_HOLD_CAP` and `APP_SPAWN_MAX_EXTRA_BUDGET` values	Keep the committed of 1000 for the extra (`src/apps/provision` hold cap = 5 × median cost per model
OD-37	Scope of event sourcing ([M §16.1])	Add forecast resolution events	Yes, as `forecast:` str in Phase 2 (OD-12)
OD-38	Source ambiguity: the directive's Phase 2 includes "Tool invocations are events"; SPEC 36 places artifacts in WS3 (Phase 2) but the Lattice in WS7 (Phase 4)	(a) follow the directive's phases; (b) SPEC 36's	(a), per authority
OD-39	Source ambiguity: the directive's Phase 2 acceptance says a missing substrate yields `unsupported`; the spec defines `unsupported` as "passed but a dependency failed" and `not_evaluated` as "precondition missing"	(a) use `not_evaluated` for missing substrate, and `unsupported` only per the spec; (b) use `unsupported` for both	(a): the spec ranks hig and the directive's int ("never `pass`") is pres

Appendix A: gap closure map

Gap	Directive disposition [R §4]	Closed by (this document)	Phase
G01 credit mint	Do now	Phase 0 fix committed at `3268680b` (sponsor plus caller transfers, §9.6 [K]); target §9.6 `app.spawn` plus §9.1 double entry (database rejects a mint); open quota OD-25	0 (done on branch) / 3 (ledger)
G02 A2A poll cross-tenant read	Do now	§13.1 tenancy extractor and 404 matrix; §12.4	0
G03 SSRF	Do now	§12.5 `net_guard`, T18	0
G04 money atomicity	Do now; completes in Phase 3	§9.2–§9.4: hold in TX-A, settle in TX-C, ledger-derived balance	0 / 3
G05 `running` unparseable	Do now	§4.6 status enum includes `running`; ratchet `unwrap_on_db_decode`	0
G06 UTF-8 slice	Do now	§4.4	0
G07 orphan `running` rows	Do now	§4.5 ⟨6⟩ `EpisodeGuard`, closer loop, reaper	0 / 2
G08 gate write-behind	Re-scope: interim per-row fallback, no new outbox; events in Phase 3	§8.1 decisions in the effect transaction; queue deleted	0 (interim) / 3
G09 `tool_verified` content proxy	Do now (bridge)	§7.3, §7.5 B1 → B3	0 / 2
G10 consensus re-run	Do now	§8.3 replay plus re-evaluation	0
G11 pre-filter fails open	Do now	§12.2	0
G12 judge independence and injection	Do now	§5.9 rules 1–3, `interpretive` type	0 / 2

Gap	Directive disposition [R §4]	Closed by (this document)	Phase
G13 seeder overwrites edits	Do now	§4.2 seed on insert only; field origins	0 / 2
G14 versions not identifying	Re-scope: JCS hash identity; stream in Phase 2	§4.2 `card_sha`, `agent:` stream	0 / 2
G15 append-only by comment	Phase 1	§3.1 triggers, §13.2 roles, `ON DELETE RESTRICT` everywhere	1
G16 lenient migrations	Do now	§14.6 note below: the runner skips on recorded success plus checksum, fails boot on failure, and the duplicate DDL path is deleted [M §12]	0
G17 unrated LLM routes	Do now	§13.4 classification by handler	0
G18 no streams	Phase 1	§3.1.2, §3.2	1
G19 no artifact store	Phase 1	§3.1.5	1
G20 DPP lineage, signing, token	Phase 3; hash token now	§10.2	0 (token) / 3
G21 per-process state	Defer: single replica until Phase 3 completes	§2.6 fence now; scale-out path listed	Deferred (fence 0, externalisation 3)
G22 hash formats	Phase 1	§3.6	1
G23 constant θ, no loops	Phase 2	§11.2, §11.4	2
G24 every agent gets every tool	Do now	§4.3	0

Gap	Directive disposition [R §4]	Closed by (this document)	Phase
G25 A2A semantics	Defer, except HMAC and credential encryption: do now	§12.4 webhooks signed and encrypted, via outbox; `returnImmediately` and `CANCELED` deferred (OD-28)	0 (partial) / Deferred
G26 attachments dropped	Do now	§4.4	0
G27 CLI key in query, merged scopes	Do now	§13.3	0
G28 consolidation lease	Defer	Deferred: no consolidation work is scheduled. The projector lease pattern (§3.5) is the template when it is picked up	Deferred
G29 dead columns, wrong export audit	Defer: do not write these columns in the old model	§6.6 written only through the ledger in Phase 3	Deferred to 3
G30 misleading `Refused`	Do now	§8.2, §8.4; DB CHECK on `proj.gate_decisions`	0
G31 per-client `PgListener`	Do now	§3.2.1 step 14: `NOTIFY abw_events` into the shared listener; the SSE handler uses the `pg_notify` broadcast	0
G32 editor tooling drift	Defer	Deferred: outside the trust domain; no invariant depends on it	Deferred
G33 no contract versioning	Defer; designed into event types in Phase 1	§3.1.3, §12.6	1

The G16 runner (from [M §12]) works as follows:

- a migration whose recorded `content_sha256` matches, and which has `first_succeeded_at`, is skipped;

- a changed checksum on an applied migration fails boot;
- any failure fails boot;
- `SCHEMA_STRICT` becomes the default and the flag is removed;
- `ensure_critical_schema` (`src/api_server.rs:1615` [K: deleted]) moves into a numbered migration.

The runner lives in `abw-ledger::migrate` and connects as `abw_owner`.

Appendix B: divergence from SPEC 36

SPEC 36 is rank 3. Where it conflicts with the Lattice spec or the directive, this document follows the higher rank.

#	SPEC 36 says	This document says	Reason
B1	`CommandBus::dispatch` in the application (§5.1, R2.4), with commands, aggregates and projectors in the root crate	`Ledger::dispatch` **inside** **`abw-ledger`**. Command types live in domain crates that depend on `abw-ledger`; the write handle never leaves the crate	[R §5 Phase 1]: "The crate is the only code that holds a write-capable pool handle". An in-process bus outside the crate would need a handle
B2	`outbox` table described in WS2 (R2.7) without a crate boundary; outbox workers anywhere	`ledger.outbox`, written only by `abw-ledger`; handlers enqueue only through `NewEvent.outbox`	[R §2 freeze 4]: no outbox, queue or event infrastructure outside the ledger crate
B3	Ladder rung renames `schema_exists`, `sink_fired`, `rollup_consistent`, `field_contracted`, `port_declared` (R7.1)	`platform_health` rungs `schema_present`, `write_path_live`, `rollup_consistent`, `field_tool_sourceable`, `input_port_match`	[R §5 Phase 0] names these; this design treats the directive's example as normative
B4	`LlmPermit` type passed to `execute` (R0.8) as the whole mechanism	Classification by handler at route registration, **and** a permit required by `execute` (OD-33)	[R §4 G17]: "Classify LLM spend by handler, not by path allowlist". R0.8's permit does not by itself classify handlers
B5	Phases 0–5 with the Lattice in Phase 4 (§14)	Directive phases 0–3; the Lattice in Phase 2 with Fermi, DPP and money-as-ledger in Phase 3	[R §5]
B6	Money atomicity via `charge_tx`, and a `credit_ledger_apply` BEFORE INSERT trigger on `credit_ledger` in Phase 1 (R1.1–R1.2)	Phase 0: `*_tx` primitives (G04 Do now). Phase 3: wallets are `wallet:` streams and `proj.wallets` is applied by trigger from `ledger.events`; `credit_ledger` becomes a projection	[R §4 G04]: "Moves into the ledger in Phase 3"

#	SPEC 36 says	This document says	Reason
B7	`ExpectedVersion::Any` permitted for commutative kinds	Kept, but as a separate type usable only on `telemetry` streams	Compile-time enforcement instead of a runtime check
B8	Async projector fence: `global_pos > checkpoint AND txid < xmin` (R2.6)	`(txid, global_pos) > (fence_txid, fence_pos) AND txid < xmin`, ordered by `(txid, global_pos)`	SPEC 36's form skips events when xid order differs from insert order; reproduced in Appendix D case 26
B9	`lattice.deferred` if evaluation exceeds 200 ms in TX2 (R7.5)	Lattice always in its own TX-D after the judge	"Routing waits for both the lattice evaluation and the judge report" [S §Routing], and the judge is an LLM call that may not run inside a transaction
B10	Gate decisions in Phase 2 (WS6)	Phase 3 ([R §5 Phase 3] "Gate decisions as events written with their effect (completes G08)"). Phase 2 does it for the Fermi path's gates	[R §4 G08], [R §5]
B11	`prov_edges` rejects interpretive objects only for `wasDerivedFrom`, `hadPrimarySource`, `quotedFrom` (R5.2)	Adds `edges_interpretive_unreachable` (no evidentiary node may point into an interpretive one except by informational relations), and composite FKs on epistemic class	T10 requires unreachability "in any Grounding walk". R5.2 alone allows `used` and `wasGeneratedBy` edges into interpretive nodes, which a looser walk could traverse

#	SPEC 36 says	This document says	Reason
B12	Scores and θ as floats (R7.3 `f64`)	Integer ratios, basis points, ppm	T11 bit-for-bit replay (OD-07)
B13	`lattice_policies` table (R7.10)	`ledger.policy_versions`, materialising the `policy:` stream; `theta_version = <policy_id>@<version>`	[R §5 Phase 2]: "θ lives in a versioned `policy` table and `theta_version` is part of the vector"
B14	Episode reaper as part of Phase 0 R0.5 with `Pulse::abandon`	The same, plus `EpisodeGuard` with a closer channel; the reaper also releases holds atomically	[R §4 G07]: "via a drop guard"
B15	Missing substrate wording	Missing substrate yields `not_evaluated`, per the spec (OD-39)	[S §Verdict states] outranks the directive's shorthand
B16	`/api/v1` from WS2	From Phase 1, plus event-type versioning as the primary mechanism	[R §4 G33]

Appendix C: Rust → C++ mental model

These are the idioms this design introduces on top of LLD Appendix A.

Rust idiom in this design	C++ equivalent	Why it matters here
Private field + no public constructor (`Ledger` , `GuardedUrl` , `LlmPermit` , `ProjTx`)	Class with a private constructor and a `friend` factory	Capability by construction: if you hold the value, the check ran
`ReadDb` newtype over a pool	Strong typedef (`struct ReadDb { Pool p; }`) exposing only const methods	A read handle cannot be passed where a write handle is expected
`read_query!` / `execute_crud!` macros that `compile_error!`	`consteval` function validating a string literal, or a `static_assert` over a template string parameter	SQL kind checked at compile time
`#[must_use]` on `EpisodeGuard` , `GateRecord` , write results	`[[nodiscard]]`	Discarding is a warning, and `-D warnings` makes it an error
Consuming method `close(self)`	`&&` -qualified member function on a move-only type	After `close` , the guard no longer exists; double close does not compile
`Drop` impl as a backstop	Destructor; must not throw, so it posts to a channel instead of doing I/O	RAII close without blocking in the destructor
`!Send` marker (`PhantomData<*const ()>`) on `Tx`	A type whose use from another thread is a compile error (no direct analogue; think `thread_local` affinity enforced by the type system)	No ledger transaction can be held across an LLM `.await` on a spawned task
Trait with associated `const` and `type` (`Command` , `Aggregate`)	Concept with required static members and nested typedefs	Static dispatch; each command declares its aggregate
`enum DimState { Pass, Fail, Unsupported, NotEvaluated }` with exhaustive `match`	`enum class` + `switch` with `-Werror=switch-enum`	Closed vocabulary (T12): adding a variant breaks every non-exhaustive match

Rust idiom in this design	C++ equivalent	Why it matters here
Crate dependency graph as an architecture rule	Separate static libraries with link-time visibility; a target not linked cannot be called	Evidence cannot call verdict (T8), and handlers cannot reach `sqlx`
`SECURITY DEFINER` trigger	A setuid helper: runs with the definer's rights	The money projection is writable only through the event
`DEFERRABLE INITIALLY DEFERRED` constraint trigger	An invariant checked in the commit path (like a destructor-time `assert` that can abort the transaction)	Gap-free streams, double entry and projection justification are checked once, at commit
`xid8` snapshot fence	A sequence-lock reader that consumes only below the lowest in-flight writer ticket	Async consumers never skip a late-committing writer

---

Appendix D: DDL validation record

The DDL and SQL in §3, §6 and §9 were executed on a scratch PostgreSQL 18.6 cluster with the roles of §13.2. The scratch files were not added to the repository. Each case below was run under the stated role and gave the stated result.

#	Case	Role	Result
1	`UPDATE ledger.events`	`abw_app`	`permission denied for table events`
2	`DELETE FROM ledger.events`	`abw_app`	`permission denied`
3	`TRUNCATE ledger.events`	`abw_app`	`permission denied`
4	`UPDATE ledger.events`	`abw_owner`	`ABW-T6: UPDATE on ledger.events is forbidden` (trigger)
5	`UPDATE proj.wallets SET balance = 999`	`abw_app`	`permission denied for table wallets`
6	`UPDATE proj.wallets` with no event	`abw_owner`	`ABW-T3: proj.wallets changed in a transaction that appended no event`
7	Wallet event with `balance_after` ≠ previous + delta	`abw_app`	`ABW-T9: … chain broken at seq 3: 100 + 5 <> 9999`
8	Advance `head` without inserting events	`abw_app`	`ABW-T2: stream … advanced 2 -> 3 but 0 events were written` (at commit)
9	Hold exceeding balance	`abw_app`	`violates check constraint` on `proj.wallets` (`held <= balance`)
10	Second event with the same `idempotency_key` on a stream	`abw_app`	`duplicate key value violates unique constraint "events_money_idem"`
11	Move `head` backwards	`abw_app`	`ABW-T2: streams.… may only advance head (2 -> 1)`
12	`INSERT`	`abw_read`	`cannot execute INSERT in a read-only transaction`
13	`INSERT INTO ledger.events`	`abw_read`	`cannot execute INSERT in a read-only transaction`
14	Artifact inline bytes with a wrong SHA	`abw_app`	`violates check constraint "artifacts_check2"`
15	Artifact `hello, ABW\n`	`abw_app`	Stored as `sha256:0d4251dd…712a`, which equals V4
16	`NoStream` append, then `Exact(1)` append; projection applied by trigger	`abw_app`	`proj.wallets.balance = 100, applied_seq = 2`

#	Case	Role	Result
17	Stream row inserted in autocommit with no event	`abw_app`	Rejected by `t_streams_gap_free` at commit
18	`BEGIN READ WRITE; INSERT INTO ledger.events`	`abw_read`	`permission denied for table events`
19	Two writers, CAS on `head = 2`, READ COMMITTED	`abw_app` × 2	Writer A `cas_rows=1`, committed; writer B blocked, then `cas_rows=0`, rolled back. One event at seq 3
20	Same, REPEATABLE READ	`abw_app` × 2	Loser: `could not serialize access due to concurrent update`
21	`lock_streams` with overlapping read and write sets	`abw_app`	Two rows in `COLLATE "C"` order; the overlapping stream locked once in write mode
22	Projection insert citing an old event	`abw_app`	`ABW-T3: … cites event 1 that this transaction did not append`
23	Projection insert citing the transaction's own event	`abw_app`	Committed
24	Later `UPDATE` of that row keeping the old position	`abw_app`	`ABW-T3: … cites event 6 that this transaction did not append`
25	Fence with one open writer: later xid committed	`abw_read`	The later event's `txid` is not below `xmin`, so it is excluded until the earlier writer finishes
26	Early xid, late insert, with a third transaction open	`abw_read`	`global_pos` order ≠ `txid` order (H: pos 9, xid 1075; L: pos 10, xid 1073). SPEC 36's `global_pos > 10` fence then never returns H
27	Target fence `(txid, global_pos) > (1073, 10)`	`abw_read`	Returns H. Plan: bitmap index scan on `events_fence`
28	V1 and V3 hashes recomputed with Postgres `sha256(convert_to(…))`	`postgres`	Equal to the Python JCS implementation

#	Case	Role	Result
29	`jsonb` round trip of `{"b":1,"a":[1.0,"x"]}`; `\u0000` in `jsonb`	`postgres`	Reordered and spaced; `\u0000 cannot be converted to text`
30	`quotedFrom` edge to a judge report	`abw_app`	`violates check constraint "edges_interpretive_never_source"`
31	Same edge with `object_epistemic` falsely `evidentiary`	`abw_app`	`violates foreign key constraint "edges_object_object_epistemic_fkey"`
32	Evidentiary artifact `wasGeneratedBy` a judge activity	`abw_app`	`violates check constraint "edges_interpretive_unreachable"`
33	Episode `used` a judge report	`abw_app`	`violates check constraint "edges_interpretive_unreachable"`
34	Chain walk on the fixture: quoted, fabricated, derived (all-of), cycle, depth bound	`abw_read`	As tabulated in §6.5
35	Wallet credit of 1 000 000 with no counter-leg (a G01-style mint)	`abw_app`	`ABW-T9: transaction … moves 1000000 credits net; money is conserved`
36	Grant via issuer counter-leg; Σ of all balances	`abw_app` / `abw_read`	Issuer −100, user +100, sum 0
37	`SET ROLE abw_crud`	`abw_read`	`permission denied to set role`
38	`INSERT INTO public.teams` without `SET ROLE`	`abw_app`	`permission denied for table teams`
39	`SET LOCAL ROLE abw_crud`, then `INSERT INTO ledger.streams`	`abw_app`	`permission denied for schema ledger`
40	`SET LOCAL ROLE abw_crud` CRUD insert, `RESET ROLE`, ledger append, `COMMIT`	`abw_app`	Both committed atomically